

CIS 6270

Module 3: Score Matching

Lecture 3.4: Guidance for Continuous Generative Models

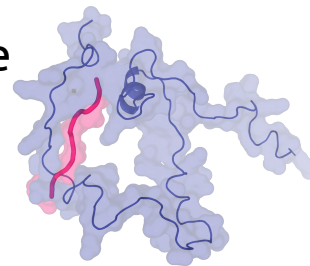
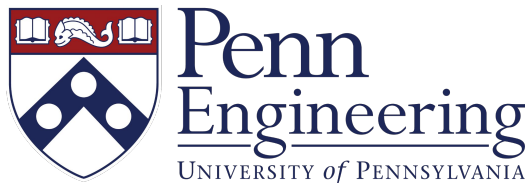
Thu, Sep 24, 2026

Pranam Chatterjee, PhD

Assistant Professor of Bioengineering

Assistant Professor of Computer and Information Science

University of Pennsylvania



Remember that our coded DDPM network did not explicitly predict the score.

It predicted the Gaussian noise ε that had been added to the clean state:

$$X_t = \alpha(t)X_0 + \sigma(t)\varepsilon$$

For this Gaussian perturbation,

$$\nabla_x \log p_t(X_t \mid X_0) = -\frac{\varepsilon}{\sigma(t)}$$

Remember that our coded DDPM network did not explicitly predict the score.

It predicted the Gaussian noise ε that had been added to the clean state:

$$X_t = \alpha(t)X_0 + \sigma(t)\varepsilon$$

For this Gaussian perturbation,

$$\nabla_x \log p_t(X_t \mid X_0) = -\frac{\varepsilon}{\sigma(t)}$$

A trained noise predictor therefore defines a score model through

$$s_\theta(X_t, t) \approx -\frac{\varepsilon_\theta(X_t, t)}{\sigma(t)}$$

Remember that our coded DDPM network did not explicitly predict the score.

It predicted the Gaussian noise ε that had been added to the clean state:

$$X_t = \alpha(t)X_0 + \sigma(t)\varepsilon$$

For this Gaussian perturbation,

$$\nabla_x \log p_t(X_t | X_0) = -\frac{\varepsilon}{\sigma(t)}$$

A trained noise predictor therefore defines a score model through

$$s_\theta(X_t, t) \approx -\frac{\varepsilon_\theta(X_t, t)}{\sigma(t)}$$

Where the learned score $s_\theta(X_t, t)$ is approximately the negative predicted noise $\varepsilon_\theta(X_t, t)$ divided by the noise standard deviation $\sigma(t)$.

Substituting this relationship into the VP probability-flow ODE gives

$$\frac{dX_t}{dt} = -\frac{1}{2}\beta(t)X_t + \frac{\beta(t)}{2\sigma(t)}\varepsilon_{\theta}(X_t, t)$$

Substituting this relationship into the VP probability-flow ODE gives

$$\frac{dX_t}{dt} = -\frac{1}{2}\beta(t)X_t + \frac{\beta(t)}{2\sigma(t)}\varepsilon_{\theta}(X_t, t)$$

Here, the probability-flow velocity combines the VP contraction $-(1/2)\beta(t)X_t$ with a denoising correction determined by the network prediction $\varepsilon_{\theta}(X_t, t)$, scaled by $\beta(t)$ divided by $2\sigma(t)$.

Substituting this relationship into the VP probability-flow ODE gives

$$\frac{dX_t}{dt} = -\frac{1}{2}\beta(t)X_t + \frac{\beta(t)}{2\sigma(t)}\varepsilon_{\theta}(X_t, t)$$

Here, the probability-flow velocity combines the VP contraction $-(1/2)\beta(t)X_t$ with a denoising correction determined by the network prediction $\varepsilon_{\theta}(X_t, t)$, scaled by $\beta(t)$ divided by $2\sigma(t)$.

The same U-Net or latent Transformer that predicted noise for the DDPM sampler can provide the score needed by the probability-flow ODE.

Substituting this relationship into the VP probability-flow ODE gives

$$\frac{dX_t}{dt} = -\frac{1}{2}\beta(t)X_t + \frac{\beta(t)}{2\sigma(t)}\varepsilon_{\theta}(X_t, t)$$

Here, the probability-flow velocity combines the VP contraction $-(1/2)\beta(t)X_t$ with a denoising correction determined by the network prediction $\varepsilon_{\theta}(X_t, t)$, scaled by $\beta(t)$ divided by $2\sigma(t)$.

The same U-Net or latent Transformer that predicted noise for the DDPM sampler can provide the score needed by the probability-flow ODE.

The sampling procedure changes from stochastic Gaussian updates to numerical ODE integration.

This leads to deterministic generation

Generation begins from the same Gaussian prior:

$$X_T \sim \mathcal{N}(0, I)$$

This leads to deterministic generation

Generation begins from the same Gaussian prior:

$$X_T \sim \mathcal{N}(0, I)$$

We then solve the probability-flow ODE backward from time T to time 0 . For a small positive step Δt ,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t(X_t)$$

This leads to deterministic generation

Generation begins from the same Gaussian prior:

$$X_T \sim \mathcal{N}(0, I)$$

We then solve the probability-flow ODE backward from time T to time 0 . For a small positive step Δt ,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t(X_t)$$

The sampling procedure is therefore:

This leads to deterministic generation

Generation begins from the same Gaussian prior:

$$X_T \sim \mathcal{N}(0, I)$$

We then solve the probability-flow ODE backward from time T to time 0 . For a small positive step Δt ,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t(X_t)$$

The sampling procedure is therefore:

1. Sample X_T from the Gaussian prior.

This leads to deterministic generation

Generation begins from the same Gaussian prior:

$$X_T \sim \mathcal{N}(0, I)$$

We then solve the probability-flow ODE backward from time T to time 0 . For a small positive step Δt ,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t(X_t)$$

The sampling procedure is therefore:

1. Sample X_T from the Gaussian prior.
2. Evaluate $\varepsilon_\theta(X_t, t)$ or $s_\theta(X_t, t)$.

This leads to deterministic generation

Generation begins from the same Gaussian prior:

$$X_T \sim \mathcal{N}(0, I)$$

We then solve the probability-flow ODE backward from time T to time 0 . For a small positive step Δt ,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t(X_t)$$

The sampling procedure is therefore:

1. Sample X_T from the Gaussian prior.
2. Evaluate $\varepsilon_\theta(X_t, t)$ or $s_\theta(X_t, t)$.
3. Construct the probability-flow velocity $v_t(X_t)$.

This leads to deterministic generation

Generation begins from the same Gaussian prior:

$$X_T \sim \mathcal{N}(0, I)$$

We then solve the probability-flow ODE backward from time T to time 0 . For a small positive step Δt ,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t(X_t)$$

The sampling procedure is therefore:

1. Sample X_T from the Gaussian prior.
2. Evaluate $\varepsilon_\theta(X_t, t)$ or $s_\theta(X_t, t)$.
3. Construct the probability-flow velocity $v_t(X_t)$.
4. Use an ODE solver to move backward toward $t = 0$.

This leads to deterministic generation

Generation begins from the same Gaussian prior:

$$X_T \sim \mathcal{N}(0, I)$$

We then solve the probability-flow ODE backward from time T to time 0 . For a small positive step Δt ,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t(X_t)$$

The sampling procedure is therefore:

1. Sample X_T from the Gaussian prior.
2. Evaluate $\varepsilon_\theta(X_t, t)$ or $s_\theta(X_t, t)$.
3. Construct the probability-flow velocity $v_t(X_t)$.
4. Use an ODE solver to move backward toward $t = 0$.
5. Return X_0 as the generated sample.

This leads to deterministic generation

Generation begins from the same Gaussian prior:

$$X_T \sim \mathcal{N}(0, I)$$

We then solve the probability-flow ODE backward from time T to time 0 . For a small positive step Δt ,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t(X_t)$$

The sampling procedure is therefore:

1. Sample X_T from the Gaussian prior.
2. Evaluate $\varepsilon_\theta(X_t, t)$ or $s_\theta(X_t, t)$.
3. Construct the probability-flow velocity $v_t(X_t)$.
4. Use an ODE solver to move backward toward $t = 0$.
5. Return X_0 as the generated sample.

Once X_T is fixed, the ODE trajectory is deterministic. Different initial draws of X_T still produce different generated samples.

Let's calculate one backward sampling step

At the current time t , take state $X_t = 1.2$, noise rate $\beta(t) = 2$, corruption standard deviation $\sigma(t) = 0.5$, and predicted noise $\varepsilon_\theta(X_t, t) = 0.8$.

$$s_\theta(X_t, t) = -\frac{\varepsilon_\theta(X_t, t)}{\sigma(t)} = -\frac{0.8}{0.5} = -1.6$$

Let's calculate one backward sampling step

At the current time t , take state $X_t = 1.2$, noise rate $\beta(t) = 2$, corruption standard deviation $\sigma(t) = 0.5$, and predicted noise $\varepsilon_\theta(X_t, t) = 0.8$.

$$s_\theta(X_t, t) = -\frac{\varepsilon_\theta(X_t, t)}{\sigma(t)} = -\frac{0.8}{0.5} = -1.6$$

The learned score s_θ at current state X_t and time t equals negative predicted noise ε_θ divided by corruption standard deviation $\sigma(t)$, giving minus 1.6.

Use that score to calculate the probability-flow velocity

Keep the current state $X_t = 1.2$, noise rate $\beta(t) = 2$, and learned score $s_\theta(X_t, t) = -1.6$.

$$v_t(X_t) = -\frac{1}{2}\beta(t) [X_t + s_\theta(X_t, t)] = -1.2 + 1.6 = 0.4$$

Use that score to calculate the probability-flow velocity

Keep the current state $X_t = 1.2$, noise rate $\beta(t) = 2$, and learned score $s_\theta(X_t, t) = -1.6$.

$$v_t(X_t) = -\frac{1}{2}\beta(t) [X_t + s_\theta(X_t, t)] = -1.2 + 1.6 = 0.4$$

The forward-time velocity v_t equals the contraction of minus 1.2 plus the score correction of 1.6, giving a net velocity of 0.4.

Use that score to calculate the probability-flow velocity

Keep the current state $X_t = 1.2$, noise rate $\beta(t) = 2$, and learned score $s_\theta(X_t, t) = -1.6$.

$$v_t(X_t) = -\frac{1}{2}\beta(t) [X_t + s_\theta(X_t, t)] = -1.2 + 1.6 = 0.4$$

The forward-time velocity v_t equals the contraction of minus 1.2 plus the score correction of 1.6, giving a net velocity of 0.4.

This velocity points toward increasing noising time. Generation takes a step toward decreasing time.

Step backward by subtracting velocity times the step size

Use a positive step size $\Delta t = 0.1$, the current state $X_t = 1.2$, and the forward-time velocity $v_t(X_t) = 0.4$.

$$X_{t-\Delta t} \approx X_t - \Delta t v_t(X_t) = 1.2 - (0.1)(0.4) = 1.16$$

Step backward by subtracting velocity times the step size

Use a positive step size $\Delta t = 0.1$, the current state $X_t = 1.2$, and the forward-time velocity $v_t(X_t) = 0.4$.

$$X_{t-\Delta t} \approx X_t - \Delta t v_t(X_t) = 1.2 - (0.1)(0.4) = 1.16$$

The estimated earlier state $X_{t-\Delta t}$ equals the current state X_t minus step size Δt times its forward-time velocity v_t , moving this sample from 1.2 to 1.16.

Step backward by subtracting velocity times the step size

Use a positive step size $\Delta t = 0.1$, the current state $X_t = 1.2$, and the forward-time velocity $v_t(X_t) = 0.4$.

$$X_{t-\Delta t} \approx X_t - \Delta t v_t(X_t) = 1.2 - (0.1)(0.4) = 1.16$$

The estimated earlier state $X_{t-\Delta t}$ equals the current state X_t minus step size Δt times its forward-time velocity v_t , moving this sample from 1.2 to 1.16.

Reevaluate the network at the new state and time before taking the next step. The approximation here comes from using a finite Euler step.

So we can now compare the two!

Property	Reverse-time SDE	Probability-flow ODE
Trajectory	Stochastic	Deterministic given X_T
Score coefficient	$g^2(t)$	$\frac{1}{2}g^2(t)$
Brownian increment	Present	Absent
Marginal densities	p_t	The same p_t with an exact score
Neural prediction	Score or noise	The same score or noise
Numerical method	SDE sampler	ODE solver

So we can now compare the two!

Property	Reverse-time SDE	Probability-flow ODE
Trajectory	Stochastic	Deterministic given X_T
Score coefficient	$g^2(t)$	$\frac{1}{2}g^2(t)$
Brownian increment	Present	Absent
Marginal densities	p_t	The same p_t with an exact score
Neural prediction	Score or noise	The same score or noise
Numerical method	SDE sampler	ODE solver

The factor of $1/2$ in the score coefficient is essential.

So we can now compare the two!

Property	Reverse-time SDE	Probability-flow ODE
Trajectory	Stochastic	Deterministic given X_T
Score coefficient	$g^2(t)$	$\frac{1}{2}g^2(t)$
Brownian increment	Present	Absent
Marginal densities	p_t	The same p_t with an exact score
Neural prediction	Score or noise	The same score or noise
Numerical method	SDE sampler	ODE solver

The factor of $1/2$ in the score coefficient is essential.

The reverse SDE uses both its score-adjusted drift and stochastic diffusion to produce the required density evolution.

So we can now compare the two!

Property	Reverse-time SDE	Probability-flow ODE
Trajectory	Stochastic	Deterministic given X_T
Score coefficient	$g^2(t)$	$\frac{1}{2}g^2(t)$
Brownian increment	Present	Absent
Marginal densities	p_t	The same p_t with an exact score
Neural prediction	Score or noise	The same score or noise
Numerical method	SDE sampler	ODE solver

The factor of $1/2$ in the score coefficient is essential.

The reverse SDE uses both its score-adjusted drift and stochastic diffusion to produce the required density evolution.

The probability-flow ODE removes the diffusion term, so its deterministic velocity absorbs exactly the amount needed to preserve the same marginals.

Because the probability-flow model is an ODE, the continuity equation also tracks how density changes along each trajectory:

$$\frac{d}{dt} \log p_t(X_t) = -\nabla_x \cdot v_t(X_t)$$

Because the probability-flow model is an ODE, the continuity equation also tracks how density changes along each trajectory:

$$\frac{d}{dt} \log p_t(X_t) = -\nabla_x \cdot v_t(X_t)$$

where the rate of change of the log-density along the trajectory X_t equals the negative divergence of the probability-flow velocity v_t evaluated at X_t .

Because the probability-flow model is an ODE, the continuity equation also tracks how density changes along each trajectory:

$$\frac{d}{dt} \log p_t(X_t) = -\nabla_x \cdot v_t(X_t)$$

Integrating from data time 0 to prior time T gives

$$\log p_0(X_0) = \log p_T(X_T) + \int_0^T \nabla_x \cdot v_t(X_t) dt$$

Because the probability-flow model is an ODE, the continuity equation also tracks how density changes along each trajectory:

$$\frac{d}{dt} \log p_t(X_t) = -\nabla_x \cdot v_t(X_t)$$

Integrating from data time 0 to prior time T gives

$$\log p_0(X_0) = \log p_T(X_T) + \int_0^T \nabla_x \cdot v_t(X_t) dt$$

So the data log-density $\log p_0(X_0)$ equals the Gaussian prior log-density $\log p_T(X_T)$ plus the accumulated divergence of the probability-flow velocity along the trajectory from time zero to time T.

Because the probability-flow model is an ODE, the continuity equation also tracks how density changes along each trajectory:

$$\frac{d}{dt} \log p_t(X_t) = -\nabla_x \cdot v_t(X_t)$$

Integrating from data time 0 to prior time T gives

$$\log p_0(X_0) = \log p_T(X_T) + \int_0^T \nabla_x \cdot v_t(X_t) dt$$

This reconnects diffusion models to Lecture 1. The accumulated divergence plays the continuous-time role of the log-determinant Jacobian used in change of variables.

Because the probability-flow model is an ODE, the continuity equation also tracks how density changes along each trajectory:

$$\frac{d}{dt} \log p_t(X_t) = -\nabla_x \cdot v_t(X_t)$$

Integrating from data time 0 to prior time T gives

$$\log p_0(X_0) = \log p_T(X_T) + \int_0^T \nabla_x \cdot v_t(X_t) dt$$

In high dimensions, we can estimate the divergence without constructing the complete Jacobian, commonly through a stochastic trace estimator.

Return to the expanding Gaussian and calculate its divergence

For the Brownian example, the probability-flow velocity is $v_t(x) = x / [2(1 + t)]$, where x is position and t is noising time.

$$\frac{\partial v_t(x)}{\partial x} = \frac{1}{2(1 + t)}$$

Return to the expanding Gaussian and calculate its divergence

For the Brownian example, the probability-flow velocity is $v_t(x) = x / [2(1 + t)]$, where x is position and t is noising time.

$$\frac{\partial v_t(x)}{\partial x} = \frac{1}{2(1 + t)}$$

The spatial derivative of the velocity v_t , which is the divergence in one dimension, equals one divided by twice one plus time t .

Return to the expanding Gaussian and calculate its divergence

For the Brownian example, the probability-flow velocity is $v_t(x) = x / [2(1 + t)]$, where x is position and t is noising time.

$$\frac{\partial v_t(x)}{\partial x} = \frac{1}{2(1 + t)}$$

The spatial derivative of the velocity v_t , which is the divergence in one dimension, equals one divided by twice one plus time t .

The positive divergence means nearby trajectories separate, so the density along each trajectory decreases.

Accumulate that expansion from time zero to time three

Integrate the one-dimensional divergence along the same trajectory. Here it depends only on time.

$$\int_0^3 \frac{\partial v_t}{\partial x}(X_t) dt = \int_0^3 \frac{dt}{2(1+t)} = \frac{1}{2} \log 4 = \log 2 \approx 0.6931$$

Accumulate that expansion from time zero to time three

Integrate the one-dimensional divergence along the same trajectory. Here it depends only on time.

$$\int_0^3 \frac{\partial v_t}{\partial x}(X_t) dt = \int_0^3 \frac{dt}{2(1+t)} = \frac{1}{2} \log 4 = \log 2 \approx 0.6931$$

The accumulated divergence along state X_t from time zero to three equals one-half the natural logarithm of four, or approximately 0.6931.

Evaluate the final Gaussian density at the transported sample

The initial sample $X_0 = 2$ moves to $X_3 = 4$. The final density p_3 is Gaussian with mean zero and variance four.

$$\log p_3(4) = -\frac{1}{2} \log(8\pi) - \frac{4^2}{8} \approx -3.6121$$

Evaluate the final Gaussian density at the transported sample

The initial sample $X_0 = 2$ moves to $X_3 = 4$. The final density p_3 is Gaussian with mean zero and variance four.

$$\log p_3(4) = -\frac{1}{2} \log(8\pi) - \frac{4^2}{8} \approx -3.6121$$

The final log-density $\log p_3$ at the transported sample four equals the Gaussian normalization term minus its squared displacement scaled by twice the variance, giving about minus 3.6121.

Recover the starting density by adding the divergence

Use the final log-density $\log p_3(4) \approx -3.6121$ and accumulated divergence $\log 2 \approx 0.6931$.

$$\log p_0(2) = \log p_3(4) + \log 2 \approx -3.6121 + 0.6931 = -2.9189$$

Recover the starting density by adding the divergence

Use the final log-density $\log p_3(4) \approx -3.6121$ and accumulated divergence $\log 2 \approx 0.6931$.

$$\log p_0(2) = \log p_3(4) + \log 2 \approx -3.6121 + 0.6931 = -2.9189$$

The starting log-density $\log p_0$ at sample two equals the final log-density at sample four plus the accumulated divergence, giving approximately minus 2.9189.

Recover the starting density by adding the divergence

Use the final log-density $\log p_3(4) \approx -3.6121$ and accumulated divergence $\log 2 \approx 0.6931$.

$$\log p_0(2) = \log p_3(4) + \log 2 \approx -3.6121 + 0.6931 = -2.9189$$

The starting log-density $\log p_0$ at sample two equals the final log-density at sample four plus the accumulated divergence, giving approximately minus 2.9189.

We recover the standard Gaussian log-density at two. Adding $\log 2$ corrects for the twofold expansion along the flow.

So recap:

So recap:

The forward SDE:

$$dX_t = f_t(X_t) dt + g(t) dW_t$$

And the probability-flow ODE

$$\frac{dX_t}{dt} = f_t(X_t) - \frac{1}{2}g^2(t)s_t(X_t)$$

So recap:

The forward SDE:

$$dX_t = f_t(X_t) dt + g(t) dW_t$$

And the probability-flow ODE

$$\boxed{\frac{dX_t}{dt} = f_t(X_t) - \frac{1}{2}g^2(t)s_t(X_t)}$$

produce the same marginal densities p_t when the score is exact.

So recap:

The forward SDE:

$$dX_t = f_t(X_t) dt + g(t) dW_t$$

And the probability-flow ODE

$$\boxed{\frac{dX_t}{dt} = f_t(X_t) - \frac{1}{2}g^2(t)s_t(X_t)}$$

produce the same marginal densities p_t when the score is exact.

The reverse-time SDE generates samples through stochastic denoising.

So recap:

The forward SDE:

$$dX_t = f_t(X_t) dt + g(t) dW_t$$

And the probability-flow ODE

$$\boxed{\frac{dX_t}{dt} = f_t(X_t) - \frac{1}{2}g^2(t)s_t(X_t)}$$

produce the same marginal densities p_t when the score is exact.

The reverse-time SDE generates samples through stochastic denoising.

The probability-flow ODE generates samples through deterministic transport.

So recap:

The forward SDE:

$$dX_t = f_t(X_t) dt + g(t) dW_t$$

And the probability-flow ODE

$$\boxed{\frac{dX_t}{dt} = f_t(X_t) - \frac{1}{2}g^2(t)s_t(X_t)}$$

produce the same marginal densities p_t when the score is exact.

The reverse-time SDE generates samples through stochastic denoising.

The probability-flow ODE generates samples through deterministic transport.

Both can use the same learned score or noise-prediction network.

For the Variance Preserving diffusion used by DDPM,

$$\frac{dX_t}{dt} = -\frac{1}{2}\beta(t) [X_t + s_t(X_t)]$$

For the Variance Preserving diffusion used by DDPM,

$$\frac{dX_t}{dt} = -\frac{1}{2}\beta(t) [X_t + s_t(X_t)]$$

and the DDPM noise predictor gives the score through

$$s_\theta(X_t, t) \approx -\frac{\varepsilon_\theta(X_t, t)}{\sigma(t)}$$

For the Variance Preserving diffusion used by DDPM,

$$\frac{dX_t}{dt} = -\frac{1}{2}\beta(t) [X_t + s_t(X_t)]$$

and the DDPM noise predictor gives the score through

$$s_\theta(X_t, t) \approx -\frac{\varepsilon_\theta(X_t, t)}{\sigma(t)}$$

The complete connection is therefore

DDPM noise predictor $\longrightarrow$ score estimate $\longrightarrow$ probability-flow velocity $\longrightarrow$ deterministic generation.

We now have two ways to generate from the same learned diffusion model.

We now have two ways to generate from the same learned diffusion model.

The reverse-time SDE follows a stochastic path, while the probability-flow ODE follows a deterministic path with the same ideal marginal densities.

We now have two ways to generate from the same learned diffusion model.

The reverse-time SDE follows a stochastic path, while the probability-flow ODE follows a deterministic path with the same ideal marginal densities.

Both methods currently use the unconditional score $s_t(x)$, which only directs samples toward regions favored by the training distribution.

We now have two ways to generate from the same learned diffusion model.

The reverse-time SDE follows a stochastic path, while the probability-flow ODE follows a deterministic path with the same ideal marginal densities.

Both methods currently use the unconditional score $s_t(x)$, which only directs samples toward regions favored by the training distribution.

But high-quality generation usually requires more control.

But high-quality generation usually requires more control.

We may want a peptide with a particular function, a molecule with a desired property, or a sample that satisfies a structural constraint.



Peptide			
Hemolysis	+	-	+
Solubility	+	+	-
Non-Fouling	-	+	+
Toxicity	-	+	+
Permeability	-7.25	-7.65	-7.06
Half-Life	5.2 hrs	6.2 hrs	4.9 hrs

Suppose c describes a desired condition.

Suppose c describes a desired condition.

It might represent an image class, protein family, binding target, molecular activity, or biological phenotype.

Suppose c describes a desired condition.

The unconditional model describes:

$$p_t(x)$$

Suppose c describes a desired condition.

The unconditional model describes:

$$p_t(x)$$

the probability density of state x at time t .

Suppose c describes a desired condition.

The unconditional model describes:

$$p_t(x)$$

the probability density of state x at time t . Controlled generation instead requires:

$$p_t(x \mid c)$$

Suppose c describes a desired condition.

The unconditional model describes:

$$p_t(x)$$

the probability density of state x at time t . Controlled generation instead requires:

$$p_t(x \mid c)$$

Where, the density of states that are both plausible and compatible with c .

Suppose c describes a desired condition.

The unconditional model describes:

$$p_t(x)$$

the probability density of state x at time t . Controlled generation instead requires:

$$p_t(x \mid c)$$

Where, the density of states that are both plausible and compatible with c .

How should the conditional score differ from the unconditional score?

Suppose c describes a desired condition.

The unconditional model describes:

$$p_t(x)$$

the probability density of state x at time t . Controlled generation instead requires:

$$p_t(x \mid c)$$

Where, the density of states that are both plausible and compatible with c .

How should the conditional score differ from the unconditional score?

Bayes' rule connects the two densities

$$p_t(x \mid c) = \frac{p_t(x)p_t(c \mid x)}{p_t(c)}$$

Bayes' rule connects the two densities

$$p_t(x \mid c) = \frac{p_t(x)p_t(c \mid x)}{p_t(c)}$$

The conditional density $p_t(x \mid c)$ equals the unconditional state density $p_t(x)$ multiplied by the probability that state x satisfies condition c , then divided by the overall probability of condition c .

Bayes' rule connects the two densities

$$p_t(x | c) = \frac{p_t(x)p_t(c | x)}{p_t(c)}$$

Take the logarithm:

$$\log p_t(x | c) = \log p_t(x) + \log p_t(c | x) - \log p_t(c)$$

Bayes' rule connects the two densities

$$p_t(x | c) = \frac{p_t(x)p_t(c | x)}{p_t(c)}$$

Take the logarithm:

$$\log p_t(x | c) = \log p_t(x) + \log p_t(c | x) - \log p_t(c)$$

The generative dynamics need a direction in state space, so we differentiate with respect to x :

$$\nabla_x \log p_t(x | c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c | x)$$

Bayes' rule connects the two densities

$$p_t(x \mid c) = \frac{p_t(x)p_t(c \mid x)}{p_t(c)}$$

Take the logarithm:

$$\log p_t(x \mid c) = \log p_t(x) + \log p_t(c \mid x) - \log p_t(c)$$

The generative dynamics need a direction in state space, so we differentiate with respect to x :

$$\nabla_x \log p_t(x \mid c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c \mid x)$$

The term $\log p_t(c)$ disappears because it does not depend on x .

Using the definitions:

$$s_t(x \mid c) = \nabla_x \log p_t(x \mid c)$$

Bayes' rule connects the two densities

$$p_t(x | c) = \frac{p_t(x)p_t(c | x)}{p_t(c)}$$

Take the logarithm:

$$\log p_t(x | c) = \log p_t(x) + \log p_t(c | x) - \log p_t(c)$$

The generative dynamics need a direction in state space, so we differentiate with respect to x :

$$\nabla_x \log p_t(x | c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c | x)$$

The term $\log p_t(c)$ disappears because it does not depend on x .

Using the definitions:

$$s_t(x | c) = \nabla_x \log p_t(x | c) \quad s_t(x) = \nabla_x \log p_t(x)$$

Bayes' rule connects the two densities

$$p_t(x | c) = \frac{p_t(x)p_t(c | x)}{p_t(c)}$$

Take the logarithm:

$$\log p_t(x | c) = \log p_t(x) + \log p_t(c | x) - \log p_t(c)$$

The generative dynamics need a direction in state space, so we differentiate with respect to x :

$$\nabla_x \log p_t(x | c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c | x)$$

The term $\log p_t(c)$ disappears because it does not depend on x .

We get:

$$s_t(x | c) = s_t(x) + \nabla_x \log p_t(c | x)$$

Bayes' rule connects the two densities

$$p_t(x | c) = \frac{p_t(x)p_t(c | x)}{p_t(c)}$$

Take the logarithm:

$$\log p_t(x | c) = \log p_t(x) + \log p_t(c | x) - \log p_t(c)$$

The generative dynamics need a direction in state space, so we differentiate with respect to x :

$$\nabla_x \log p_t(x | c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c | x)$$

The term $\log p_t(c)$ disappears because it does not depend on x .

We get:

$$s_t(x | c) = s_t(x) + \nabla_x \log p_t(c | x)$$

The conditional score $s_t(x | c)$ equals the unconditional score $s_t(x)$, which points toward states supported by the data distribution, plus the gradient of $\log p_t(c | x)$, which points toward states that are increasingly compatible with condition c .

The unconditional score keeps the trajectory near regions supported by the training data.

Take the logarithm:

$$\log p_t(x \mid c) = \log p_t(x) + \log p_t(c \mid x) - \log p_t(c)$$

The generative dynamics need a direction in state space, so we differentiate with respect to x :

$$\nabla_x \log p_t(x \mid c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c \mid x)$$

The term $\log p_t(c)$ disappears because it does not depend on x .

We get:

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x)$$

The conditional score $s_t(x \mid c)$ equals the unconditional score $s_t(x)$, which points toward states supported by the data distribution, plus the gradient of $\log p_t(c \mid x)$, which points toward states that are increasingly compatible with condition c .

The unconditional score keeps the trajectory near regions supported by the training data.

Take the logarithm:

$$\log p_t(x \mid c) = \log p_t(x) + \log p_t(c \mid x) - \log p_t(c)$$

The generative dynamics need a direction in state space, so we differentiate with respect to x :

$$\nabla_x \log p_t(x \mid c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c \mid x)$$

The term $\log p_t(c)$ disappears because it does not depend on x .

We get:

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x)$$

The conditional score $s_t(x \mid c)$ equals the unconditional score $s_t(x)$, which points toward states supported by the data distribution, plus the gradient of $\log p_t(c \mid x)$, which points toward states that are increasingly compatible with condition c .

The classifier gradient changes which region of that data distribution the trajectory approaches.

Take the logarithm:

$$\log p_t(x \mid c) = \log p_t(x) + \log p_t(c \mid x) - \log p_t(c)$$

The generative dynamics need a direction in state space, so we differentiate with respect to x :

$$\nabla_x \log p_t(x \mid c) = \nabla_x \log p_t(x) + \nabla_x \log p_t(c \mid x)$$

The term $\log p_t(c)$ disappears because it does not depend on x .

We get:

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x)$$

The conditional score $s_t(x \mid c)$ equals the unconditional score $s_t(x)$, which points toward states supported by the data distribution, plus the gradient of $\log p_t(c \mid x)$, which points toward states that are increasingly compatible with condition c .

In practice, we may want to strengthen or weaken the condition.

In practice, we may want to strengthen or weaken the condition.

So we introduce a guidance scale γ :

$$s_{\text{guided}}(X_t, t, c) = s_{\theta}(X_t, t) + \gamma \nabla_{X_t} \log p_{\phi}(c \mid X_t, t)$$

In practice, we may want to strengthen or weaken the condition.

So we introduce a guidance scale γ :

$$s_{\text{guided}}(X_t, t, c) = s_{\theta}(X_t, t) + \gamma \nabla_{X_t} \log p_{\phi}(c \mid X_t, t)$$

The guided score at state X_t equals the generative model's unconditional score $s_{\theta}(X_t, t)$ plus the classifier gradient of $\log p_{\phi}(c \mid X_t, t)$, now multiplied by the guidance strength γ .

In practice, we may want to strengthen or weaken the condition.

So we introduce a guidance scale γ :

$$s_{\text{guided}}(X_t, t, c) = s_{\theta}(X_t, t) + \gamma \nabla_{X_t} \log p_{\phi}(c \mid X_t, t)$$

The guided score at state X_t equals the generative model's unconditional score $s_{\theta}(X_t, t)$ plus the classifier gradient of $\log p_{\phi}(c \mid X_t, t)$, now multiplied by the guidance strength γ .

- When $\gamma = 0$, generation is unconditional.

In practice, we may want to strengthen or weaken the condition.

So we introduce a guidance scale γ :

$$s_{\text{guided}}(X_t, t, c) = s_{\theta}(X_t, t) + \gamma \nabla_{X_t} \log p_{\phi}(c \mid X_t, t)$$

The guided score at state X_t equals the generative model's unconditional score $s_{\theta}(X_t, t)$ plus the classifier gradient of $\log p_{\phi}(c \mid X_t, t)$, now multiplied by the guidance strength γ .

- When $\gamma = 0$, generation is unconditional.
- When $\gamma = 1$ and both models are exact, the expression recovers the ordinary conditional score.

In practice, we may want to strengthen or weaken the condition.

So we introduce a guidance scale γ :

$$s_{\text{guided}}(X_t, t, c) = s_{\theta}(X_t, t) + \gamma \nabla_{X_t} \log p_{\phi}(c \mid X_t, t)$$

The guided score at state X_t equals the generative model's unconditional score $s_{\theta}(X_t, t)$ plus the classifier gradient of $\log p_{\phi}(c \mid X_t, t)$, now multiplied by the guidance strength γ .

- When $\gamma = 0$, generation is unconditional.
- When $\gamma = 1$ and both models are exact, the expression recovers the ordinary conditional score.
- When $\gamma > 1$, generation places greater emphasis on the condition.

The resulting target density is

$$p_{t,\gamma}(x \mid c) \propto p_t(x)p_t(c \mid x)^\gamma$$

The resulting target density is

$$p_{t,\gamma}(x \mid c) \propto p_t(x)p_t(c \mid x)^\gamma$$

Larger γ concentrates probability around states that receive high classifier probability.

The resulting target density is

$$p_{t,\gamma}(x \mid c) \propto p_t(x)p_t(c \mid x)^\gamma$$

Larger γ concentrates probability around states that receive high classifier probability.
BUT, excessive guidance can reduce diversity and may exploit errors in the classifier.

Suppose that, at one state:

$$s_t(x) = -0.4$$

and

$$\nabla_x \log p_t(c \mid x) = 0.7$$

Suppose that, at one state:

$$s_t(x) = -0.4$$

And

$$\nabla_x \log p_t(c \mid x) = 0.7$$

With $\gamma = 1$,

$$s_{\text{guided}} = -0.4 + 1(0.7) = 0.3$$

Suppose that, at one state:

$$s_t(x) = -0.4$$

And

$$\nabla_x \log p_t(c \mid x) = 0.7$$

With $\gamma = 1$,

$$s_{\text{guided}} = -0.4 + 1(0.7) = 0.3$$

The unconditional score points in the negative direction, while the condition favors the positive direction. Their sum points slightly in the positive direction.

Suppose that, at one state:

$$s_t(x) = -0.4$$

And

$$\nabla_x \log p_t(c \mid x) = 0.7$$

With $\gamma = 1$,

$$s_{\text{guided}} = -0.4 + 1(0.7) = 0.3$$

The unconditional score points in the negative direction, while the condition favors the positive direction. Their sum points slightly in the positive direction.

If $\gamma = 2$,

Suppose that, at one state:

$$s_t(x) = -0.4$$

And

$$\nabla_x \log p_t(c \mid x) = 0.7$$

With $\gamma = 1$,

$$s_{\text{guided}} = -0.4 + 1(0.7) = 0.3$$

The unconditional score points in the negative direction, while the condition favors the positive direction. Their sum points slightly in the positive direction.

If $\gamma = 2$,

$$s_{\text{guided}} = -0.4 + 2(0.7) = 1.0$$

Suppose that, at one state:

$$s_t(x) = -0.4$$

And

$$\nabla_x \log p_t(c \mid x) = 0.7$$

With $\gamma = 1$,

$$s_{\text{guided}} = -0.4 + 1(0.7) = 0.3$$

The unconditional score points in the negative direction, while the condition favors the positive direction. Their sum points slightly in the positive direction.

If $\gamma = 2$,

$$s_{\text{guided}} = -0.4 + 2(0.7) = 1.0$$

Increasing γ strengthens movement toward the condition enough to dominate the unconditional direction.

A classifier is one way to define desirability

A classifier is one way to define desirability

Continuous properties such as binding, solubility, stability, or activity may instead be represented by a reward $R_t(x)$.

A classifier is one way to define desirability

Continuous properties such as binding, solubility, stability, or activity may instead be represented by a reward $R_t(x)$.

To turn that reward into a probability distribution, we assign high-reward states greater probability:

$$p_t^{(R)}(x) = \frac{p_t(x) \exp [\eta R_t(x)]}{Z_t}$$

A classifier is one way to define desirability

Continuous properties such as binding, solubility, stability, or activity may instead be represented by a reward $R_t(x)$.

To turn that reward into a probability distribution, we assign high-reward states greater probability:

$$p_t^{(R)}(x) = \frac{p_t(x) \exp[\eta R_t(x)]}{Z_t}$$

The reward-guided density $p_t^{(R)}(x)$ equals the original density $p_t(x)$ multiplied by the positive reward weight $\exp[\eta R_t(x)]$, then divided by the normalizing constant Z_t , where η controls the overall reward strength.

A classifier is one way to define desirability

Continuous properties such as binding, solubility, stability, or activity may instead be represented by a reward $R_t(x)$.

To turn that reward into a probability distribution, we assign high-reward states greater probability:

$$p_t^{(R)}(x) = \frac{p_t(x) \exp [\eta R_t(x)]}{Z_t}$$

Taking the gradient of its log-density gives

$$\nabla_x \log p_t^{(R)}(x) = \nabla_x \log p_t(x) + \eta \nabla_x R_t(x)$$

A classifier is one way to define desirability

Continuous properties such as binding, solubility, stability, or activity may instead be represented by a reward $R_t(x)$.

To turn that reward into a probability distribution, we assign high-reward states greater probability:

$$p_t^{(R)}(x) = \frac{p_t(x) \exp[\eta R_t(x)]}{Z_t}$$

Taking the gradient of its log-density gives

$$\nabla_x \log p_t^{(R)}(x) = \nabla_x \log p_t(x) + \eta \nabla_x R_t(x)$$

because $\log Z_t$ does not depend on x .

A classifier is one way to define desirability

Continuous properties such as binding, solubility, stability, or activity may instead be represented by a reward $R_t(x)$.

To turn that reward into a probability distribution, we assign high-reward states greater probability:

$$p_t^{(R)}(x) = \frac{p_t(x) \exp [\eta R_t(x)]}{Z_t}$$

Taking the gradient of its log-density gives

$$\nabla_x \log p_t^{(R)}(x) = \nabla_x \log p_t(x) + \eta \nabla_x R_t(x)$$

because $\log Z_t$ does not depend on x . Therefore,

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

A classifier is one way to define desirability

Continuous properties such as binding, solubility, stability, or activity may instead be represented by a reward $R_t(x)$.

To turn that reward into a probability distribution, we assign high-reward states greater probability:

$$p_t^{(R)}(x) = \frac{p_t(x) \exp[\eta R_t(x)]}{Z_t}$$

Taking the gradient of its log-density gives

$$\nabla_x \log p_t^{(R)}(x) = \nabla_x \log p_t(x) + \eta \nabla_x R_t(x)$$

because $\log Z_t$ does not depend on x . Therefore,

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

The reward-guided score $s_t^{(R)}(x)$ equals the unconditional score $s_t(x)$ plus the reward gradient $\nabla_x R_t(x)$, multiplied by the guidance strength η .

A classifier is one way to define desirability

Continuous properties such as binding, solubility, stability, or activity may instead be represented by a reward $R_t(x)$.

To turn that reward into a probability distribution, we assign high-reward states greater probability:

$$p_t^{(R)}(x) = \frac{p_t(x) \exp[\eta R_t(x)]}{Z_t}$$

Taking the gradient of its log-density gives

$$\nabla_x \log p_t^{(R)}(x) = \nabla_x \log p_t(x) + \eta \nabla_x R_t(x)$$

because $\log Z_t$ does not depend on x . Therefore,

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

Classifier guidance is the special case

$$R_t(x) = \log p_t(c \mid x)$$

We now know how guidance changes the score.

We now know how guidance changes the score.

The next step is to place that score inside the generative dynamics.

Recall that the unconditional reverse-time SDE is:

$$dX_t = [f_t(X_t) - g^2(t)s_\theta(X_t, t)] dt + g(t) d\bar{W}_t$$

Recall that the unconditional reverse-time SDE is:

$$dX_t = [f_t(X_t) - g^2(t)s_\theta(X_t, t)] dt + g(t) d\bar{W}_t$$

The score is the only term that tells the reverse process where the data density is concentrated.

Recall that the unconditional reverse-time SDE is:

$$dX_t = [f_t(X_t) - g^2(t)s_\theta(X_t, t)] dt + g(t) d\bar{W}_t$$

The score is the only term that tells the reverse process where the data density is concentrated. We therefore guide the process by replacing the unconditional score with the guided score:

$$dX_t = [f_t(X_t) - g^2(t) (s_\theta(X_t, t) + \gamma \nabla_{X_t} \log p_\phi(c \mid X_t, t))] dt + g(t) d\bar{W}_t$$

Recall that the unconditional reverse-time SDE is:

$$dX_t = [f_t(X_t) - g^2(t)s_\theta(X_t, t)] dt + g(t) d\bar{W}_t$$

The score is the only term that tells the reverse process where the data density is concentrated. We therefore guide the process by replacing the unconditional score with the guided score:

$$dX_t = [f_t(X_t) - g^2(t) (s_\theta(X_t, t) + \gamma \nabla_{X_t} \log p_\phi(c | X_t, t))] dt + g(t) d\bar{W}_t$$

So here, the classifier-guided reverse SDE combines the original forward drift $f_t(X_t)$, the learned score $s_\theta(X_t, t)$, and the classifier gradient for condition c inside the reverse drift, while retaining a Brownian increment $d\bar{W}_t$ scaled by $g(t)$.

Recall that the unconditional reverse-time SDE is:

$$dX_t = [f_t(X_t) - g^2(t)s_\theta(X_t, t)] dt + g(t) d\bar{W}_t$$

The score is the only term that tells the reverse process where the data density is concentrated. We therefore guide the process by replacing the unconditional score with the guided score:

$$dX_t = [f_t(X_t) - g^2(t) (s_\theta(X_t, t) + \gamma \nabla_{X_t} \log p_\phi(c | X_t, t))] dt + g(t) d\bar{W}_t$$

The learned score performs denoising toward the data distribution. The classifier gradient changes the destination toward the part of that distribution associated with c . And the Brownian increment preserves stochastic variation between trajectories.

Recall that the unconditional reverse-time SDE is:

$$dX_t = [f_t(X_t) - g^2(t)s_\theta(X_t, t)] dt + g(t) d\bar{W}_t$$

The score is the only term that tells the reverse process where the data density is concentrated. We therefore guide the process by replacing the unconditional score with the guided score:

$$dX_t = [f_t(X_t) - g^2(t) (s_\theta(X_t, t) + \gamma \nabla_{X_t} \log p_\phi(c | X_t, t))] dt + g(t) d\bar{W}_t$$

The learned score performs denoising toward the data distribution. The classifier gradient changes the destination toward the part of that distribution associated with c . And the Brownian increment preserves stochastic variation between trajectories.

Diffusion generation runs from T toward zero, so dt is negative. The negative classifier term inside the reverse drift therefore produces a backward update in the positive classifier-gradient direction.

But the DDPM predicts noise instead of predicting the score directly:

$$s_{\theta}(X_t, t) \approx -\frac{\varepsilon_{\theta}(X_t, t)}{\sigma(t)}$$

But the DDPM predicts noise instead of predicting the score directly:

$$s_{\theta}(X_t, t) \approx -\frac{\varepsilon_{\theta}(X_t, t)}{\sigma(t)}$$

The guided score must also satisfy

$$s_{\text{guided}}(X_t, t, c) = -\frac{\varepsilon_{\text{guided}}(X_t, t, c)}{\sigma(t)}$$

But the DDPM predicts noise instead of predicting the score directly:

$$s_{\theta}(X_t, t) \approx -\frac{\varepsilon_{\theta}(X_t, t)}{\sigma(t)}$$

The guided score must also satisfy

$$s_{\text{guided}}(X_t, t, c) = -\frac{\varepsilon_{\text{guided}}(X_t, t, c)}{\sigma(t)}$$

Substitute the classifier-guided score:

$$-\frac{\varepsilon_{\text{guided}}}{\sigma(t)} = -\frac{\varepsilon_{\theta}}{\sigma(t)} + \gamma \nabla_{X_t} \log p_{\phi}(c \mid X_t, t)$$

But the DDPM predicts noise instead of predicting the score directly:

$$s_{\theta}(X_t, t) \approx -\frac{\varepsilon_{\theta}(X_t, t)}{\sigma(t)}$$

The guided score must also satisfy

$$s_{\text{guided}}(X_t, t, c) = -\frac{\varepsilon_{\text{guided}}(X_t, t, c)}{\sigma(t)}$$

Substitute the classifier-guided score:

$$-\frac{\varepsilon_{\text{guided}}}{\sigma(t)} = -\frac{\varepsilon_{\theta}}{\sigma(t)} + \gamma \nabla_{X_t} \log p_{\phi}(c \mid X_t, t)$$

But the DDPM predicts noise instead of predicting the score directly:

$$s_{\theta}(X_t, t) \approx -\frac{\varepsilon_{\theta}(X_t, t)}{\sigma(t)}$$

The guided score must also satisfy

$$s_{\text{guided}}(X_t, t, c) = -\frac{\varepsilon_{\text{guided}}(X_t, t, c)}{\sigma(t)}$$

Substitute the classifier-guided score:

$$-\frac{\varepsilon_{\text{guided}}}{\sigma(t)} = -\frac{\varepsilon_{\theta}}{\sigma(t)} + \gamma \nabla_{X_t} \log p_{\phi}(c \mid X_t, t)$$

Multiply both sides by $-\sigma(t)$:

$$\boxed{\varepsilon_{\text{guided}}(X_t, t, c) = \varepsilon_{\theta}(X_t, t) - \gamma \sigma(t) \nabla_{X_t} \log p_{\phi}(c \mid X_t, t)}$$

But the DDPM predicts noise instead of predicting the score directly:

$$s_{\theta}(X_t, t) \approx -\frac{\varepsilon_{\theta}(X_t, t)}{\sigma(t)}$$

The guided score must also satisfy

$$s_{\text{guided}}(X_t, t, c) = -\frac{\varepsilon_{\text{guided}}(X_t, t, c)}{\sigma(t)}$$

Substitute the classifier-guided score:

$$-\frac{\varepsilon_{\text{guided}}}{\sigma(t)} = -\frac{\varepsilon_{\theta}}{\sigma(t)} + \gamma \nabla_{X_t} \log p_{\phi}(c \mid X_t, t)$$

Multiply both sides by $-\sigma(t)$:

$$\boxed{\varepsilon_{\text{guided}}(X_t, t, c) = \varepsilon_{\theta}(X_t, t) - \gamma \sigma(t) \nabla_{X_t} \log p_{\phi}(c \mid X_t, t)}$$

The guided noise prediction $\varepsilon_{\text{guided}}(X_t, t, c)$ equals the original predicted noise $\varepsilon_{\theta}(X_t, t)$ minus the classifier gradient multiplied by the current noise standard deviation $\sigma(t)$ and guidance strength γ .

This equation connects directly to the DDPM sampler:

1. Predict ε_θ from X_t .

$$\varepsilon_{\text{guided}}(X_t, t, c) = \varepsilon_\theta(X_t, t) - \gamma \sigma(t) \nabla_{X_t} \log p_\phi(c \mid X_t, t)$$

This equation connects directly to the DDPM sampler:

1. Predict ε_θ from X_t .
2. Evaluate the classifier on X_t .

$$\varepsilon_{\text{guided}}(X_t, t, c) = \varepsilon_\theta(X_t, t) - \gamma \sigma(t) \nabla_{X_t} \log p_\phi(c \mid X_t, t)$$

This equation connects directly to the DDPM sampler:

1. Predict ε_θ from X_t .
2. Evaluate the classifier on X_t .
3. Differentiate the classifier log-probability with respect to X_t .

$$\varepsilon_{\text{guided}}(X_t, t, c) = \varepsilon_\theta(X_t, t) - \gamma \sigma(t) \nabla_{X_t} \log p_\phi(c \mid X_t, t)$$

This equation connects directly to the DDPM sampler:

1. Predict ε_θ from X_t .
2. Evaluate the classifier on X_t .
3. Differentiate the classifier log-probability with respect to X_t .
4. Construct $\varepsilon_{\text{guided}}$.

$$\varepsilon_{\text{guided}}(X_t, t, c) = \varepsilon_\theta(X_t, t) - \gamma \sigma(t) \nabla_{X_t} \log p_\phi(c \mid X_t, t)$$

This equation connects directly to the DDPM sampler:

1. Predict ε_θ from X_t .
2. Evaluate the classifier on X_t .
3. Differentiate the classifier log-probability with respect to X_t .
4. Construct $\varepsilon_{\text{guided}}$.
5. Use $\varepsilon_{\text{guided}}$ in the existing reverse-mean equation.

$$\varepsilon_{\text{guided}}(X_t, t, c) = \varepsilon_\theta(X_t, t) - \gamma \sigma(t) \nabla_{X_t} \log p_\phi(c \mid X_t, t)$$

This equation connects directly to the DDPM sampler:

1. Predict ε_θ from X_t .
2. Evaluate the classifier on X_t .
3. Differentiate the classifier log-probability with respect to X_t .
4. Construct $\varepsilon_{\text{guided}}$.
5. Use $\varepsilon_{\text{guided}}$ in the existing reverse-mean equation.

$$\varepsilon_{\text{guided}}(X_t, t, c) = \varepsilon_\theta(X_t, t) - \gamma \sigma(t) \nabla_{X_t} \log p_\phi(c \mid X_t, t)$$

The reverse sampler itself does not need to be redesigned. Guidance changes the neural prediction inserted into it.

The same guided score produces a deterministic flow

The same guided score produces a deterministic flow

The probability-flow velocity is

$$v_t(x) = f_t(x) - \frac{1}{2}g^2(t)s_t(x)$$

The same guided score produces a deterministic flow

The probability-flow velocity is

$$v_t(x) = f_t(x) - \frac{1}{2}g^2(t)s_t(x)$$

Replace $s_t(x)$ with the conditional score:

$$v_t^c(x) = f_t(x) - \frac{1}{2}g^2(t) [s_t(x) + \gamma \nabla_x \log p_t(c | x)]$$

The same guided score produces a deterministic flow

The probability-flow velocity is

$$v_t(x) = f_t(x) - \frac{1}{2}g^2(t)s_t(x)$$

Replace $s_t(x)$ with the conditional score:

$$v_t^c(x) = f_t(x) - \frac{1}{2}g^2(t)[s_t(x) + \gamma \nabla_x \log p_t(c | x)]$$

Separating the unconditional velocity gives

$$v_t^c(x) = v_t(x) - \frac{\gamma}{2}g^2(t)\nabla_x \log p_t(c | x)$$

The same guided score produces a deterministic flow

The probability-flow velocity is

$$v_t(x) = f_t(x) - \frac{1}{2}g^2(t)s_t(x)$$

Replace $s_t(x)$ with the conditional score:

$$v_t^c(x) = f_t(x) - \frac{1}{2}g^2(t)[s_t(x) + \gamma \nabla_x \log p_t(c | x)]$$

Separating the unconditional velocity gives

$$v_t^c(x) = v_t(x) - \frac{\gamma}{2}g^2(t)\nabla_x \log p_t(c | x)$$

Here, the guided probability-flow velocity $v_t^c(x)$ equals the unconditional velocity $v_t(x)$ minus one-half of $\gamma g^2(t)$ multiplied by the classifier gradient, where the factor one-half converts the score correction into the deterministic probability-flow correction.

The same guided score produces a deterministic flow

The probability-flow velocity is

$$v_t(x) = f_t(x) - \frac{1}{2}g^2(t)s_t(x)$$

Replace $s_t(x)$ with the conditional score:

$$v_t^c(x) = f_t(x) - \frac{1}{2}g^2(t)[s_t(x) + \gamma \nabla_x \log p_t(c | x)]$$

Separating the unconditional velocity gives

$$v_t^c(x) = v_t(x) - \frac{\gamma}{2}g^2(t)\nabla_x \log p_t(c | x)$$

The reverse SDE uses the complete factor $g^2(t)$ because it retains stochastic diffusion. The probability-flow ODE uses one-half of that factor because its deterministic velocity accounts for the corresponding density evolution without Brownian noise.

Let's do a small backward-step example

Let's do a small backward-step example, where:

$$v_t(x) = 0.5, \quad g^2(t) = 1, \quad \gamma = 2, \quad \nabla_x \log p_t(c \mid x) = 0.8$$

Let's do a small backward-step example, where:

$$v_t(x) = 0.5, \quad g^2(t) = 1, \quad \gamma = 2, \quad \nabla_x \log p_t(c \mid x) = 0.8$$

Then

$$v_t^c(x) = 0.5 - \frac{2}{2}(1)(0.8) = -0.3$$

Let's do a small backward-step example, where:

$$v_t(x) = 0.5, \quad g^2(t) = 1, \quad \gamma = 2, \quad \nabla_x \log p_t(c \mid x) = 0.8$$

Then

$$v_t^c(x) = 0.5 - \frac{2}{2}(1)(0.8) = -0.3$$

For a backward step with $\Delta t = 0.1$,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t^c(X_t) = X_t + 0.03$$

Let's do a small backward-step example, where:

$$v_t(x) = 0.5, \quad g^2(t) = 1, \quad \gamma = 2, \quad \nabla_x \log p_t(c \mid x) = 0.8$$

Then

$$v_t^c(x) = 0.5 - \frac{2}{2}(1)(0.8) = -0.3$$

For a backward step with $\Delta t = 0.1$,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t^c(X_t) = X_t + 0.03$$

The probability-flow velocity contains a negative classifier term, and backward integration converts it into positive movement along the classifier gradient.

Let's do a small backward-step example, where:

$$v_t(x) = 0.5, \quad g^2(t) = 1, \quad \gamma = 2, \quad \nabla_x \log p_t(c | x) = 0.8$$

Then

$$v_t^c(x) = 0.5 - \frac{2}{2}(1)(0.8) = -0.3$$

For a backward step with $\Delta t = 0.1$,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t^c(X_t) = X_t + 0.03$$

The probability-flow velocity contains a negative classifier term, and backward integration converts it into positive movement along the classifier gradient.

Diffusion guidance is therefore understood in both its stochastic and deterministic forms.

Let's do a small backward-step example, where:

$$v_t(x) = 0.5, \quad g^2(t) = 1, \quad \gamma = 2, \quad \nabla_x \log p_t(c | x) = 0.8$$

Then

$$v_t^c(x) = 0.5 - \frac{2}{2}(1)(0.8) = -0.3$$

For a backward step with $\Delta t = 0.1$,

$$X_{t-\Delta t} \approx X_t - \Delta t v_t^c(X_t) = X_t + 0.03$$

The probability-flow velocity contains a negative classifier term, and backward integration converts it into positive movement along the classifier gradient.

Diffusion guidance is therefore understood in both its stochastic and deterministic forms. We can now connect the same reasoning to a flow-matching model that predicts velocity directly.

A conditional flow-matching model predicts

$$v_{\theta}(X_t, t, c)$$

During training, the condition accompanies the endpoint data, so the model learns the velocity associated with the conditional probability path.

A conditional flow-matching model predicts

$$v_{\theta}(X_t, t, c)$$

During training, the condition accompanies the endpoint data, so the model learns the velocity associated with the conditional probability path.

Sampling then follows

$$\frac{dX_t}{dt} = v_{\theta}(X_t, t, c)$$

A conditional flow-matching model predicts

$$v_{\theta}(X_t, t, c)$$

During training, the condition accompanies the endpoint data, so the model learns the velocity associated with the conditional probability path.

Sampling then follows

$$\frac{dX_t}{dt} = v_{\theta}(X_t, t, c)$$

This gives the direction and speed required to transport the current state toward data associated with condition c .

A conditional flow-matching model predicts

$$v_{\theta}(X_t, t, c)$$

During training, the condition accompanies the endpoint data, so the model learns the velocity associated with the conditional probability path.

Sampling then follows

$$\frac{dX_t}{dt} = v_{\theta}(X_t, t, c)$$

This gives the direction and speed required to transport the current state toward data associated with condition c .

Diffusion classifier guidance begins with an unconditional score and adds a conditional correction. Conditional flow matching can learn the complete conditional velocity directly.

A conditional flow-matching model predicts

$$v_{\theta}(X_t, t, c)$$

During training, the condition accompanies the endpoint data, so the model learns the velocity associated with the conditional probability path.

Sampling then follows

$$\frac{dX_t}{dt} = v_{\theta}(X_t, t, c)$$

This gives the direction and speed required to transport the current state toward data associated with condition c .

Diffusion classifier guidance begins with an unconditional score and adds a conditional correction. Conditional flow matching can learn the complete conditional velocity directly.

The condition may be a label, property vector, target embedding, protein representation, or another input available during training.

We can add a differentiable reward to an existing flow

We can add a differentiable reward to an existing flow

Suppose we already have an unconditional flow and a differentiable reward $R_t(X_t)$. For the common flow-matching convention, generation moves forward from a prior at $t = 0$ to data at $t = 1$.

We can add a differentiable reward to an existing flow

Suppose we already have an unconditional flow and a differentiable reward $R_t(X_t)$. For the common flow-matching convention, generation moves forward from a prior at $t = 0$ to data at $t = 1$.

We can bend the learned trajectory toward increasing reward:

$$v_{\text{guided}}(X_t, t) = v_{\theta}(X_t, t) + \kappa(t) \nabla_{X_t} R_t(X_t)$$

We can add a differentiable reward to an existing flow

Suppose we already have an unconditional flow and a differentiable reward $R_t(X_t)$. For the common flow-matching convention, generation moves forward from a prior at $t = 0$ to data at $t = 1$.

We can bend the learned trajectory toward increasing reward:

$$v_{\text{guided}}(X_t, t) = v_{\theta}(X_t, t) + \kappa(t) \nabla_{X_t} R_t(X_t)$$

Where, The guided flow velocity equals the learned velocity $v_{\theta}(X_t, t)$, which follows the generative probability path, plus the reward gradient $\nabla_{X_t} R_t(X_t)$, which points toward increasing reward, scaled by the time-dependent strength $\kappa(t)$.

We can add a differentiable reward to an existing flow

Suppose we already have an unconditional flow and a differentiable reward $R_t(X_t)$. For the common flow-matching convention, generation moves forward from a prior at $t = 0$ to data at $t = 1$.

We can bend the learned trajectory toward increasing reward:

$$v_{\text{guided}}(X_t, t) = v_{\theta}(X_t, t) + \kappa(t) \nabla_{X_t} R_t(X_t)$$

Where, The guided flow velocity equals the learned velocity $v_{\theta}(X_t, t)$, which follows the generative probability path, plus the reward gradient $\nabla_{X_t} R_t(X_t)$, which points toward increasing reward, scaled by the time-dependent strength $\kappa(t)$.

So here, what is derived and what is chosen?

$$v_{\text{guided}}(X_t, t) = v_{\theta}(X_t, t) + \kappa(t) \nabla_{X_t} R_t(X_t)$$

So here, what is derived and what is chosen?

$$v_{\text{guided}}(X_t, t) = v_{\theta}(X_t, t) + \kappa(t) \nabla_{X_t} R_t(X_t)$$

For a probability-flow ODE derived from diffusion, the exact score-to-velocity conversion gives $-(1/2)g^2(t)$.

So here, what is derived and what is chosen?

$$v_{\text{guided}}(X_t, t) = v_{\theta}(X_t, t) + \kappa(t) \nabla_{X_t} R_t(X_t)$$

For a probability-flow ODE derived from diffusion, the exact score-to-velocity conversion gives $-(1/2)g^2(t)$.

For a general flow-matching path, there is no diffusion coefficient $g(t)$, so the function $\kappa(t)$ is therefore a guidance design choice that determines how strongly the reward bends the learned flow.

So here, what is derived and what is chosen?

$$v_{\text{guided}}(X_t, t) = v_{\theta}(X_t, t) + \kappa(t) \nabla_{X_t} R_t(X_t)$$

For a probability-flow ODE derived from diffusion, the exact score-to-velocity conversion gives $-(1/2)g^2(t)$.

For a general flow-matching path, there is no diffusion coefficient $g(t)$, so the function $\kappa(t)$ is therefore a guidance design choice that determines how strongly the reward bends the learned flow.

This matters because the two models may use opposite time conventions:

- Diffusion commonly places data at $t = 0$ and noise at $t = T$, so generation runs backward.
- Flow matching commonly places the prior at $t = 0$ and data at $t = 1$, so generation runs forward.

So here, what is derived and what is chosen?

$$v_{\text{guided}}(X_t, t) = v_{\theta}(X_t, t) + \kappa(t) \nabla_{X_t} R_t(X_t)$$

For a probability-flow ODE derived from diffusion, the exact score-to-velocity conversion gives $-(1/2)g^2(t)$.

For a general flow-matching path, there is no diffusion coefficient $g(t)$, so the function $\kappa(t)$ is therefore a guidance design choice that determines how strongly the reward bends the learned flow.

The sign of a manually added gradient must agree with the direction in which generation is integrated.

Some properties have little meaning for an intermediate state

Some properties have little meaning for an intermediate state

A partially denoised protein latent, for example, may not yet correspond to a valid sequence.

Some properties have little meaning for an intermediate state

A partially denoised protein latent, for example, may not yet correspond to a valid sequence.

The flow model can estimate the final state

$$\widehat{X}_1(X_t, t)$$

Some properties have little meaning for an intermediate state

A partially denoised protein latent, for example, may not yet correspond to a valid sequence.

The flow model can estimate the final state

$$\widehat{X}_1(X_t, t)$$

and the reward model can evaluate that estimate:

$$R\left(\widehat{X}_1(X_t, t)\right)$$

Some properties have little meaning for an intermediate state

A partially denoised protein latent, for example, may not yet correspond to a valid sequence.

The flow model can estimate the final state

$$\widehat{X}_1(X_t, t)$$

and the reward model can evaluate that estimate:

$$R\left(\widehat{X}_1(X_t, t)\right)$$

We then differentiate through the endpoint prediction:

$$\nabla_{X_t} R\left(\widehat{X}_1(X_t, t)\right)$$

Some properties have little meaning for an intermediate state

A partially denoised protein latent, for example, may not yet correspond to a valid sequence.

The flow model can estimate the final state

$$\widehat{X}_1(X_t, t)$$

and the reward model can evaluate that estimate:

$$R\left(\widehat{X}_1(X_t, t)\right)$$

We then differentiate through the endpoint prediction:

$$\nabla_{X_t} R\left(\widehat{X}_1(X_t, t)\right)$$

The endpoint-guidance direction measures how changing the current state X_t would change the reward of the predicted completed sample $\widehat{X}_1$.

Some properties have little meaning for an intermediate state

A partially denoised protein latent, for example, may not yet correspond to a valid sequence.

The flow model can estimate the final state

$$\widehat{X}_1(X_t, t)$$

and the reward model can evaluate that estimate:

$$R\left(\widehat{X}_1(X_t, t)\right)$$

We then differentiate through the endpoint prediction:

$$\nabla_{X_t} R\left(\widehat{X}_1(X_t, t)\right)$$

The endpoint-guidance direction measures how changing the current state X_t would change the reward of the predicted completed sample $\widehat{X}_1$.

This approach lets a property predictor defined on clean samples influence earlier states. It requires access to gradients through both the reward model and the endpoint estimator.

But what if a classifier gradient is unavailable?

A partially denoised protein latent, for example, may not yet correspond to a valid sequence.

The flow model can estimate the final state

$$\widehat{X}_1(X_t, t)$$

and the reward model can evaluate that estimate:

$$R\left(\widehat{X}_1(X_t, t)\right)$$

We then differentiate through the endpoint prediction:

$$\nabla_{X_t} R\left(\widehat{X}_1(X_t, t)\right)$$

The endpoint-guidance direction measures how changing the current state X_t would change the reward of the predicted completed sample $\widehat{X}_1$.

This approach lets a property predictor defined on clean samples influence earlier states. It requires access to gradients through both the reward model and the endpoint estimator.

Suppose we randomly remove condition c during training and replace it with a null condition $\emptyset$.

The same model then learns:

$$F_{\theta}(X_t, t, c)$$

Suppose we randomly remove condition c during training and replace it with a null condition $\emptyset$.

The same model then learns:

$$F_{\theta}(X_t, t, c)$$

the conditional field

Suppose we randomly remove condition c during training and replace it with a null condition $\emptyset$.

The same model then learns:

$$F_{\theta}(X_t, t, c)$$

the conditional field, and

$$F_{\theta}(X_t, t, \emptyset)$$

Suppose we randomly remove condition c during training and replace it with a null condition $\emptyset$.

The same model then learns:

$$F_{\theta}(X_t, t, c)$$

the conditional field, and

$$F_{\theta}(X_t, t, \emptyset)$$

the unconditional field.

Suppose we randomly remove condition c during training and replace it with a null condition $\emptyset$.

The same model then learns:

$$F_{\theta}(X_t, t, c)$$

the conditional field, and

$$F_{\theta}(X_t, t, \emptyset)$$

the unconditional field.

Here, F_{θ} may represent a score, noise prediction, or velocity. Why should their difference contain a guidance direction?

Suppose we randomly remove condition c during training and replace it with a null condition $\emptyset$.

The same model then learns:

$$F_{\theta}(X_t, t, c)$$

the conditional field, and

$$F_{\theta}(X_t, t, \emptyset)$$

the unconditional field.

Here, F_{θ} may represent a score, noise prediction, or velocity. Why should their difference contain a guidance direction?

For exact scores, Bayes' rule already showed that

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x)$$

Suppose we randomly remove condition c during training and replace it with a null condition $\emptyset$.

The same model then learns:

$$F_{\theta}(X_t, t, c)$$

the conditional field, and

$$F_{\theta}(X_t, t, \emptyset)$$

the unconditional field.

Here, F_{θ} may represent a score, noise prediction, or velocity. Why should their difference contain a guidance direction?

For exact scores, Bayes' rule already showed that

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x)$$

Rearranging

$$s_t(x \mid c) - s_t(x) = \nabla_x \log p_t(c \mid x)$$

Suppose we randomly remove condition c during training and replace it with a null condition $\emptyset$.

The same model then learns:

$$F_{\theta}(X_t, t, c)$$

the conditional field, and

$$F_{\theta}(X_t, t, \emptyset)$$

the unconditional field.

Here, F_{θ} may represent a score, noise prediction, or velocity. Why should their difference contain a guidance direction?

For exact scores, Bayes' rule already showed that

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x)$$

Rearranging

$$s_t(x \mid c) - s_t(x) = \nabla_x \log p_t(c \mid x)$$

The conditional score minus the unconditional score equals the classifier-gradient direction associated with condition c .

The generative model can therefore learn the effect of the condition without requiring a separately differentiated classifier.

The same model then learns:

$$F_{\theta}(X_t, t, c)$$

the conditional field, and

$$F_{\theta}(X_t, t, \emptyset)$$

the unconditional field.

Here, F_{θ} may represent a score, noise prediction, or velocity. Why should their difference contain a guidance direction?

For exact scores, Bayes' rule already showed that

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x)$$

Rearranging

$$s_t(x \mid c) - s_t(x) = \nabla_x \log p_t(c \mid x)$$

The conditional score minus the unconditional score equals the classifier-gradient direction associated with condition c .

We begin with the unconditional field and amplify the conditional difference:

$$F_{\text{CFG}}(X_t, t, c) = F_{\theta}(X_t, t, \emptyset) + w [F_{\theta}(X_t, t, c) - F_{\theta}(X_t, t, \emptyset)]$$

The classifier-free guided field begins with the unconditional model prediction and adds w times the change in that prediction caused by condition c , where w controls the strength of conditional generation.

We begin with the unconditional field and amplify the conditional difference:

$$F_{\text{CFG}}(X_t, t, c) = F_{\theta}(X_t, t, \emptyset) + w [F_{\theta}(X_t, t, c) - F_{\theta}(X_t, t, \emptyset)]$$

The classifier-free guided field begins with the unconditional model prediction and adds w times the change in that prediction caused by condition c , where w controls the strength of conditional generation.

- $w = 0$ gives the unconditional field.
- $w = 1$ gives the ordinary conditional field.
- $w > 1$ extrapolates beyond the conditional field.

We begin with the unconditional field and amplify the conditional difference:

$$F_{\text{CFG}}(X_t, t, c) = F_{\theta}(X_t, t, \emptyset) + w [F_{\theta}(X_t, t, c) - F_{\theta}(X_t, t, \emptyset)]$$

The classifier-free guided field begins with the unconditional model prediction and adds w times the change in that prediction caused by condition c , where w controls the strength of conditional generation.

- $w = 0$ gives the unconditional field.
- $w = 1$ gives the ordinary conditional field.
- $w > 1$ extrapolates beyond the conditional field.

The generic field equation applies to both model families. For DDPM noise prediction,

$$\varepsilon_{\text{CFG}} = \varepsilon_{\text{uncond}} + w (\varepsilon_{\text{cond}} - \varepsilon_{\text{uncond}})$$

We begin with the unconditional field and amplify the conditional difference:

$$F_{\text{CFG}}(X_t, t, c) = F_{\theta}(X_t, t, \emptyset) + w [F_{\theta}(X_t, t, c) - F_{\theta}(X_t, t, \emptyset)]$$

The classifier-free guided field begins with the unconditional model prediction and adds w times the change in that prediction caused by condition c , where w controls the strength of conditional generation.

- $w = 0$ gives the unconditional field.
- $w = 1$ gives the ordinary conditional field.
- $w > 1$ extrapolates beyond the conditional field.

The classifier-free guided noise ε_{CFG} equals the unconditional predicted noise plus w times the change in predicted noise produced by condition c .

The generic field equation applies to both model families. For DDPM noise prediction,

$$\varepsilon_{\text{CFG}} = \varepsilon_{\text{uncond}} + w (\varepsilon_{\text{cond}} - \varepsilon_{\text{uncond}})$$

We begin with the unconditional field and amplify the conditional difference:

$$F_{\text{CFG}}(X_t, t, c) = F_{\theta}(X_t, t, \emptyset) + w [F_{\theta}(X_t, t, c) - F_{\theta}(X_t, t, \emptyset)]$$

The classifier-free guided field begins with the unconditional model prediction and adds w times the change in that prediction caused by condition c , where w controls the strength of conditional generation.

- $w = 0$ gives the unconditional field.
- $w = 1$ gives the ordinary conditional field.
- $w > 1$ extrapolates beyond the conditional field.

The classifier-free guided velocity v_{CFG} equals the unconditional velocity plus w times the change in velocity produced by condition c .

The generic field equation applies to both model families. For **flow matching**,

$$v_{\text{CFG}} = v_{\text{uncond}} + w (v_{\text{cond}} - v_{\text{uncond}})$$

We begin with the unconditional field and amplify the conditional difference:

$$F_{\text{CFG}}(X_t, t, c) = F_{\theta}(X_t, t, \emptyset) + w [F_{\theta}(X_t, t, c) - F_{\theta}(X_t, t, \emptyset)]$$

The classifier-free guided field begins with the unconditional model prediction and adds w times the change in that prediction caused by condition c , where w controls the strength of conditional generation.

- $w = 0$ gives the unconditional field.
- $w = 1$ gives the ordinary conditional field.
- $w > 1$ extrapolates beyond the conditional field.

The classifier-free guided velocity v_{CFG} equals the unconditional velocity plus w times the change in velocity produced by condition c .

The generic field equation applies to both model families. For **flow matching**,

$$v_{\text{CFG}} = v_{\text{uncond}} + w (v_{\text{cond}} - v_{\text{uncond}})$$

So if we have:

$$\varepsilon_{\text{uncond}} = 0.6, \quad \varepsilon_{\text{cond}} = 0.2, \quad w = 2$$

So if we have:

$$\varepsilon_{\text{uncond}} = 0.6, \quad \varepsilon_{\text{cond}} = 0.2, \quad w = 2$$

We get:

$$\varepsilon_{\text{CFG}} = 0.6 + 2(0.2 - 0.6) = -0.2$$

So if we have:

$$\varepsilon_{\text{uncond}} = 0.6, \quad \varepsilon_{\text{cond}} = 0.2, \quad w = 2$$

We get:

$$\varepsilon_{\text{CFG}} = 0.6 + 2(0.2 - 0.6) = -0.2$$

The condition changes the predicted noise by -0.4 . Guidance with $w = 2$ applies twice that conditional change to the unconditional prediction.

Classifier-free guidance still requires a model trained with the condition.

Classifier-free guidance still requires a model trained with the condition.

Suppose instead that an external objective $R(x)$ returns a value but supplies no gradients.

Classifier-free guidance still requires a model trained with the condition.

Suppose instead that an external objective $R(x)$ returns a value but supplies no gradients.

We can generate J proposals and convert their rewards into selection probabilities:

$$\pi_j = \frac{\exp \left[\eta R(X_t^{(j)}) \right]}{\sum_{\ell=1}^J \exp \left[\eta R(X_t^{(\ell)}) \right]}$$

Classifier-free guidance still requires a model trained with the condition.

Suppose instead that an external objective $R(x)$ returns a value but supplies no gradients.

We can generate J proposals and convert their rewards into selection probabilities:

$$\pi_j = \frac{\exp \left[\eta R(X_t^{(j)}) \right]}{\sum_{\ell=1}^J \exp \left[\eta R(X_t^{(\ell)}) \right]}$$

The probability π_j of retaining proposal $X_t^{(j)}$ equals the exponential of its reward scaled by η , divided by the corresponding exponential rewards summed across all J proposals.

Higher-reward proposals receive greater probability. This supports ranking, resampling, rejection, and repeated candidate refinement.

Classifier-free guidance still requires a model trained with the condition.

Suppose instead that an external objective $R(x)$ returns a value but supplies no gradients.

We can generate J proposals and convert their rewards into selection probabilities:

$$\pi_j = \frac{\exp \left[\eta R(X_t^{(j)}) \right]}{\sum_{\ell=1}^J \exp \left[\eta R(X_t^{(\ell)}) \right]}$$

So while gradient guidance changes a trajectory using a local direction, black-box guidance compares candidates after they have been proposed.

Classifier-free guidance still requires a model trained with the condition.

Suppose instead that an external objective $R(x)$ returns a value but supplies no gradients.

We can generate J proposals and convert their rewards into selection probabilities:

$$\pi_j = \frac{\exp \left[\eta R(X_t^{(j)}) \right]}{\sum_{\ell=1}^J \exp \left[\eta R(X_t^{(\ell)}) \right]}$$

So while gradient guidance changes a trajectory using a local direction, black-box guidance compares candidates after they have been proposed. It generally requires more samples because it does not know which infinitesimal change would improve the reward.

All of these methods have so far used one condition or reward.

All of these methods have so far used one condition or reward.

Biological generation usually requires several properties simultaneously.

Binds Target



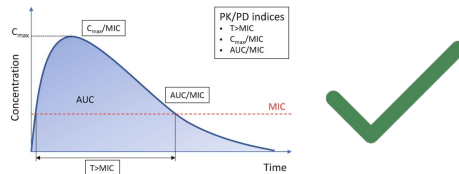
Soluble



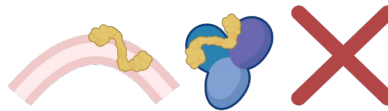
Non-Toxic



Long Half-Life



Non-Fouling



Orally Bioavailable



Suppose we have M objectives:

$$r_1(x), r_2(x), \dots, r_M(x)$$

Suppose we have M objectives:

$$r_1(x), r_2(x), \dots, r_M(x)$$

One objective might be a probability between zero and one, while another might be an energy measured over a much larger numerical range.

Suppose we have M objectives:

$$r_1(x), r_2(x), \dots, r_M(x)$$

One objective might be a probability between zero and one, while another might be an energy measured over a much larger numerical range. Adding the raw values would allow units and scale to determine the tradeoff.

Suppose we have M objectives:

$$r_1(x), r_2(x), \dots, r_M(x)$$

One objective might be a probability between zero and one, while another might be an energy measured over a much larger numerical range. Adding the raw values would allow units and scale to determine the tradeoff.

We therefore normalize each objective:

$$\tilde{r}_m(x) = \frac{r_m(x) - \mu_m}{\tau_m}$$

Suppose we have M objectives:

$$r_1(x), r_2(x), \dots, r_M(x)$$

One objective might be a probability between zero and one, while another might be an energy measured over a much larger numerical range. Adding the raw values would allow units and scale to determine the tradeoff.

We therefore normalize each objective:

$$\tilde{r}_m(x) = \frac{r_m(x) - \mu_m}{\tau_m}$$

The normalized objective $\tilde{r}_m(x)$ equals the original objective $r_m(x)$ after subtracting its reference mean μ_m and dividing by its reference scale τ_m .

Suppose we have M objectives:

$$r_1(x), r_2(x), \dots, r_M(x)$$

One objective might be a probability between zero and one, while another might be an energy measured over a much larger numerical range. Adding the raw values would allow units and scale to determine the tradeoff.

We therefore normalize each objective:

$$\tilde{r}_m(x) = \frac{r_m(x) - \mu_m}{\tau_m}$$

The normalized objective $\tilde{r}_m(x)$ equals the original objective $r_m(x)$ after subtracting its reference mean μ_m and dividing by its reference scale τ_m .

For a property that should be minimized, such as toxicity, we reverse its sign so that larger normalized values consistently represent more desirable outcomes.

Choose nonnegative weights satisfying

$$\lambda_m \geq 0, \quad \sum_{m=1}^M \lambda_m = 1$$

Choose nonnegative weights satisfying

$$\lambda_m \geq 0, \quad \sum_{m=1}^M \lambda_m = 1$$

We then define one scalar reward

$$R_{\lambda}(x) = \sum_{m=1}^M \lambda_m \tilde{r}_m(x)$$

Choose nonnegative weights satisfying

$$\lambda_m \geq 0, \quad \sum_{m=1}^M \lambda_m = 1$$

We then define one scalar reward

$$R_{\lambda}(x) = \sum_{m=1}^M \lambda_m \tilde{r}_m(x)$$

The scalarized reward $R_{\lambda}(x)$ is the weighted sum of M normalized objectives, where λ_m represents the relative importance of objective m and all weights sum to one.

Choose nonnegative weights satisfying

$$\lambda_m \geq 0, \quad \sum_{m=1}^M \lambda_m = 1$$

We then define one scalar reward

$$R_{\lambda}(x) = \sum_{m=1}^M \lambda_m \tilde{r}_m(x)$$

The scalarized reward $R_{\lambda}(x)$ is the weighted sum of M normalized objectives, where λ_m represents the relative importance of objective m and all weights sum to one.

This is YOUR modeling choice. The weights define what tradeoff you want the generator to pursue.

Choose nonnegative weights satisfying

$$\lambda_m \geq 0, \quad \sum_{m=1}^M \lambda_m = 1$$

We then define one scalar reward

$$R_{\lambda}(x) = \sum_{m=1}^M \lambda_m \tilde{r}_m(x)$$

The scalarized reward $R_{\lambda}(x)$ is the weighted sum of M normalized objectives, where λ_m represents the relative importance of objective m and all weights sum to one.

Taking the gradient gives

$$\nabla_x R_{\lambda}(x) = \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_m(x)$$

Choose nonnegative weights satisfying

$$\lambda_m \geq 0, \quad \sum_{m=1}^M \lambda_m = 1$$

We then define one scalar reward

$$R_{\lambda}(x) = \sum_{m=1}^M \lambda_m \tilde{r}_m(x)$$

The scalarized reward gradient equals the weighted sum of the individual normalized objective gradients, so each property contributes a direction according to its tradeoff weight λ_m .

The scalarized reward $R_{\lambda}(x)$ is the weighted sum of M normalized objectives, where λ_m represents the relative importance of objective m and all weights sum to one.

Taking the gradient gives

$$\nabla_x R_{\lambda}(x) = \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_m(x)$$

Now we can insert the scalarized reward into guidance

Now we can insert the scalarized reward into guidance

The reward-guided score was

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

Now we can insert the scalarized reward into guidance

The reward-guided score was

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

Replacing R_t with R_λ gives

$$s_{\text{multi}}(x, t) = s_\theta(x, t) + \eta \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

Now we can insert the scalarized reward into guidance

The reward-guided score was

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

Replacing R_t with R_λ gives

$$s_{\text{multi}}(x, t) = s_\theta(x, t) + \eta \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

The multi-objective guided score equals the learned score $s_\theta(x, t)$ plus η times the weighted sum of M normalized objective gradients, where λ_m controls the relative importance of each objective and η controls the total guidance strength.

Now we can insert the scalarized reward into guidance

The reward-guided score was

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

Replacing R_t with R_λ gives

$$s_{\text{multi}}(x, t) = s_\theta(x, t) + \eta \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

For the diffusion probability-flow ODE,

$$v_{\text{multi}}(x, t) = v_t(x) - \frac{\eta}{2} g^2(t) \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

Now we can insert the scalarized reward into guidance

The reward-guided score was

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

Replacing R_t with R_λ gives

$$s_{\text{multi}}(x, t) = s_\theta(x, t) + \eta \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

For the diffusion probability-flow ODE,

$$v_{\text{multi}}(x, t) = v_t(x) - \frac{\eta}{2} g^2(t) \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

The multi-objective probability-flow velocity equals the unconditional velocity minus one-half of $\eta g^2(t)$ times the weighted objective gradient, which moves backward generation toward increasing scalarized reward.

Now we can insert the scalarized reward into guidance

The reward-guided score was

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

Replacing R_t with R_λ gives

$$s_{\text{multi}}(x, t) = s_\theta(x, t) + \eta \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

For a direct flow that generates forward from prior to data,

$$v_{\text{multi}}(x, t) = v_\theta(x, t) + \kappa(t) \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

Now we can insert the scalarized reward into guidance

The reward-guided score was

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

Replacing R_t with R_λ gives

$$s_{\text{multi}}(x, t) = s_\theta(x, t) + \eta \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

For a direct flow that generates forward from prior to data,

$$v_{\text{multi}}(x, t) = v_\theta(x, t) + \kappa(t) \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

The multi-objective flow velocity equals the learned velocity plus a time-scaled weighted combination of objective gradients, causing the forward trajectory to bend toward states with larger scalarized reward.

Now we can insert the scalarized reward into guidance

The reward-guided score was

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

Replacing R_t with R_λ gives

$$s_{\text{multi}}(x, t) = s_\theta(x, t) + \eta \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

For a direct flow that generates forward from prior to data,

$$v_{\text{multi}}(x, t) = v_\theta(x, t) + \kappa(t) \sum_{m=1}^M \lambda_m \nabla_x \tilde{r}_{m,t}(x)$$

The multi-objective flow velocity equals the learned velocity plus a time-scaled weighted combination of objective gradients, causing the forward trajectory to bend toward states with larger scalarized reward.

How about multi-objective classifier-free guidance?

Suppose F_u is the unconditional field and F_m is the field conditioned on objective m .

How about multi-objective classifier-free guidance?

Suppose F_u is the unconditional field and F_m is the field conditioned on objective m .

We can combine the conditional differences using the same weights:

$$F_{\text{multi}} = F_u + \eta \sum_{m=1}^M \lambda_m (F_m - F_u)$$

How about multi-objective classifier-free guidance?

Suppose F_u is the unconditional field and F_m is the field conditioned on objective m .

We can combine the conditional differences using the same weights:

$$F_{\text{multi}} = F_u + \eta \sum_{m=1}^M \lambda_m (F_m - F_u)$$

The multi-objective classifier-free field begins with the unconditional prediction F_u and adds a weighted sum of the field changes caused by the individual conditions, with λ_m controlling their relative importance and η controlling the total guidance strength.

How about multi-objective classifier-free guidance?

Suppose F_u is the unconditional field and F_m is the field conditioned on objective m .

We can combine the conditional differences using the same weights:

$$F_{\text{multi}} = F_u + \eta \sum_{m=1}^M \lambda_m (F_m - F_u)$$

The field F can represent predicted noise, a score, or a flow velocity.

How about multi-objective classifier-free guidance?

Suppose F_u is the unconditional field and F_m is the field conditioned on objective m .

We can combine the conditional differences using the same weights:

$$F_{\text{multi}} = F_u + \eta \sum_{m=1}^M \lambda_m (F_m - F_u)$$

The field F can represent predicted noise, a score, or a flow velocity.

If the objectives interact strongly, a jointly conditioned model can learn those interactions directly. Combining separate fields assumes that their conditional contributions can be meaningfully scalarized.

Let's do an example.

Let's do an example.

Suppose two candidates have normalized binding and solubility objectives:

	$\tilde{r}_{\text{bind}}$	$\tilde{r}_{\text{sol}}$
x	0.8	0.3
x'	0.6	0.7

Let's do an example.

Suppose two candidates have normalized binding and solubility objectives:

	$\tilde{r}_{\text{bind}}$	$\tilde{r}_{\text{sol}}$
x	0.8	0.3
x'	0.6	0.7

Give binding weight 0.7 and solubility weight 0.3:

$$R_{\lambda}(x) = 0.7(0.8) + 0.3(0.3) = 0.65,$$

$$R_{\lambda}(x') = 0.7(0.6) + 0.3(0.7) = 0.63.$$

Let's do an example.

Suppose two candidates have normalized binding and solubility objectives:

	$\tilde{r}_{\text{bind}}$	$\tilde{r}_{\text{sol}}$
x	0.8	0.3
x'	0.6	0.7

Give binding weight 0.7 and solubility weight 0.3:

$$R_{\lambda}(x) = 0.7(0.8) + 0.3(0.3) = 0.65,$$

$$R_{\lambda}(x') = 0.7(0.6) + 0.3(0.7) = 0.63.$$

Candidate x receives the larger reward.

Let's do an example.

Suppose two candidates have normalized binding and solubility objectives:

	$\tilde{r}_{\text{bind}}$	$\tilde{r}_{\text{sol}}$
x	0.8	0.3
x'	0.6	0.7

Give binding weight 0.7 and solubility weight 0.3:

$$R_{\lambda}(x) = 0.7(0.8) + 0.3(0.3) = 0.65,$$

$$R_{\lambda}(x') = 0.7(0.6) + 0.3(0.7) = 0.63.$$

Candidate x receives the larger reward.

Now give binding weight 0.3 and solubility weight 0.7: $R_{\lambda}(x) = 0.3(0.8) + 0.7(0.3) = 0.45,$

$$R_{\lambda}(x') = 0.3(0.6) + 0.7(0.7) = 0.67.$$

Let's do an example.

Suppose two candidates have normalized binding and solubility objectives:

	$\tilde{r}_{\text{bind}}$	$\tilde{r}_{\text{sol}}$
x	0.8	0.3
x'	0.6	0.7

Give binding weight 0.7 and solubility weight 0.3:

$$R_{\lambda}(x) = 0.7(0.8) + 0.3(0.3) = 0.65,$$

$$R_{\lambda}(x') = 0.7(0.6) + 0.3(0.7) = 0.63.$$

Candidate x' now
receives the larger
reward.

Candidate x receives the larger reward.

Now give binding weight 0.3 and solubility weight 0.7: $R_{\lambda}(x) = 0.3(0.8) + 0.7(0.3) = 0.45,$

$$R_{\lambda}(x') = 0.3(0.6) + 0.7(0.7) = 0.67.$$

Scalarization defines which compromise is preferred for a particular weight vector λ .

Suppose two candidates have normalized binding and solubility objectives:

	$\tilde{r}_{\text{bind}}$	$\tilde{r}_{\text{sol}}$
x	0.8	0.3
x'	0.6	0.7

Give binding weight 0.7 and solubility weight 0.3:

$$R_{\lambda}(x) = 0.7(0.8) + 0.3(0.3) = 0.65,$$

$$R_{\lambda}(x') = 0.7(0.6) + 0.3(0.7) = 0.63.$$

Candidate x' now
receives the larger
reward.

Candidate x receives the larger reward.

Now give binding weight 0.3 and solubility weight 0.7:

$$R_{\lambda}(x) = 0.3(0.8) + 0.7(0.3) = 0.45,$$

$$R_{\lambda}(x') = 0.3(0.6) + 0.7(0.7) = 0.67.$$

Generating with several choices of λ explores different tradeoffs along the Pareto frontier.

Suppose two candidates have normalized binding and solubility objectives:

	$\tilde{r}_{\text{bind}}$	$\tilde{r}_{\text{sol}}$
x	0.8	0.3
x'	0.6	0.7

Give binding weight 0.7 and solubility weight 0.3:

$$R_{\lambda}(x) = 0.7(0.8) + 0.3(0.3) = 0.65,$$

$$R_{\lambda}(x') = 0.7(0.6) + 0.3(0.7) = 0.63.$$

Candidate x' now
receives the larger
reward.

Candidate x receives the larger reward.

Now give binding weight 0.3 and solubility weight 0.7:

$$R_{\lambda}(x) = 0.3(0.8) + 0.7(0.3) = 0.45,$$

$$R_{\lambda}(x') = 0.3(0.6) + 0.7(0.7) = 0.67.$$

Here's an example of competing gradients

Here's an example of competing gradients

Suppose two objective gradients are

$$\nabla_x \tilde{r}_1(x) = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad \nabla_x \tilde{r}_2(x) = \begin{bmatrix} -0.5 \\ 1.5 \end{bmatrix}$$

Here's an example of competing gradients

Suppose two objective gradients are

$$\nabla_x \tilde{r}_1(x) = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad \nabla_x \tilde{r}_2(x) = \begin{bmatrix} -0.5 \\ 1.5 \end{bmatrix}$$

With $\lambda_1 = 0.6$ and $\lambda_2 = 0.4$,

$$\nabla_x R_\lambda(x) = 0.6 \begin{bmatrix} 2 \\ -1 \end{bmatrix} + 0.4 \begin{bmatrix} -0.5 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Here's an example of competing gradients

Suppose two objective gradients are

$$\nabla_x \tilde{r}_1(x) = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad \nabla_x \tilde{r}_2(x) = \begin{bmatrix} -0.5 \\ 1.5 \end{bmatrix}$$

With $\lambda_1 = 0.6$ and $\lambda_2 = 0.4$,

$$\nabla_x R_\lambda(x) = 0.6 \begin{bmatrix} 2 \\ -1 \end{bmatrix} + 0.4 \begin{bmatrix} -0.5 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

The objectives cancel along the second coordinate. Their scalarized direction moves entirely along the first coordinate.

Here's an example of competing gradients

Suppose two objective gradients are

$$\nabla_x \tilde{r}_1(x) = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad \nabla_x \tilde{r}_2(x) = \begin{bmatrix} -0.5 \\ 1.5 \end{bmatrix}$$

With $\lambda_1 = 0.6$ and $\lambda_2 = 0.4$,

$$\nabla_x R_\lambda(x) = 0.6 \begin{bmatrix} 2 \\ -1 \end{bmatrix} + 0.4 \begin{bmatrix} -0.5 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

The objectives cancel along the second coordinate. Their scalarized direction moves entirely along the first coordinate.

Scalarization has now converted several possibly conflicting property directions into one field that either diffusion or flow matching can use.

Scalarization tells the model which valid outcomes are preferred.

A constraint specifies which outcomes are allowed.

A constraint specifies which outcomes are allowed.

Suppose valid samples satisfy

$$h_j(x) \leq 0.$$

A constraint specifies which outcomes are allowed.

Suppose valid samples satisfy

$$h_j(x) \leq 0.$$

A soft constraint can be incorporated through a penalty:

$$R_{\text{constrained}}(x) = R_{\lambda}(x) - \sum_{j=1}^J \rho_j [\max(0, h_j(x))]^2$$

A constraint specifies which outcomes are allowed.

Suppose valid samples satisfy

$$h_j(x) \leq 0.$$

A soft constraint can be incorporated through a penalty:

$$R_{\text{constrained}}(x) = R_{\lambda}(x) - \sum_{j=1}^J \rho_j [\max(0, h_j(x))]^2$$

The constrained reward equals the scalarized multi-objective reward minus a squared penalty for every violated constraint $h_j(x)$, with ρ_j controlling the severity of violation j .

A constraint specifies which outcomes are allowed.

Suppose valid samples satisfy

$$h_j(x) \leq 0.$$

A soft constraint can be incorporated through a penalty:

$$R_{\text{constrained}}(x) = R_{\lambda}(x) - \sum_{j=1}^J \rho_j [\max(0, h_j(x))]^2$$

The constrained reward equals the scalarized multi-objective reward minus a squared penalty for every violated constraint $h_j(x)$, with ρ_j controlling the severity of violation j .

Possible constraints include fixed sequence positions, required motifs, molecular composition, charge, sequence length, structural geometry, maximum toxicity, or minimum activity.

A constraint specifies which outcomes are allowed.

Suppose valid samples satisfy

$$h_j(x) \leq 0.$$

A soft constraint can be incorporated through a penalty:

$$R_{\text{constrained}}(x) = R_{\lambda}(x) - \sum_{j=1}^J \rho_j [\max(0, h_j(x))]^2$$

If an exact projection onto the valid set $\mathcal{C}$ is available, we can use

$$X_{\text{updated}} \leftarrow \Pi_{\mathcal{C}}(X_{\text{updated}})$$

A constraint specifies which outcomes are allowed.

Suppose valid samples satisfy

$$h_j(x) \leq 0.$$

A soft constraint can be incorporated through a penalty:

$$R_{\text{constrained}}(x) = R_{\lambda}(x) - \sum_{j=1}^J \rho_j [\max(0, h_j(x))]^2$$

If an exact projection onto the valid set $\mathcal{C}$ is available, we can use

$$X_{\text{updated}} \leftarrow \Pi_{\mathcal{C}}(X_{\text{updated}})$$

For discrete sequences, constraints can also be enforced through token masks, fixed residues, constrained decoding, rejection, or post-generation filtering.

Guidance should reflect property reliability through time

Guidance should reflect property reliability through time

At high noise, a detailed property predictor may have little reliable information. At lower noise, the emerging structure may support stronger property guidance.

Guidance should reflect property reliability through time

At high noise, a detailed property predictor may have little reliable information. At lower noise, the emerging structure may support stronger property guidance.

We can therefore use:

$$\gamma(t) = \gamma_{\max} a(t)$$

Guidance should reflect property reliability through time

At high noise, a detailed property predictor may have little reliable information. At lower noise, the emerging structure may support stronger property guidance.

We can therefore use:

$$\gamma(t) = \gamma_{\max} a(t)$$

The time-dependent guidance strength $\gamma(t)$ equals the maximum strength $\gamma_{\max}$ multiplied by a schedule $a(t)$, which determines when the condition becomes influential during generation.

Here are some tips:

Here are some tips:

- Normalize objectives before combining them;
- Clip unusually large guidance gradients;
- Apply weaker guidance at high noise;
- Evaluate rewards on a predicted clean endpoint;
- Compare several guidance strengths;
- Preserve an unconditional contribution as a plausibility anchor.

Here are some tips:

- Normalize objectives before combining them;
- Clip unusually large guidance gradients;
- Apply weaker guidance at high noise;
- Evaluate rewards on a predicted clean endpoint;
- Compare several guidance strengths;
- Preserve an unconditional contribution as a plausibility anchor.

These precautions matter because guidance optimizes the prediction made by the property model.

Here are some tips:

- Normalize objectives before combining them;
- Clip unusually large guidance gradients;
- Apply weaker guidance at high noise;
- Evaluate rewards on a predicted clean endpoint;
- Compare several guidance strengths;
- Preserve an unconditional contribution as a plausibility anchor.

These precautions matter because guidance optimizes the prediction made by the property model.

Strong guidance can exploit errors in that model and produce samples with high predicted reward and poor actual behavior.

So let's recap!

So let's recap!

Guidance begins by defining which samples should receive greater probability.

So let's recap!

Guidance begins by defining which samples should receive greater probability.

For a condition c , Bayes' rule gives

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x)$$

So let's recap!

Guidance begins by defining which samples should receive greater probability.

For a condition c , Bayes' rule gives

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x)$$

For a differentiable reward $R_t(x)$, exponential reweighting gives

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

So let's recap!

Guidance begins by defining which samples should receive greater probability.

For a condition c , Bayes' rule gives

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x)$$

For a differentiable reward $R_t(x)$, exponential reweighting gives

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

Diffusion inserts this guided score into the reverse SDE or probability-flow ODE.

So let's recap!

Guidance begins by defining which samples should receive greater probability.

For a condition c , Bayes' rule gives

$$s_t(x \mid c) = s_t(x) + \nabla_x \log p_t(c \mid x)$$

For a differentiable reward $R_t(x)$, exponential reweighting gives

$$s_t^{(R)}(x) = s_t(x) + \eta \nabla_x R_t(x)$$

Diffusion inserts this guided score into the reverse SDE or probability-flow ODE. Flow matching learns a conditional velocity or adds an appropriately scaled reward direction to its velocity.

So let's recap!

When classifier gradients are unavailable, classifier-free guidance uses:

$$F_{\text{CFG}} = F_{\text{uncond}} + w (F_{\text{cond}} - F_{\text{uncond}})$$

So let's recap!

When classifier gradients are unavailable, classifier-free guidance uses:

$$F_{\text{CFG}} = F_{\text{uncond}} + w (F_{\text{cond}} - F_{\text{uncond}})$$

For M objectives, scalarization first defines

$$R_{\lambda}(x) = \sum_{m=1}^M \lambda_m \tilde{r}_m(x)$$

So let's recap!

When classifier gradients are unavailable, classifier-free guidance uses:

$$F_{\text{CFG}} = F_{\text{uncond}} + w (F_{\text{cond}} - F_{\text{uncond}})$$

For M objectives, scalarization first defines

$$R_{\lambda}(x) = \sum_{m=1}^M \lambda_m \tilde{r}_m(x)$$

and then applies the same single-objective guidance mathematics to R_{λ} .

We now have one framework that applies to both model families.

We now have one framework that applies to both model families.

For diffusion, we will modify ϵ_θ or s_θ before computing each reverse update.

We now have one framework that applies to both model families.

For diffusion, we will modify ϵ_θ or s_θ before computing each reverse update.

For flow matching, we will modify v_θ before taking each ODE step.

We now have one framework that applies to both model families.

For diffusion, we will modify ϵ_θ or s_θ before computing each reverse update.

For flow matching, we will modify v_θ before taking each ODE step.

Both implementations will follow the same sequence:

1. Predict the base field

We now have one framework that applies to both model families.

For diffusion, we will modify ϵ_θ or s_θ before computing each reverse update.

For flow matching, we will modify v_θ before taking each ODE step.

Both implementations will follow the same sequence:

1. Predict the base field
2. Compute the guidance signal

We now have one framework that applies to both model families.

For diffusion, we will modify ϵ_θ or s_θ before computing each reverse update.

For flow matching, we will modify v_θ before taking each ODE step.

Both implementations will follow the same sequence:

1. Predict the base field
2. Compute the guidance signal
3. Combine the objectives

We now have one framework that applies to both model families.

For diffusion, we will modify ϵ_θ or s_θ before computing each reverse update.

For flow matching, we will modify v_θ before taking each ODE step.

Both implementations will follow the same sequence:

1. Predict the base field
2. Compute the guidance signal
3. Combine the objectives
4. Take the next generative step

We now have one framework that applies to both model families.

For diffusion, we will modify ϵ_θ or s_θ before computing each reverse update.

For flow matching, we will modify v_θ before taking each ODE step.

Both implementations will follow the same sequence:

1. Predict the base field
2. Compute the guidance signal
3. Combine the objectives
4. Take the next generaitve step

Let's see this in code!

We'll implement latent generative models on ESM-2 embeddings.

Science [Current Issue](#) [First release papers](#) [Archive](#) [About](#) [Submit manuscript](#)

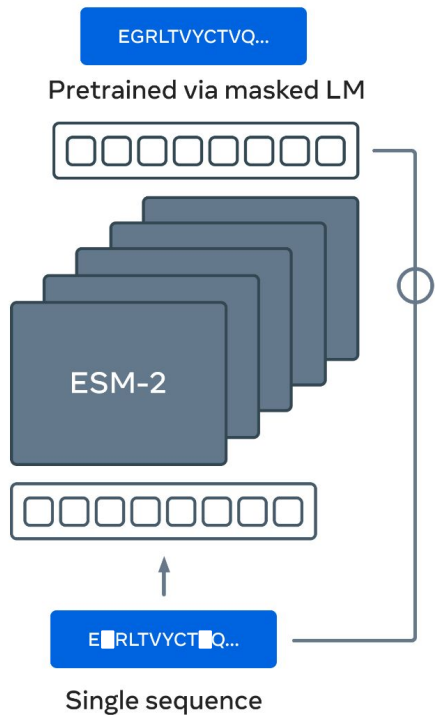
[HOME](#) > [SCIENCE](#) > [VOL. 379, NO. 6637](#) > [EVOLUTIONARY-SCALE PREDICTION OF ATOMIC-LEVEL PROTEIN STRUCTURE WITH A LANGUAGE MODEL](#)

[RESEARCH ARTICLE](#) | [STRUCTURE PREDICTION](#) [f](#) [X](#) [b](#) [in](#) [r](#) [w](#) [e](#) [m](#)

Evolutionary-scale prediction of atomic-level protein structure with a language model

[ZEMING LIN](#) [id](#), [HALIL AKIN](#) [id](#), [ROSHAN RAO](#) [id](#), [BRIAN HIE](#) [id](#), [ZHONGKAI ZHU](#), [WENTING LU](#), [NIKITA SMETANIN](#), [ROBERT VERKUIJL](#) [id](#), [ORI KABELI](#) [id](#), [...], AND [ALEXANDER RIVES](#) [id](#) [+5 authors](#) [Authors Info & Affiliations](#)

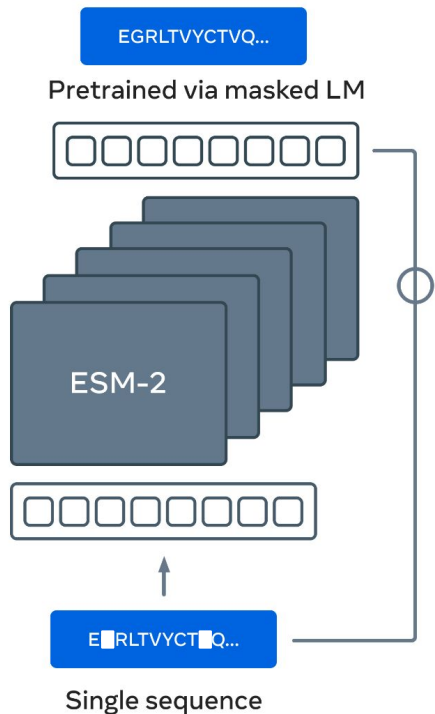
We'll implement latent generative models on ESM-2 embeddings.



- Trained on ~65 million sequences to fill in the “masked”/hidden amino acids

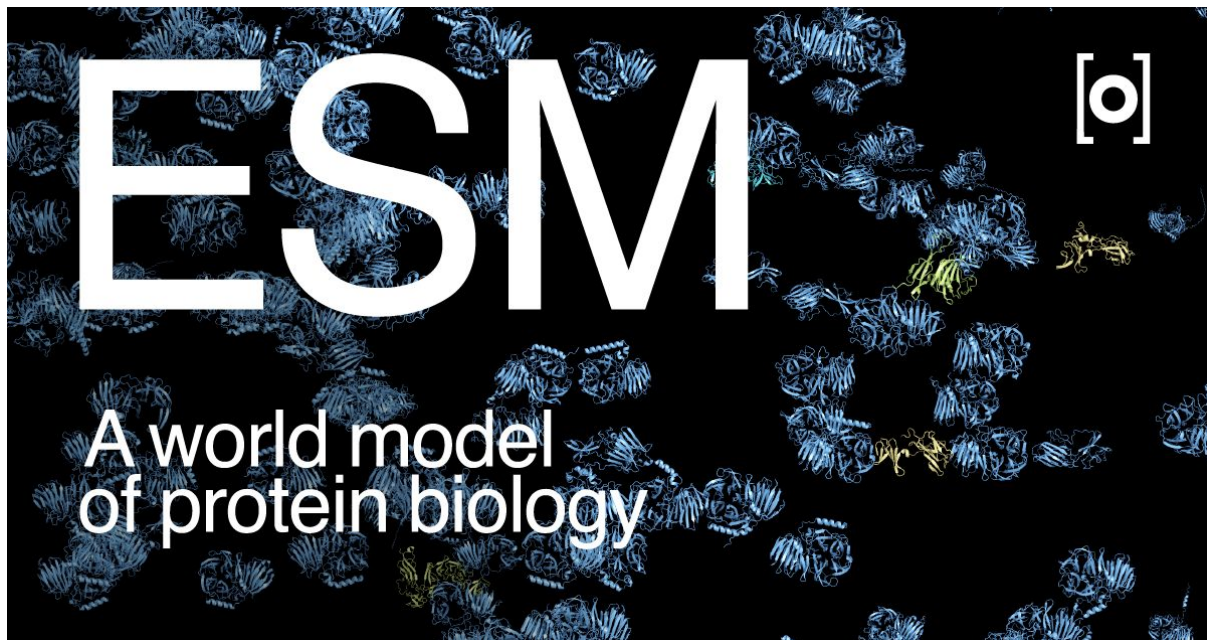
$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(x_i | x_{\setminus \mathcal{M}})$$

We'll implement latent generative models on ESM-2 embeddings.



- Trained on ~65 million sequences to fill in the “masked”/hidden amino acids
- By doing this simple task well, ESM-2 embeddings capture patterns governing structure, function, interaction, biophysical, and chemical properties of input sequences!

BioHub has acquired ESM and are using new ESM-C embeddings for biological world models



We now have one framework that applies to both model families.

For diffusion, we will modify ϵ_θ or s_θ before computing each reverse update.

For flow matching, we will modify v_θ before taking each ODE step.

Both implementations will follow the same sequence:

1. Predict the base field
2. Compute the guidance signal

We will now implement guidance for two generative models using these latents.

We will now implement guidance for two generative models using these latents.

- First, we will train a flow and a diffusion model to generate continuous residue embeddings.

$$\mathcal{L}_{\text{FM}} = \mathbb{E} \left[\frac{1}{Ld} \|v_{\theta}((1-t)\varepsilon + tZ, t, c) - (Z - \varepsilon)\|_F^2 \right].$$

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E} \left[\frac{1}{Ld} \|\varepsilon_{\theta}(\sqrt{\bar{\alpha}_k}Z + \sqrt{1 - \bar{\alpha}_k}\varepsilon, k, c) - \varepsilon\|_F^2 \right].$$

We will now implement guidance for two generative models using these latents.

- First, we will train a flow and a diffusion model to generate continuous residue embeddings.
- We will then compare classifier-free conditioning, steering toward one property, and steering toward a weighted combination of properties.

$$F_{\text{CFG}}(z, t, c) = F_{\theta}(z, t, \emptyset) + w [F_{\theta}(z, t, c) - F_{\theta}(z, t, \emptyset)].$$

$$R_{\lambda}(z, t) = \begin{cases} \hat{r}_1(z, t), & \lambda = (1, 0), \\ \lambda_1 \hat{r}_1(z, t) + \lambda_2 \hat{r}_2(z, t), & \lambda_1, \lambda_2 \geq 0, \quad \lambda_1 + \lambda_2 = 1. \end{cases}$$

$$\begin{aligned} v_{\text{guided}}(z, t) &= v_{\text{CFG}}(z, t, c) + \kappa(t) \nabla_z R_{\lambda}(z, t), \\ \varepsilon_{\text{guided}}(z, k) &= \varepsilon_{\text{CFG}}(z, k, c) - \eta \sqrt{1 - \bar{\alpha}_k} \nabla_z R_{\lambda}(z, k/K). \end{aligned}$$

We will now implement guidance for two generative models using these latents.

- First, we will train a flow and a diffusion model to generate continuous residue embeddings.
- We will then compare classifier-free conditioning, steering toward one property, and steering toward a weighted combination of properties.
- After generation, we will finally decode the embeddings into amino-acid sequences and enforce a minimum number of polar or charged residues.

$$s^* = \arg \max_{s \in \mathcal{A}^L} \sum_{i=1}^L \ell_{i,s_i} \quad \text{subject to} \quad \sum_{i=1}^L \mathbf{1}\{s_i \in \mathcal{P}\} \geq M.$$

Let's start with our flow model.

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKQNRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```

Let's start with our flow model.

Start by choosing CUDA to run.

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path( file ).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKQNRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```


Let's start with our flow model.

Select an ESM checkpoint from HuggingFace

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKNQRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```

Let's start with our flow model.

Set the training config.

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKQRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```

Let's start with our flow model.

We will reserve 20% condition dropout for classifier-free training.

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKNQRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```

This will be our hard constraint

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKQNRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
         for s in sequences
    ], dtype=torch.float32)
```

This will be our hard constraint

- Count K and R, subtract D and E, and divide by sequence length to obtain the charge-count proxy.

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKNQRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```

This will be our hard constraint

- Count K and R, subtract D and E, and divide by sequence length to obtain the charge-count proxy.
- Count residues in POLAR_RESIDUES and divide by sequence length to obtain the polar/charged fraction.

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKNQRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```

This will be our hard constraint

- Count K and R, subtract D and E, and divide by sequence length to obtain the charge-count proxy.
- Count residues in POLAR_RESIDUES and divide by sequence length to obtain the polar/charged fraction.
- Return two property values per sequence.

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKQNRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```

Now load data!

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```


Now load data!

Read each CSV row.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Now load data!

Read each CSV row.

Clean and uppercase the sequences.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu())
    z = torch.cat(encoded)
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Now load data!

Read each CSV row.

Clean and upercase the sequences.

Collect the conditioning labels

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Now load data!

Read each CSV row.

Clean and uppercase the sequences.

Collect the conditioning labels

Load the provided properties or
calculate them with
`composition_proxies()`.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Now we encode the seqs.

Load the tokenizer and ESM-2.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```


Now we encode the seqs.

Load the tokenizer and ESM-2.

Use evaluation mode and freeze ESM-2's parameters.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []

    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Now we encode the seqs.

Load the tokenizer and ESM-2.

Use evaluation mode and freeze ESM-2's parameters.

Obtain the token embeddings.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []

    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu())

    z = torch.cat(encoded)
    # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Now we encode the seqs.

Load the tokenizer and ESM-2.

Use evaluation mode and freeze ESM-2's parameters.

Obtain the token embeddings.

Remove the beginning and end tokens.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```


Now we encode the seqs.

Load the tokenizer and ESM-2.

Use evaluation mode and freeze ESM-2's parameters.

Obtain the token embeddings.

Remove the beginning and end tokens.

Assemble the $N \times L \times D$ embedding tensor.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS: retain all L residue positions
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Now we encode the seqs.

Load the tokenizer and ESM-2.

Use evaluation mode and freeze ESM-2's parameters.

Obtain the token embeddings.

Remove the beginning and end tokens.

Assemble the $N \times L \times D$ embedding tensor.

Here, N is the number of sequences, L is sequence length, and D is the embedding dimension.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS: retain all L residue positions
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Normalize and package

Calculate each feature's mean across sequences and residue positions.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Normalize and package

Calculate each feature's mean across sequences and residue positions.

Calculate the corresponding feature standard deviations.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Normalize and package

Calculate each feature's mean across sequences and residue positions.

Calculate the corresponding feature standard deviations.

Calculate separate statistics for each property.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```


Normalize and package

Calculate each feature's mean across sequences and residue positions.

Calculate the corresponding feature standard deviations.

Calculate separate statistics for each property.

clamp_min(...): keeps the normalization denominators positive.

$$\tilde{Z} = \frac{Z - \mu_Z}{\sigma_Z}, \quad \tilde{r}_m = \frac{r_m - \mu_m}{\sigma_m}.$$

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Normalize and package

Keep normalized embeddings, conditions, and normalized properties aligned.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Normalize and package

Keep normalized embeddings, conditions, and normalized properties aligned.

Save the normalization values for later use.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline="") as handle:
        rows = list(csv.DictReader(handle))
        sequences = [row["sequence"].strip().upper() for row in rows]
        if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
            raise ValueError("Supply at least four sequences using the 20 standard amino acids")
        lengths = {len(s) for s in sequences}
        if len(lengths) != 1 or max(lengths) > 128:
            raise ValueError("This compact example requires one fixed sequence length, at most 128")
        c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
        # With sequence, c only, compute example rewards directly from residue counts.
        # Optional r1/r2 columns override them with supplied labels, such as measurements.
        if {"r1", "r2"}.issubset(rows[0]):
            r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
        else:
            r = composition_proxies(sequences)
        if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
            raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```


Now the model

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class FlowModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.skip = nn.Linear(32, 1) # Preserve full-dimensional state/noise through a time gate.
        self.condition = nn.Embedding(3, 16) # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        return self.skip(time)[:, :, None] * z + self.net(inputs).reshape_as(z)
        # A time-dependent linear part plus a learned nonlinear velocity correction.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.
```

Now the model

Map scalar time into 32 learned features

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class FlowModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.skip = nn.Linear(32, 1) # Preserve full-dimensional state/noise through a time gate.
        self.condition = nn.Embedding(3, 16) # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        return self.skip(time)[:, :, None] * z + self.net(inputs).reshape_as(z)
        # A time-dependent linear part plus a learned nonlinear velocity correction.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.
```

Now the model

Map scalar time into 32 learned features

Map each condition into 16 features; index 2 represents the null condition.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class FlowModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.skip = nn.Linear(32, 1) # Preserve full-dimensional state/noise through a time gate.
        self.condition = nn.Embedding(3, 16) # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        return self.skip(time)[:, :, None] * z + self.net(inputs).reshape_as(z)
        # A time-dependent linear part plus a learned nonlinear velocity correction.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.
```

Now the model

Map scalar time into 32 learned features

Map each condition into 16 features; index 2 represents the null condition.

Combine the flattened latent with 32 time features and 16 condition features, and predict one output for every latent coordinate.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class FlowModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.skip = nn.Linear(32, 1) # Preserve full-dimensional state/noise through a time gate.
        self.condition = nn.Embedding(3, 16) # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        return self.skip(time)[:, :, None] * z + self.net(inputs).reshape_as(z)
        # A time-dependent linear part plus a learned nonlinear velocity correction.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.
```

Now the model

`z.flatten(1)`: flatten each sequence's embeddings.

`torch.cat(...)`: append time and condition features.

`self.skip(time) * z`: add a learned, time-dependent linear contribution.

`.reshape_as(z)`: return a velocity with the same shape as the latent.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class FlowModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.skip = nn.Linear(32, 1) # Preserve full-dimensional state/noise through a time gate.
        self.condition = nn.Embedding(3, 16) # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        return self.skip(time)[:, :, None] * z + self.net(inputs).reshape_as(z)
        # A time-dependent linear part plus a learned nonlinear velocity correction.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.
```

The Reward Model

Concatenate the flattened latent with scalar time.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class FlowModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.skip = nn.Linear(32, 1) # Preserve full-dimensional state/noise through a time gate.
        self.condition = nn.Embedding(3, 16) # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        return self.skip(time)[:, :, None] * z + self.net(inputs).reshape_as(z)
        # A time-dependent linear part plus a learned nonlinear velocity correction.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.
```

The Reward Model

Concatenate the flattened latent with scalar time.

The final Linear(HIDDEN, 2) returns two predicted standardized endpoint properties.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class FlowModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.skip = nn.Linear(32, 1) # Preserve full-dimensional state/noise through a time gate.
        self.condition = nn.Embedding(3, 16) # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        return self.skip(time)[:, :, None] * z + self.net(inputs).reshape_as(z)
        # A time-dependent linear part plus a learned nonlinear velocity correction.
```

```
class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.
```

Now let's train

```
# 3. Flow matching:  $Z_t = (1-t)Z_0 + tZ_1$ ; target velocity is  $Z_1 - Z_0$ .
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = FlowModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z1, c, r_tilde in loader:
            z1, c, r_tilde = z1.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            z0 = torch.randn_like(z1) # Flow convention: Z_0=noise, Z_1=clean latent.
            t = torch.rand(len(z1), device=DEVICE)
            zt = (1 - t[:, None, None]) * z0 + t[:, None, None] * z1
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_flow = F.mse_loss(model(zt, t, dropped), z1 - z0)
            loss_reward = F.mse_loss(reward_model(zt, t), r_tilde)
            loss = loss_flow + loss_reward # Independent parameter sets; one optimizer suffices.
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False) # Freeze weights; still allow gradients of input z.
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```


Now let's train

`DataLoader(...)`: shuffle the dataset and form batches.

```
# 3. Flow matching:  $Z_t = (1-t)Z_0 + tZ_1$ ; target velocity is  $Z_1 - Z_0$ .
def train(dataset, epochs):
    length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = FlowModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z1, c, r_tilde in loader:
            z1, c, r_tilde = z1.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            z0 = torch.randn_like(z1) # Flow convention: Z_0=noise, Z_1=clean latent.
            t = torch.rand(len(z1), device=DEVICE)
            zt = (1 - t[:, None, None]) * z0 + t[:, None, None] * z1
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_flow = F.mse_loss(model(zt, t, dropped), z1 - z0)
            loss_reward = F.mse_loss(reward_model(zt, t), r_tilde)
            loss = loss_flow + loss_reward # Independent parameter sets; one optimizer suffices.
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False) # Freeze weights; still allow gradients of input z.
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Now let's train

`DataLoader(...)`: shuffle the dataset and form batches.

`Adam(...)`: optimize the velocity network and property predictor together.

```
# 3. Flow matching:  $Z_t = (1-t)Z_0 + tZ_1$ ; target velocity is  $Z_1 - Z_0$ .
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = FlowModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)

    for epoch in range(epochs):
        total = 0.0
        for z1, c, r_tilde in loader:
            z1, c, r_tilde = z1.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            z0 = torch.randn_like(z1) # Flow convention: Z_0=noise, Z_1=clean latent.
            t = torch.rand(len(z1), device=DEVICE)
            zt = (1 - t[:, None, None]) * z0 + t[:, None, None] * z1
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_flow = F.mse_loss(model(zt, t, dropped), z1 - z0)
            loss_reward = F.mse_loss(reward_model(zt, t), r_tilde)
            loss = loss_flow + loss_reward # Independent parameter sets; one optimizer suffices.
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()

        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")

    for network in (model, reward_model):
        network.eval().requires_grad_(False) # Freeze weights; still allow gradients of input z.
        for parameter in network.parameters():
            parameter.grad = None

    return model, reward_model
```

Now let's train

`DataLoader(...)`: shuffle the dataset and form batches.

`Adam(...)`: optimize the velocity network and property predictor together.

`z0 = torch.randn_like(z1)`: sample Gaussian noise.

```
# 3. Flow matching:  $Z_t = (1-t)Z_0 + tZ_1$ ; target velocity is  $Z_1 - Z_0$ .
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = FlowModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z1, c, r_tilde in loader:
            z1, c, r_tilde = z1.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            z0 = torch.randn_like(z1) # Flow convention: Z_0=noise, Z_1=clean latent.
            t = torch.rand(len(z1), device=DEVICE)
            zt = (1 - t[:, None, None]) * z0 + t[:, None, None] * z1
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_flow = F.mse_loss(model(zt, t, dropped), z1 - z0)
            loss_reward = F.mse_loss(reward_model(zt, t), r_tilde)
            loss = loss_flow + loss_reward # Independent parameter sets; one optimizer suffices.
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False) # Freeze weights; still allow gradients of input z.
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Now let's train

`DataLoader(...)`: shuffle the dataset and form batches.

`Adam(...)`: optimize the velocity network and property predictor together.

`z0 = torch.randn_like(z1)`: sample Gaussian noise.

`t = torch.rand(...)`: sample an intermediate time.

```
# 3. Flow matching:  $Z_t = (1-t)Z_0 + tZ_1$ ; target velocity is  $Z_1 - Z_0$ .
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = FlowModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z1, c, r_tilde in loader:
            z1, c, r_tilde = z1.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            z0 = torch.randn_like(z1) # Flow convention: Z_0=noise, Z_1=clean latent.
            t = torch.rand(len(z1), device=DEVICE)
            zt = (1 - t[:, None, None]) * z0 + t[:, None, None] * z1
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_flow = F.mse_loss(model(zt, t, dropped), z1 - z0)
            loss_reward = F.mse_loss(reward_model(zt, t), r_tilde)
            loss = loss_flow + loss_reward # Independent parameter sets; one optimizer suffices.
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False) # Freeze weights; still allow gradients of input z.
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Now let's train

Construct the training pair.

$$Z_t = (1 - t)Z_0 + tZ_1, \quad U_t = Z_1 - Z_0.$$

```
# 3. Flow matching:  $Z_t = (1-t)Z_0 + tZ_1$ ; target velocity is  $Z_1 - Z_0$ .
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = FlowModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z1, c, r_tilde in loader:
            z1, c, r_tilde = z1.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            z0 = torch.randn_like(z1) # Flow convention:  $Z_0$ =noise,  $Z_1$ =clean latent.
            t = torch.rand(len(z1), device=DEVICE)
            zt = (1 - t[:, None, None]) * z0 + t[:, None, None] * z1
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, z)
            loss_flow = F.mse_loss(model(zt, t, dropped), z1 - z0)
            loss_reward = F.mse_loss(reward_model(zt, t), r_tilde)
            loss = loss_flow + loss_reward # Independent parameter sets; one optimizer suffices.
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False) # Freeze weights; still allow gradients of input z.
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Now let's train

Construct that intermediate latent.

```
# 3. Flow matching:  $Z_t = (1-t)Z_0 + tZ_1$ ; target velocity is  $Z_1 - Z_0$ .
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = FlowModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z1, c, r_tilde in loader:
            z1, c, r_tilde = z1.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            z0 = torch.randn_like(z1) # Flow convention: Z_0=noise, Z_1=clean latent.
            t = torch.rand(len(z1), device=DEVICE)
            zt = (1 - t[:, None, None]) * z0 + t[:, None, None] * z1
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, z)
            loss_flow = F.mse_loss(model(zt, t, dropped), z1 - z0)
            loss_reward = F.mse_loss(reward_model(zt, t), r_tilde)
            loss = loss_flow + loss_reward # Independent parameter sets; one optimizer suffices.
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False) # Freeze weights; still allow gradients of input z.
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```


Now let's train

Replace randomly selected conditions with null index 2.

```
# 3. Flow matching:  $Z_t = (1-t)Z_0 + tZ_1$ ; target velocity is  $Z_1 - Z_0$ .
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = FlowModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z1, c, r_tilde in loader:
            z1, c, r_tilde = z1.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            z0 = torch.randn_like(z1) # Flow convention: Z_0=noise, Z_1=clean latent.
            t = torch.rand(len(z1), device=DEVICE)
            zt = (1 - t[:, None, None]) * z0 + t[:, None, None] * z1
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_flow = F.mse_loss(model(zt, t, dropped), z1 - z0)
            loss_reward = F.mse_loss(reward_model(zt, t), r_tilde)
            loss = loss_flow + loss_reward # Independent parameter sets; one optimizer suffices.
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False) # Freeze weights; still allow gradients of input z.
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Now let's train

loss_flow: compare predicted velocity with $z_1 - z_0$.

loss_reward: compare predicted properties with r_{tilde} .

loss = ...: add the two training losses.

zero_grad(...): clear the previous gradients.

```
# 3. Flow matching:  $Z_t = (1-t)Z_0 + tZ_1$ ; target velocity is  $Z_1 - Z_0$ .
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = FlowModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z1, c, r_tilde in loader:
            z1, c, r_tilde = z1.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            z0 = torch.randn_like(z1) # Flow convention:  $Z_0$ =noise,  $Z_1$ =clean latent.
            t = torch.rand(len(z1), device=DEVICE)
            zt = (1 - t[:, None, None]) * z0 + t[:, None, None] * z1
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_flow = F.mse_loss(model(zt, t, dropped), z1 - z0)
            loss_reward = F.mse_loss(reward_model(zt, t), r_tilde)
            loss = loss_flow + loss_reward # Independent parameter sets; one optimizer suffices.
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
        for network in (model, reward_model):
            network.eval().requires_grad_(False) # Freeze weights; still allow gradients of input z.
            for parameter in network.parameters():
                parameter.grad = None
    return model, reward_model
```


Now let's train

backward(): calculate
parameter gradients.

step(): update both networks.

The final loop freezes their
parameters for sampling.

```
# 3. Flow matching:  $Z_t = (1-t)Z_0 + tZ_1$ ; target velocity is  $Z_1 - Z_0$ .
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = FlowModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z1, c, r_tilde in loader:
            z1, c, r_tilde = z1.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            z0 = torch.randn_like(z1) # Flow convention: Z_0=noise, Z_1=clean latent.
            t = torch.rand(len(z1), device=DEVICE)
            zt = (1 - t[:, None, None]) * z0 + t[:, None, None] * z1
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_flow = F.mse_loss(model(zt, t, dropped), z1 - z0)
            loss_reward = F.mse_loss(reward_model(zt, t), r_tilde)
            loss = loss_flow + loss_reward # Independent parameter sets; one optimizer suffices.
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
        for network in (model, reward_model):
            network.eval().requires_grad_(False) # Freeze weights; still allow gradients of input z.
            for parameter in network.parameters():
                parameter.grad = None
    return model, reward_model
```

Now let's train

backward(): calculate
parameter gradients.

step(): update both networks.

The final loop freezes their
parameters for sampling.

```
# 3. Flow matching:  $Z_t = (1-t)Z_0 + tZ_1$ ; target velocity is  $Z_1 - Z_0$ .
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = FlowModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z1, c, r_tilde in loader:
            z1, c, r_tilde = z1.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            z0 = torch.randn_like(z1) # Flow convention: Z_0=noise, Z_1=clean latent.
            t = torch.rand(len(z1), device=DEVICE)
            zt = (1 - t[:, None, None]) * z0 + t[:, None, None] * z1
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_flow = F.mse_loss(model(zt, t, dropped), z1 - z0)
            loss_reward = F.mse_loss(reward_model(zt, t), r_tilde)
            loss = loss_flow + loss_reward # Independent parameter sets; one optimizer suffices.
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
        for network in (model, reward_model):
            network.eval().requires_grad_(False) # Freeze weights; still allow gradients of input z.
            for parameter in network.parameters():
                parameter.grad = None
    return model, reward_model
```

Combine the objectives

normalize_weights(...): check the weights and divide by their sum.

```
# 4. R_lambda = sum_m lambda_m * r_tilde_m; differentiate the current latent only.
def reward_gradient(reward_model, z, t, lambdas):
    with torch.enable_grad():
        # Re-enable input gradients inside sampling.
        state = z.detach().requires_grad_(True)
        R_lambda = (reward_model(state, t) * lambdas).sum(dim=1)
        grad = torch.autograd.grad(R_lambda.sum(), state)[0]
    return grad.detach()
    # Do not retain graphs across sampling steps.
```

```
def normalize_weights(lambdas):
    values = torch.as_tensor(lambdas, dtype=torch.float32, device=DEVICE)
    if values.shape != (2,) or not torch.isfinite(values).all() or (values < 0).any() or values.sum() <= 0:
        raise ValueError("Use two finite, nonnegative weights with a positive sum")
    return values / values.sum()
    # lambda controls tradeoffs, eta controls strength.
```

Combine the objectives

`normalize_weights(...)`: check the weights and divide by their sum.

`reward_model(state, t)`: predict the two standardized properties.

```
# 4. R_lambda = sum_m lambda_m * r_tilde_m; differentiate the current latent only.
def reward_gradient(reward_model, z, t, lambdas):
    with torch.enable_grad():
        # Re-enable input gradients inside sampling.
        state = z.detach().requires_grad_(True)
        R_lambda = (reward_model(state, t) * lambdas).sum(dim=1)
        grad = torch.autograd.grad(R_lambda.sum(), state)[0]
    return grad.detach()
    # Do not retain graphs across sampling steps.

def normalize_weights(lambdas):
    values = torch.as_tensor(lambdas, dtype=torch.float32, device=DEVICE)
    if values.shape != (2,) or not torch.isfinite(values).all() or (values < 0).any() or values.sum() <= 0:
        raise ValueError("Use two finite, nonnegative weights with a positive sum")
    return values / values.sum()
    # lambda controls tradeoffs, eta controls strength.
```

Combine the objectives

(... * lambdas).sum(dim=1): calculate one weighted reward per sequence.

$$R_{\lambda}(z, t) = \lambda_1 \hat{r}_1(z, t) + \lambda_2 \hat{r}_2(z, t)$$

```
# 4. R_lambda = sum_m lambda_m * r_tilde_m; differentiate the current latent only.
def reward_gradient(reward_model, z, t, lambdas):
    with torch.enable_grad():
        # Re-enable input gradients inside sampling.
        state = z.detach().requires_grad_(True)
        R_lambda = (reward_model(state, t) * lambdas).sum(dim=1)
        grad = torch.autograd.grad(R_lambda.sum(), state)[0]
    return grad.detach()
    # Do not retain graphs across sampling steps.

def normalize_weights(lambdas):
    values = torch.as_tensor(lambdas, dtype=torch.float32, device=DEVICE)
    if values.shape != (2,) or not torch.isfinite(values).all() or (values < 0).any() or values.sum() <= 0:
        raise ValueError("Use two finite, nonnegative weights with a positive sum")
    return values / values.sum()
    # lambda controls tradeoffs, eta controls strength.
```

Combine the objectives

(... * lambdas).sum(dim=1): calculate one weighted reward per sequence.

$$R_{\lambda}(z, t) = \lambda_1 \hat{r}_1(z, t) + \lambda_2 \hat{r}_2(z, t)$$

The reward R_{λ} combines predicted standardized properties $\hat{r}_1$ and $\hat{r}_2$ at latent z and time t , using tradeoff weights λ_1 and λ_2 .

```
# 4. R_lambda = sum_m lambda_m * r_tilde_m; differentiate the current latent only.
def reward_gradient(reward_model, z, t, lambdas):
    with torch.enable_grad():
        # Re-enable input gradients inside sampling.
        state = z.detach().requires_grad_(True)
        R_lambda = (reward_model(state, t) * lambdas).sum(dim=1)
        grad = torch.autograd.grad(R_lambda.sum(), state)[0]
    return grad.detach()
    # Do not retain graphs across sampling steps.

def normalize_weights(lambdas):
    values = torch.as_tensor(lambdas, dtype=torch.float32, device=DEVICE)
    if values.shape != (2,) or not torch.isfinite(values).all() or (values < 0).any() or values.sum() <= 0:
        raise ValueError("Use two finite, nonnegative weights with a positive sum")
    return values / values.sum()
    # lambda controls tradeoffs, eta controls strength.
```

Combine the objectives

`torch.enable_grad()`: enable differentiation inside the sampling routine.

`z.detach().requires_grad_(True)`: begin a new gradient calculation from the current state.

`autograd.grad(...)`: calculate the reward gradient with respect to that state.

`grad.detach()`: return the gradient without retaining the computation graph.

```
# 4. R_lambda = sum_m lambda_m * r_tilde_m; differentiate the current latent only.
def reward_gradient(reward_model, z, t, lambdas):
    with torch.enable_grad():
        # Re-enable input gradients inside sampling.
        state = z.detach().requires_grad_(True)
        R_lambda = (reward_model(state, t) * lambdas).sum(dim=1)
        grad = torch.autograd.grad(R_lambda.sum(), state)[0]
    return grad.detach()
    # Do not retain graphs across sampling steps.

def normalize_weights(lambdas):
    values = torch.as_tensor(lambdas, dtype=torch.float32, device=DEVICE)
    if values.shape != (2,) or not torch.isfinite(values).all() or (values < 0).any() or values.sum() <= 0:
        raise ValueError("Use two finite, nonnegative weights with a positive sum")
    return values / values.sum()
    # lambda controls tradeoffs, eta controls strength.
```

Now sample the flow!

```
# 5. Sampling: CFG first; optional reward gradient then changes the velocity.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0), steps=200):
    if c not in (0, 1) or steps < 1 or w < 0 or eta < 0:
        raise ValueError("Use c=0/1, positive steps, and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Compare methods from the same initial noise.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    dt = 1.0 / steps
    for step in range(steps):
        t = torch.full((n,), step * dt, device=DEVICE)
        v_uncond = model(z, t, null)
        v = v_uncond
        if w != 0:
            v = v_uncond + w * (model(z, t, condition) - v_uncond)
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            kappa = eta * 4 * t[:, None, None] * (1 - t[:, None, None])
            v = v + kappa * grad_R # Chosen steering rule; not exact conditional transport.
        z = z + dt * v # Integrate forward from t=0 to t=1.
    return z
```


Now sample the flow!

`torch.randn(...)`: initialize the generated latents from Gaussian noise.

```
# 5. Sampling: CFG first; optional reward gradient then changes the velocity.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0), steps=200):
    if c not in (0, 1) or steps < 1 or w < 0 or eta < 0:
        raise ValueError("Use c=0/1, positive steps, and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Compare methods from the same initial noise.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    dt = 1.0 / steps
    for step in range(steps):
        t = torch.full((n,), step * dt, device=DEVICE)
        v_uncond = model(z, t, null)
        v = v_uncond
        if w != 0:
            v = v_uncond + w * (model(z, t, condition) - v_uncond)
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            kappa = eta * 4 * t[:, None, None] * (1 - t[:, None, None])
            v = v + kappa * grad_R # Chosen steering rule; not exact conditional transport.
            z = z + dt * v # Integrate forward from t=0 to t=1.
    return z
```

Now sample the flow!

`torch.randn(...)`: initialize the generated latents from Gaussian noise.

null and condition: prepare unconditional and conditional inputs.

```
# 5. Sampling: CFG first; optional reward gradient then changes the velocity.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0), steps=200):
    if c not in (0, 1) or steps < 1 or w < 0 or eta < 0:
        raise ValueError("Use c=0/1, positive steps, and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Compare methods from the same initial noise.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    dt = 1.0 / steps
    for step in range(steps):
        t = torch.full((n,), step * dt, device=DEVICE)
        v_uncond = model(z, t, null)
        v = v_uncond
        if w != 0:
            v = v_uncond + w * (model(z, t, condition) - v_uncond)
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            kappa = eta * 4 * t[:, None, None] * (1 - t[:, None, None])
            v = v + kappa * grad_R # Chosen steering rule; not exact conditional transport.
        z = z + dt * v # Integrate forward from t=0 to t=1.
    return z
```

Now sample the flow!

`torch.randn(...)`: initialize the generated latents from Gaussian noise.

`null` and `condition`: prepare unconditional and conditional inputs.

`v_uncond`: predict the unconditional velocity.

```
# 5. Sampling: CFG first; optional reward gradient then changes the velocity.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0), steps=200):
    if c not in (0, 1) or steps < 1 or w < 0 or eta < 0:
        raise ValueError("Use c=0/1, positive steps, and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Compare methods from the same initial noise.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    dt = 1.0 / steps

    for step in range(steps):
        t = torch.full((n,), step * dt, device=DEVICE)
        v_uncond = model(z, t, null)
        v = v_uncond
        if w != 0:
            v = v_uncond + w * (model(z, t, condition) - v_uncond)
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            kappa = eta * 4 * t[:, None, None] * (1 - t[:, None, None])
            v = v + kappa * grad_R # Chosen steering rule; not exact conditional transport.
            z = z + dt * v # Integrate forward from t=0 to t=1.
    return z
```

Now sample the flow!

`torch.randn(...)`: initialize the generated latents from Gaussian noise.

`null` and `condition`: prepare unconditional and conditional inputs.

`v_uncond`: predict the unconditional velocity.

`v_uncond + w * (...)`: add w times the conditional difference.

```
# 5. Sampling: CFG first; optional reward gradient then changes the velocity.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0), steps=200):
    if c not in (0, 1) or steps < 1 or w < 0 or eta < 0:
        raise ValueError("Use c=0/1, positive steps, and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123)  # Compare methods from the same initial noise.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    dt = 1.0 / steps
    for step in range(steps):
        t = torch.full((n,), step * dt, device=DEVICE)
        v_uncond = model(z, t, null)
        v = v_uncond
        if w != 0:
            v = v_uncond + w * (model(z, t, condition) - v_uncond)
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            kappa = eta * 4 * t[:, None, None] * (1 - t[:, None, None])
            v = v + kappa * grad_R  # Chosen steering rule; not exact conditional transport.
            z = z + dt * v  # Integrate forward from t=0 to t=1.
    return z
```

Steer and advance!

grad_R: calculate the weighted reward gradient.

```
# 5. Sampling: CFG first; optional reward gradient then changes the velocity.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0), steps=200):
    if c not in (0, 1) or steps < 1 or w < 0 or eta < 0:
        raise ValueError("Use c=0/1, positive steps, and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123)  # Compare methods from the same initial noise.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    dt = 1.0 / steps
    for step in range(steps):
        t = torch.full((n,), step * dt, device=DEVICE)
        v_uncond = model(z, t, null)
        v = v_uncond
        if w != 0:
            v = v_uncond + w * (model(z, t, condition) - v_uncond)
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            kappa = eta * 4 * t[:, None, None] * (1 - t[:, None, None])
            v = v + kappa * grad_R  # Chosen steering rule; not exact conditional transport.
            z = z + dt * v  # Integrate forward from t=0 to t=1.
    return z
```

Steer and advance!

grad_R: calculate the weighted reward gradient.

kappa: set the chosen schedule $\kappa(t) = 4\eta t(1-t)$.

```
# 5. Sampling: CFG first; optional reward gradient then changes the velocity.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0), steps=200):
    if c not in (0, 1) or steps < 1 or w < 0 or eta < 0:
        raise ValueError("Use c=0/1, positive steps, and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123)  # Compare methods from the same initial noise.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    dt = 1.0 / steps
    for step in range(steps):
        t = torch.full((n,), step * dt, device=DEVICE)
        v_uncond = model(z, t, null)
        v = v_uncond
        if w != 0:
            v = v_uncond + w * (model(z, t, condition) - v_uncond)
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            kappa = eta * 4 * t[:, None, None] * (1 - t[:, None, None])
            v = v + kappa * grad_R  # Chosen steering rule; not exact conditional transport.
            z = z + dt * v  # Integrate forward from t=0 to t=1.
    return z
```

Steer and advance!

grad_R: calculate the weighted reward gradient.

kappa: set the chosen schedule $\kappa(t) = 4\eta t(1-t)$.

$v = v + \text{kappa} * \text{grad_R}$: add reward steering.

$z = z + dt * v$: take an Euler step.

```
# 5. Sampling: CFG first; optional reward gradient then changes the velocity.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0), steps=200):
    if c not in (0, 1) or steps < 1 or w < 0 or eta < 0:
        raise ValueError("Use c=0/1, positive steps, and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Compare methods from the same initial noise.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    dt = 1.0 / steps
    for step in range(steps):
        t = torch.full((n,), step * dt, device=DEVICE)
        v_uncond = model(z, t, null)
        v = v_uncond
        if w != 0:
            v = v_uncond + w * (model(z, t, condition) - v_uncond)
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            kappa = eta * 4 * t[:, None, None] * (1 - t[:, None, None])
            v = v + kappa * grad_R # Chosen steering rule; not exact conditional transport.
        z = z + dt * v # Integrate forward from t=0 to t=1.
    return z
```


Steer and advance!

grad_R: calculate the weighted reward gradient.

kappa: set the chosen schedule $\kappa(t) = 4\eta t(1-t)$.

$v = v + \text{kappa} * \text{grad_R}$: add reward steering.

$z = z + dt * v$: take an Euler step.

$$z_{t+\Delta t} = z_t + \Delta t [v_{\text{CFG}} + \kappa(t) \nabla_z R_\lambda]$$

```
# 5. Sampling: CFG first; optional reward gradient then changes the velocity.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0), steps=200):
    if c not in (0, 1) or steps < 1 or w < 0 or eta < 0:
        raise ValueError("Use c=0/1, positive steps, and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Compare methods from the same initial noise.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    dt = 1.0 / steps
    for step in range(steps):
        t = torch.full((n,), step * dt, device=DEVICE)
        v_uncond = model(z, t, null)
        v = v_uncond
        if w != 0:
            v = v_uncond + w * (model(z, t, condition) - v_uncond)
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            kappa = eta * 4 * t[:, None, None] * (1 - t[:, None, None])
            v = v + kappa * grad_R # Chosen steering rule; not exact conditional transport.
            z = z + dt * v # Integrate forward from t=0 to t=1.
    return z
```


Steer and advance!

grad_R: calculate the weighted reward gradient.

kappa: set the chosen schedule $\kappa(t) = 4\eta t(1-t)$.

$v = v + \text{kappa} * \text{grad_R}$: add reward steering.

$z = z + dt * v$: take an Euler step.

$$z_{t+\Delta t} = z_t + \Delta t [v_{\text{CFG}} + \kappa(t) \nabla_z R_\lambda]$$

Advance latent z_t by time step Δt using the CFG velocity and the reward gradient, scaled by the time-dependent strength $\kappa(t)$.

```
# 5. Sampling: CFG first; optional reward gradient then changes the velocity.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0), steps=200):
    if c not in (0, 1) or steps < 1 or w < 0 or eta < 0:
        raise ValueError("Use c=0/1, positive steps, and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Compare methods from the same initial noise.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    dt = 1.0 / steps
    for step in range(steps):
        t = torch.full((n,), step * dt, device=DEVICE)
        v_uncond = model(z, t, null)
        v = v_uncond
        if w != 0:
            v = v_uncond + w * (model(z, t, condition) - v_uncond)
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            kappa = eta * 4 * t[:, None, None] * (1 - t[:, None, None])
            v = v + kappa * grad_R # Chosen steering rule; not exact conditional transport.
            z = z + dt * v # Integrate forward from t=0 to t=1.
    return z
```

Put it all together in main()

```
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, default=ROOT / "esm2_example.csv")
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--samples", type=int, default=8)
    parser.add_argument("--min-polar", type=int, default=12)
    parser.add_argument("--output", type=Path, default=ROOT / "esm2_flow_outputs")
    args = parser.parse_args()
    if args.epochs < 1 or args.samples < 1:
        parser.error("epochs and samples must be positive")
    torch.manual_seed(7)
    if DEVICE.type == "cpu":
        torch.set_num_threads(2)
    dataset, esm, tokenizer, stats = load_data(args.data)
    if not 0 <= args.min_polar <= dataset.tensors[0].shape[1]:
        parser.error("min-polar must lie between zero and the sequence length")
    model, reward_model = train(dataset, args.epochs)
    outputs = {
        "cfg": sample(model, reward_model, args.samples, c=1, w=2.0),
        "single": sample(model, reward_model, args.samples, eta=1.0, lambdas=(1.0, 0.0)),
        "multi": sample(model, reward_model, args.samples, eta=1.0, lambdas=(0.7, 0.3)),
    }
    args.output.mkdir(parents=True, exist_ok=True)
    for name, latent in outputs.items():
        if not torch.isfinite(latent).all():
            raise RuntimeError(f"Nonfinite {name} output; reduce guidance or check training")
        sequences = decode(latent, esm, tokenizer, stats, args.min_polar)
        assert all(sum(a in POLAR_RESIDUES for a in s) >= args.min_polar for s in sequences)
        fasta = "".join(f">{name}_{i+1}\n{seq}\n" for i, seq in enumerate(sequences))
        (args.output / f"{name}.fasta").write_text(fasta)
        count = sum(a in POLAR_RESIDUES for a in sequences[0])
        print(f"{name}: {sequences[0]} polar/charged residues={count}")
        print(" decoded mean composition proxies:", composition_proxies(sequences).mean(0).tolist())
    torch.save({"standardized_latents": {k: v.cpu() for k, v in outputs.items()},
               "model": model.state_dict(), "reward_model": reward_model.state_dict(),
               "stats": stats, "esm_name": ESM_NAME,
               "length": model.length, "dim": model.dim, "min_polar": args.min_polar,
               "polar_residues": POLAR_RESIDUES}, args.output / "results.pt")
    print(f"Saved latent tensors and FASTA sequences to {args.output}")

if __name__ == "__main__":
    main()
```

Put it all together in main()

load_data(...): prepare embeddings and labels.

```
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, default=ROOT / "esm2_example.csv")
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--samples", type=int, default=8)
    parser.add_argument("--min-polar", type=int, default=12)
    parser.add_argument("--output", type=Path, default=ROOT / "esm2_flow_outputs")
    args = parser.parse_args()
    if args.epochs < 1 or args.samples < 1:
        parser.error("epochs and samples must be positive")
    torch.manual_seed(7)
    if DEVICE.type == "cpu":
        torch.set_num_threads(2)
    dataset, esm, tokenizer, stats = load_data(args.data)
    if not 0 <= args.min_polar <= dataset.tensors[0].shape[1]:
        parser.error("min-polar must lie between zero and the sequence length")
    model, reward_model = train(dataset, args.epochs)
    outputs = {
        "cfg": sample(model, reward_model, args.samples, c=1, w=2.0),
        "single": sample(model, reward_model, args.samples, eta=1.0, lambdas=(1.0, 0.0)),
        "multi": sample(model, reward_model, args.samples, eta=1.0, lambdas=(0.7, 0.3)),
    }
    args.output.mkdir(parents=True, exist_ok=True)
    for name, latent in outputs.items():
        if not torch.isfinite(latent).all():
            raise RuntimeError(f"Nonfinite {name} output; reduce guidance or check training")
        sequences = decode(latent, esm, tokenizer, stats, args.min_polar)
        assert all(sum(a in POLAR_RESIDUES for a in s) >= args.min_polar for s in sequences)
        fasta = "".join(f">{name}_{i+1}\n{seq}\n" for i, seq in enumerate(sequences))
        (args.output / f"{name}.fasta").write_text(fasta)
        count = sum(a in POLAR_RESIDUES for a in sequences[0])
        print(f"{name}: {sequences[0]} polar/charged residues={count}")
        print(" decoded mean composition proxies:", composition_proxies(sequences).mean(0).tolist())
    torch.save({"standardized_latents": {k: v.cpu() for k, v in outputs.items()},
              "model": model.state_dict(), "reward_model": reward_model.state_dict(),
              "stats": stats, "esm_name": ESM_NAME,
              "length": model.length, "dim": model.dim, "min_polar": args.min_polar,
              "polar_residues": POLAR_RESIDUES}, args.output / "results.pt")
    print(f"Saved latent tensors and FASTA sequences to {args.output}")

if __name__ == "__main__":
    main()
```

Put it all together in main()

load_data(...): prepare embeddings and labels.

train(...): train both networks.

```
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, default=ROOT / "esm2_example.csv")
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--samples", type=int, default=8)
    parser.add_argument("--min-polar", type=int, default=12)
    parser.add_argument("--output", type=Path, default=ROOT / "esm2_flow_outputs")
    args = parser.parse_args()
    if args.epochs < 1 or args.samples < 1:
        parser.error("epochs and samples must be positive")
    torch.manual_seed(7)
    if DEVICE.type == "cpu":
        torch.set_num_threads(2)
    dataset, esm, tokenizer, stats = load_data(args.data)
    if not 0 <= args.min_polar <= dataset.tensors[0].shape[1]:
        parser.error("min-polar must lie between zero and the sequence length")
    model, reward_model = train(dataset, args.epochs)
    outputs = {
        "cfg": sample(model, reward_model, args.samples, c=1, w=2.0),
        "single": sample(model, reward_model, args.samples, eta=1.0, lambdas=(1.0, 0.0)),
        "multi": sample(model, reward_model, args.samples, eta=1.0, lambdas=(0.7, 0.3)),
    }
    args.output.mkdir(parents=True, exist_ok=True)
    for name, latent in outputs.items():
        if not torch.isfinite(latent).all():
            raise RuntimeError(f"Nonfinite {name} output; reduce guidance or check training")
        sequences = decode(latent, esm, tokenizer, stats, args.min_polar)
        assert all(sum(a in POLAR_RESIDUES for a in s) >= args.min_polar for s in sequences)
        fasta = "".join(f">{name}_{i+1}\n{seq}\n" for i, seq in enumerate(sequences))
        (args.output / f"{name}.fasta").write_text(fasta)
        count = sum(a in POLAR_RESIDUES for a in sequences[0])
        print(f"{name}: {sequences[0]} polar/charged residues={count}")
        print(" decoded mean composition proxies:", composition_proxies(sequences).mean(0).tolist())
    torch.save({"standardized_latents": {k: v.cpu() for k, v in outputs.items()},
              "model": model.state_dict(), "reward_model": reward_model.state_dict(),
              "stats": stats, "esm_name": ESM_NAME,
              "length": model.length, "dim": model.dim, "min_polar": args.min_polar,
              "polar_residues": POLAR_RESIDUES}, args.output / "results.pt")
    print(f"Saved latent tensors and FASTA sequences to {args.output}")

if __name__ == "__main__":
    main()
```

Put it all together in main()

load_data(...): prepare embeddings and labels.

train(...): train both networks.

"cfg": sample with class conditioning and $w = 2$.

```
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, default=ROOT / "esm2_example.csv")
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--samples", type=int, default=8)
    parser.add_argument("--min-polar", type=int, default=12)
    parser.add_argument("--output", type=Path, default=ROOT / "esm2_flow_outputs")
    args = parser.parse_args()
    if args.epochs < 1 or args.samples < 1:
        parser.error("epochs and samples must be positive")
    torch.manual_seed(7)
    if DEVICE.type == "cpu":
        torch.set_num_threads(2)
    dataset, esm, tokenizer, stats = load_data(args.data)
    if not 0 <= args.min_polar <= dataset.tensors[0].shape[1]:
        parser.error("min-polar must lie between zero and the sequence length")
    model, reward_model = train(dataset, args.epochs)
    outputs = {
        "cfg": sample(model, reward_model, args.samples, c=1, w=2.0),
        "single": sample(model, reward_model, args.samples, eta=1.0, lambdas=(1.0, 0.0)),
        "multi": sample(model, reward_model, args.samples, eta=1.0, lambdas=(0.7, 0.3)),
    }
    args.output.mkdir(parents=True, exist_ok=True)
    for name, latent in outputs.items():
        if not torch.isfinite(latent).all():
            raise RuntimeError(f"Nonfinite {name} output; reduce guidance or check training")
        sequences = decode(latent, esm, tokenizer, stats, args.min_polar)
        assert all(sum(a in POLAR_RESIDUES for a in s) >= args.min_polar for s in sequences)
        fasta = "".join(f">{name}_{i+1}\n{seq}\n" for i, seq in enumerate(sequences))
        (args.output / f"{name}.fasta").write_text(fasta)
        count = sum(a in POLAR_RESIDUES for a in sequences[0])
        print(f"{name}: {sequences[0]} polar/charged residues={count}")
        print(" decoded mean composition proxies:", composition_proxies(sequences).mean(0).tolist())
    torch.save({"standardized_latents": {k: v.cpu() for k, v in outputs.items()},
              "model": model.state_dict(), "reward_model": reward_model.state_dict(),
              "stats": stats, "esm_name": ESM_NAME,
              "length": model.length, "dim": model.dim, "min_polar": args.min_polar,
              "polar_residues": POLAR_RESIDUES}, args.output / "results.pt")
    print(f"Saved latent tensors and FASTA sequences to {args.output}")

if __name__ == "__main__":
    main()
```


Put it all together in main()

load_data(...): prepare embeddings and labels.

train(...): train both networks.

"cfg": sample with class conditioning and $w = 2$.

"single": steer with $\lambda = (1, 0)$.

"multi": steer with $\lambda = (0.7, 0.3)$.

Pass each completed latent batch to the final decoder.

```
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, default=ROOT / "esm2_example.csv")
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--samples", type=int, default=8)
    parser.add_argument("--min-polar", type=int, default=12)
    parser.add_argument("--output", type=Path, default=ROOT / "esm2_flow_outputs")
    args = parser.parse_args()
    if args.epochs < 1 or args.samples < 1:
        parser.error("epochs and samples must be positive")
    torch.manual_seed(7)
    if DEVICE.type == "cpu":
        torch.set_num_threads(2)
    dataset, esm, tokenizer, stats = load_data(args.data)
    if not 0 <= args.min_polar <= dataset.tensors[0].shape[1]:
        parser.error("min-polar must lie between zero and the sequence length")
    model, reward_model = train(dataset, args.epochs)
    outputs = {
        "cfg": sample(model, reward_model, args.samples, c=1, w=2.0),
        "single": sample(model, reward_model, args.samples, eta=1.0, lambdas=(1.0, 0.0)),
        "multi": sample(model, reward_model, args.samples, eta=1.0, lambdas=(0.7, 0.3)),
    }
    args.output.mkdir(parents=True, exist_ok=True)
    for name, latent in outputs.items():
        if not torch.isfinite(latent).all():
            raise RuntimeError(f"Nonfinite {name} output; reduce guidance or check training")
        sequences = decode(latent, esm, tokenizer, stats, args.min_polar)
        assert all(sum(a in POLAR_RESIDUES for a in s) >= args.min_polar for s in sequences)
        fasta = "".join(f">{name}_{i+1}\n{seq}\n" for i, seq in enumerate(sequences))
        (args.output / f"{name}.fasta").write_text(fasta)
        count = sum(a in POLAR_RESIDUES for a in sequences[0])
        print(f"{name}: {sequences[0]} polar/charged residues={count}")
        print(" decoded mean composition proxies:", composition_proxies(sequences).mean(0).tolist())
        torch.save({"standardized_latents": {k: v.cpu() for k, v in outputs.items()},
                    "model": model.state_dict(), "reward_model": reward_model.state_dict(),
                    "stats": stats, "esm_name": ESM_NAME,
                    "length": model.length, "dim": model.dim, "min_polar": args.min_polar,
                    "polar_residues": POLAR_RESIDUES}, args.output / "results.pt")
    print(f"Saved latent tensors and FASTA sequences to {args.output}")

if __name__ == "__main__":
    main()
```

Put it all together in main()

load_data(...): prepare embeddings and labels.

train(...): train both networks.

"cfg": sample with class conditioning and $w = 2$.

"single": steer with $\lambda = (1, 0)$.

"multi": steer with $\lambda = (0.7, 0.3)$.

```
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, default=ROOT / "esm2_example.csv")
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--samples", type=int, default=8)
    parser.add_argument("--min-polar", type=int, default=12)
    parser.add_argument("--output", type=Path, default=ROOT / "esm2_flow_outputs")
    args = parser.parse_args()
    if args.epochs < 1 or args.samples < 1:
        parser.error("epochs and samples must be positive")
    torch.manual_seed(7)
    if DEVICE.type == "cpu":
        torch.set_num_threads(2)
    dataset, esm, tokenizer, stats = load_data(args.data)
    if not 0 <= args.min_polar <= dataset.tensors[0].shape[1]:
        parser.error("min-polar must lie between zero and the sequence length")
    model, reward_model = train(dataset, args.epochs)
    outputs = {
        "cfg": sample(model, reward_model, args.samples, c=1, w=2.0),
        "single": sample(model, reward_model, args.samples, eta=1.0, lambdas=(1.0, 0.0)),
        "multi": sample(model, reward_model, args.samples, eta=1.0, lambdas=(0.7, 0.3)),
    }
    args.output.mkdir(parents=True, exist_ok=True)
    for name, latent in outputs.items():
        if not torch.isfinite(latent).all():
            raise RuntimeError(f"Nonfinite {name} output; reduce guidance or check training")
        sequences = decode(latent, esm, tokenizer, stats, args.min_polar)
        assert all(sum(a in POLAR_RESIDUES for a in s) >= args.min_polar for s in sequences)
        fasta = "".join(f">{name}_{i+1}\n{seq}\n" for i, seq in enumerate(sequences))
        (args.output / f"{name}.fasta").write_text(fasta)
        count = sum(a in POLAR_RESIDUES for a in sequences[0])
        print(f"{name}: {sequences[0]} polar/charged residues={count}")
        print(" decoded mean composition proxies:", composition_proxies(sequences).mean(0).tolist())
    torch.save({"standardized_latents": {k: v.cpu() for k, v in outputs.items()},
              "model": model.state_dict(), "reward_model": reward_model.state_dict(),
              "stats": stats, "esm_name": ESM_NAME,
              "length": model.length, "dim": model.dim, "min_polar": args.min_polar,
              "polar_residues": POLAR_RESIDUES}, args.output / "results.pt")
    print(f"Saved latent tensors and FASTA sequences to {args.output}")

if __name__ == "__main__":
    main()
```

Put it all together in main()

load_data(...): prepare embeddings and labels.

train(...): train both networks.

"cfg": sample with class conditioning and $w = 2$.

"single": steer with $\lambda = (1, 0)$.

"multi": steer with $\lambda = (0.7, 0.3)$.

Pass each completed latent batch to the final decoder.

```
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, default=ROOT / "esm2_example.csv")
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--samples", type=int, default=8)
    parser.add_argument("--min-polar", type=int, default=12)
    parser.add_argument("--output", type=Path, default=ROOT / "esm2_flow_outputs")
    args = parser.parse_args()
    if args.epochs < 1 or args.samples < 1:
        parser.error("epochs and samples must be positive")
    torch.manual_seed(7)
    if DEVICE.type == "cpu":
        torch.set_num_threads(2)
    dataset, esm, tokenizer, stats = load_data(args.data)
    if not 0 <= args.min_polar <= dataset.tensors[0].shape[1]:
        parser.error("min-polar must lie between zero and the sequence length")
    model, reward_model = train(dataset, args.epochs)
    outputs = {
        "cfg": sample(model, reward_model, args.samples, c=1, w=2.0),
        "single": sample(model, reward_model, args.samples, eta=1.0, lambdas=(1.0, 0.0)),
        "multi": sample(model, reward_model, args.samples, eta=1.0, lambdas=(0.7, 0.3)),
    }
    args.output.mkdir(parents=True, exist_ok=True)
    for name, latent in outputs.items():
        if not torch.isfinite(latent).all():
            raise RuntimeError(f"Nonfinite {name} output; reduce guidance or check training")
        sequences = decode(latent, esm, tokenizer, stats, args.min_polar)
        assert all((sum(a in POLAR_RESIDUES for a in s) >= args.min_polar for s in sequences))
        fasta = "".join(f">{name}_{i+1}\n{seq}\n" for i, seq in enumerate(sequences))
        (args.output / f"{name}.fasta").write_text(fasta)
        count = sum(a in POLAR_RESIDUES for a in sequences[0])
        print(f"{name}: {sequences[0]} polar/charged residues={count}")
        print(" decoded mean composition proxies:", composition_proxies(sequences).mean(0).tolist())
    torch.save({"standardized_latents": {k: v.cpu() for k, v in outputs.items()},
               "model": model.state_dict(), "reward_model": reward_model.state_dict(),
               "stats": stats, "esm_name": ESM_NAME,
               "length": model.length, "dim": model.dim, "min_polar": args.min_polar,
               "polar_residues": POLAR_RESIDUES}, args.output / "results.pt")
    print(f"Saved latent tensors and FASTA sequences to {args.output}")

if __name__ == "__main__":
    main()
```


Now let's see the same process for diffusion!

Now let's see the same process for diffusion!

Select ESM-2, the device, and training settings.

Define the two composition properties.

Read sequences, conditions, and optional property annotations.

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKNQRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```

Now let's see the same process for diffusion!

Select ESM-2, the device, and training settings.

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKNQRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```

Now let's see the same process for diffusion!

Select ESM-2, the device, and training settings.

Define the two composition properties.

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKNQRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```

Now let's see the same process for diffusion!

Select ESM-2, the device, and training settings.

Define the two composition properties.

Read sequences, conditions, and optional property annotations.

```
import argparse
import csv
from pathlib import Path

import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoTokenizer, EsmForMaskedLM

ROOT = Path(__file__).resolve().parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
ESM_NAME = "facebook/esm2_t6_8M_UR50D"
AMINO_ACIDS = "ACDEFGHIKLMNPQRSTVWY"
POLAR_RESIDUES = "DEHKQNRST" # Operational polar/charged set; not a solubility assay.
BATCH_SIZE, HIDDEN, LEARNING_RATE = 16, 128, 1e-3
CONDITION_DROP = 0.2 # Drop c during training so the same model learns the null condition.

def composition_proxies(sequences):
    # Transparent teaching rewards; these are not measured activity or solubility.
    return torch.tensor([
        [(sum(a in "KR" for a in s) - sum(a in "DE" for a in s)) / len(s),
         sum(a in POLAR_RESIDUES for a in s) / len(s)]
        for s in sequences
    ], dtype=torch.float32)
```

Load data

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline='') as handle:
        rows = list(csv.DictReader(handle))
    sequences = [row["sequence"].strip().upper() for row in rows]
    if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
        raise ValueError("Supply at least four sequences using the 20 standard amino acids")
    lengths = {len(s) for s in sequences}
    if len(lengths) != 1 or max(lengths) > 128:
        raise ValueError("This compact example requires one fixed sequence length, at most 128")
    c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
    # With sequence, c only, compute example rewards directly from residue counts.
    # Optional r1/r2 columns override them with supplied labels, such as measurements.
    if {"r1", "r2"}.issubset(rows[0]):
        r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
    else:
        r = composition_proxies(sequences)
    if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
        raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

Load data

Freeze ESM-2 and extract its residue embeddings.

Remove beginning/end tokens with `hidden[:, 1:-1]`.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline='') as handle:
        rows = list(csv.DictReader(handle))
    sequences = [row["sequence"].strip().upper() for row in rows]
    if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
        raise ValueError("Supply at least four sequences using the 20 standard amino acids")
    lengths = {len(s) for s in sequences}
    if len(lengths) != 1 or max(lengths) > 128:
        raise ValueError("This compact example requires one fixed sequence length, at most 128")
    c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
    # With sequence, c only, compute example rewards directly from residue counts.
    # Optional r1/r2 columns override them with supplied labels, such as measurements.
    if {"r1", "r2"}.issubset(rows[0]):
        r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
    else:
        r = composition_proxies(sequences)
    if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
        raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```


Load data

Freeze ESM-2 and extract its residue embeddings.

Remove beginning/end tokens with `hidden[:, 1:-1]`.

Standardize latent features and each property using training-set statistics.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline='') as handle:
        rows = list(csv.DictReader(handle))
    sequences = [row["sequence"].strip().upper() for row in rows]
    if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
        raise ValueError("Supply at least four sequences using the 20 standard amino acids")
    lengths = {len(s) for s in sequences}
    if len(lengths) != 1 or max(lengths) > 128:
        raise ValueError("This compact example requires one fixed sequence length, at most 128")
    c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
    # With sequence, c only, compute example rewards directly from residue counts.
    # Optional r1/r2 columns override them with supplied labels, such as measurements.
    if {"r1", "r2"}.issubset(rows[0]):
        r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
    else:
        r = composition_proxies(sequences)
    if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
        raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS: retain all L residue positions

    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```


Load data

Freeze ESM-2 and extract its residue embeddings.

Remove beginning/end tokens with `hidden[:, 1:-1]`.

Standardize latent features and each property using training-set statistics.

Return the dataset and saved statistics.

```
# 1. Load annotated sequences and encode frozen ESM-2 residue vectors.
@torch.no_grad()
def load_data(path):
    with Path(path).open(newline='') as handle:
        rows = list(csv.DictReader(handle))
    sequences = [row["sequence"].strip().upper() for row in rows]
    if len(rows) < 4 or any(not s or set(s) - set(AMINO_ACIDS) for s in sequences):
        raise ValueError("Supply at least four sequences using the 20 standard amino acids")
    lengths = {len(s) for s in sequences}
    if len(lengths) != 1 or max(lengths) > 128:
        raise ValueError("This compact example requires one fixed sequence length, at most 128")
    c = torch.tensor([int(row["c"]) for row in rows], dtype=torch.long)
    # With sequence, c only, compute example rewards directly from residue counts.
    # Optional r1/r2 columns override them with supplied labels, such as measurements.
    if {"r1", "r2"}.issubset(rows[0]):
        r = torch.tensor([[float(row["r1"]), float(row["r2"])] for row in rows])
    else:
        r = composition_proxies(sequences)
    if set(c.tolist()) != {0, 1} or not torch.isfinite(r).all():
        raise ValueError("Include both c=0 and c=1, with finite r1 and r2")

    tokenizer = AutoTokenizer.from_pretrained(ESM_NAME, cache_dir=ROOT / ".esm2_cache")
    esm = EsmForMaskedLM.from_pretrained(
        ESM_NAME, cache_dir=ROOT / ".esm2_cache", use_safetensors=True
    ).to(DEVICE).eval().requires_grad_(False)
    encoded = []
    for start in range(0, len(sequences), BATCH_SIZE):
        tokens = tokenizer(sequences[start:start + BATCH_SIZE], return_tensors="pt")
        tokens = {key: value.to(DEVICE) for key, value in tokens.items()}
        hidden = esm.esm(**tokens).last_hidden_state
        encoded.append(hidden[:, 1:-1].cpu()) # Remove BOS/EOS; retain all L residue positions.
    z = torch.cat(encoded) # [N, L, D], with D=320 for this checkpoint.
    z_mean = z.mean((0, 1), keepdim=True)
    z_std = z.std((0, 1), correction=0, keepdim=True).clamp_min(1e-4)
    r_mean, r_std = r.mean(0), r.std(0, correction=0).clamp_min(1e-6)
    dataset = TensorDataset((z - z_mean) / z_std, c, (r - r_mean) / r_std)
    stats = {"z_mean": z_mean, "z_std": z_std, "r_mean": r_mean, "r_std": r_std}
    return dataset, esm, tokenizer, stats
```

To the model class/schedule.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alpha_bars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alpha_bars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alpha_bars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alpha_bars).clamp_min(1e-20)
```

DiffusionModel

self.time and self.condition: construct time and condition features.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16) # Indices 0,1 are classes; 2 is the null class
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alphaBars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alphaBars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alphaBars[:-1]])
posteriorVariances = betas * (1 - previous) / (1 - alphaBars).clamp_min(1e-20)
```

DiffusionModel

`self.time` and `self.condition`: construct time and condition features.

`torch.cat(...)`: combine them with the flattened latent.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16) # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alpha_bars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alpha_bars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alpha_bars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alpha_bars).clamp_min(1e-20)
```

DiffusionModel

`self.time` and `self.condition`: construct time and condition features.

`torch.cat(...)`: combine them with the flattened latent.

`(t * K).round().long()`: recover the discrete diffusion-step index.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alpha_bars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alpha_bars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alpha_bars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alpha_bars).clamp_min(1e-20)
```

DiffusionModel

`self.time` and `self.condition`: construct time and condition features.

`torch.cat(...)`: combine them with the flattened latent.

`(t * K).round().long()`: recover the discrete diffusion-step index.

`alphaBars[k]`: select the cumulative signal coefficient.

The final return combines $\sqrt{(1-\bar{\alpha}_k)}z$ with a learned correction scaled by $\sqrt{\bar{\alpha}_k}$.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alphaBars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alphaBars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alphaBars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alphaBars).clamp_min(1e-20)
```

DiffusionModel

`self.time` and `self.condition`: construct time and condition features.

`torch.cat(...)`: combine them with the flattened latent.

`(t * K).round().long()`: recover the discrete diffusion-step index.

`alphaBars[k]`: select the cumulative signal coefficient.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alphaBars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alphaBars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alphaBars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alphaBars).clamp_min(1e-20)
```


DiffusionModel

`self.time` and `self.condition`: construct time and condition features.

`torch.cat(...)`: combine them with the flattened latent.

`(t * K).round().long()`: recover the discrete diffusion-step index.

`alphaBars[k]`: select the cumulative signal coefficient.

The final return combines $\sqrt{(1-\bar{\alpha}_k)}z$ with a learned correction scaled by $\sqrt{\bar{\alpha}_k}$.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alphaBars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alphaBars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alphaBars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alphaBars).clamp_min(1e-20)
```


DiffusionModel

`self.time` and `self.condition`: construct time and condition features.

`torch.cat(...)`: combine them with the flattened latent.

`(t * K).round().long()`: recover the discrete diffusion-step index.

`alphaBars[k]`: select the cumulative signal coefficient.

The final return combines $\sqrt{(1-\bar{\alpha}_k)}z$ with a learned correction scaled by $\sqrt{\bar{\alpha}_k}$. The output predicts noise at every latent coordinate.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alphaBars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alphaBars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alphaBars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alphaBars).clamp_min(1e-20)
```

RewardModel

Receive the noisy latent and normalized time.

Return two predicted standardized endpoint properties.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alpha_bars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.
```

```
class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.
```

```
# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alpha_bars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alpha_bars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alpha_bars).clamp_min(1e-20)
```

RewardModel

betas: specify per-step noise variances.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alpha_bars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alpha_bars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alpha_bars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alpha_bars).clamp_min(1e-20)
```

RewardModel

betas: specify per-step noise variances.

alphas = 1 - betas: calculate signal-retention coefficients.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alpha_bars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alpha_bars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alpha_bars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alpha_bars).clamp_min(1e-20)
```

RewardModel

betas: specify per-step noise variances.

alphas = 1 - betas: calculate signal-retention coefficients.

cumprod(0): calculate cumulative coefficients $\bar{\alpha}_k$.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alpha_bars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1 - betas
alpha_bars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alpha_bars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alpha_bars).clamp_min(1e-20)
```

RewardModel

betas: specify per-step noise variances.

alphas = 1 - betas: calculate signal-retention coefficients.

cumprod(0): calculate cumulative coefficients $\bar{\alpha}_k$.

previous: align each step with $\bar{\alpha}_{k-1}$.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alpha_bars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alpha_bars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alpha_bars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alpha_bars).clamp_min(1e-20)
```

RewardModel

betas: specify per-step noise variances.

alphas = 1 - betas: calculate signal-retention coefficients.

cumprod(0): calculate cumulative coefficients $\bar{\alpha}_k$.

previous: align each step with $\bar{\alpha}_{k-1}$.

posterior_variances: calculate the variances used during reverse sampling.

```
# 2. Small model classes: flattening lets each output depend on the entire sequence.
class DiffusionModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.length, self.dim = length, dim
        self.time = nn.Sequential(nn.Linear(1, 32), nn.SiLU(), nn.Linear(32, 32))
        self.condition = nn.Embedding(3, 16)  # Indices 0,1 are classes; 2 is the null class.
        self.net = nn.Sequential(
            nn.Linear(length * dim + 48, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, length * dim),
        )

    def forward(self, z, t, c):
        time = self.time(t[:, None])
        inputs = torch.cat([z.flatten(1), time, self.condition(c)], dim=1)
        k = (t * K).round().long().clamp(0, K)
        a = alphaBars[k, None, None]
        return (1 - a).sqrt() * z + a.sqrt() * self.net(inputs).reshape_as(z)
        # Gaussian-reference noise prediction plus a learned correction.
        # The skip carries all coordinates; the correction vanishes near pure noise.

class RewardModel(nn.Module):
    def __init__(self, length, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(length * dim + 1, HIDDEN), nn.SiLU(),
            nn.Linear(HIDDEN, HIDDEN), nn.SiLU(), nn.Linear(HIDDEN, 2),
        )

    def forward(self, z, t):
        return self.net(torch.cat([z.flatten(1), t[:, None]], dim=1))
        # Two predicted standardized endpoint objectives, given the current latent and time.

# The same DDPM schedule as Section 7; k=0 denotes the clean latent.
K = 1000
betas = torch.cat([torch.zeros(1), torch.linspace(1e-4, 0.02, K)]).to(DEVICE)
alphas = 1.0 - betas
alphaBars = alphas.cumprod(0)
previous = torch.cat([torch.ones(1, device=DEVICE), alphaBars[:-1]])
posterior_variances = betas * (1 - previous) / (1 - alphaBars).clamp_min(1e-20)
```


Now the training loop!

```
# 3. DDPM: corrupt  $Z_0$  at step  $k$  and regress the exact noise epsilon.
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = DiffusionModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z0, c, r_tilde in loader:
            z0, c, r_tilde = z0.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            k = torch.randint(1, K + 1, (len(z0),), device=DEVICE)
            t = k.float() / K # Normalized k is only the network's time input.
            a = alpha_bars[k, None, None]
            epsilon = torch.randn_like(z0)
            zk = a.sqrt() * z0 + (1 - a).sqrt() * epsilon
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_diffusion = F.mse_loss(model(zk, t, dropped), epsilon)
            loss_reward = F.mse_loss(reward_model(zk, t), r_tilde)
            loss = loss_diffusion + loss_reward
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False)
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```


Now the training loop!

$k = \text{torch.randint}(\dots)$: sample a diffusion step.

```
# 3. DDPM: corrupt  $Z_0$  at step  $k$  and regress the exact noise epsilon.
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = DiffusionModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z0, c, r_tilde in loader:
            z0, c, r_tilde = z0.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            k = torch.randint(1, K + 1, (len(z0),), device=DEVICE)
            t = k.float() / K # Normalized k is only the network's time input.
            a = alpha_bars[k, None, None]
            epsilon = torch.randn_like(z0)
            zk = a.sqrt() * z0 + (1 - a).sqrt() * epsilon
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_diffusion = F.mse_loss(model(zk, t, dropped), epsilon)
            loss_reward = F.mse_loss(reward_model(zk, t), r_tilde)
            loss = loss_diffusion + loss_reward
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False)
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Now the training loop!

$k = \text{torch.randint}(\dots)$: sample a diffusion step.

$t = k.\text{float}() / K$: normalize the step for the networks.

```
# 3. DDPM: corrupt  $Z_0$  at step  $k$  and regress the exact noise epsilon.
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = DiffusionModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z0, c, r_tilde in loader:
            z0, c, r_tilde = z0.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            k = torch.randint(1, K + 1, (len(z0),), device=DEVICE)
            t = k.float() / K # Normalized k is only the network's time input.
            a = alpha_bars[k, None, None]
            epsilon = torch.randn_like(z0)
            zk = a.sqrt() * z0 + (1 - a).sqrt() * epsilon
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_diffusion = F.mse_loss(model(zk, t, dropped), epsilon)
            loss_reward = F.mse_loss(reward_model(zk, t), r_tilde)
            loss = loss_diffusion + loss_reward
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False)
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Now the training loop!

$k = \text{torch.randint}(\dots)$: sample a diffusion step.

$t = k.\text{float}() / K$: normalize the step for the networks.

$\text{epsilon} = \text{torch.randn_like}(z_0)$:
sample Gaussian noise.

```
# 3. DDPM: corrupt  $Z_0$  at step  $k$  and regress the exact noise epsilon.
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = DiffusionModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z0, c, r_tilde in loader:
            z0, c, r_tilde = z0.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            k = torch.randint(1, K + 1, (len(z0),), device=DEVICE)
            t = k.float() / K # Normalized k is only the network's time input.
            a = alpha_bars[k, None, None]
            epsilon = torch.randn_like(z0)
            zk = a.sqrt() * z0 + (1 - a).sqrt() * epsilon
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_diffusion = F.mse_loss(model(zk, t, dropped), epsilon)
            loss_reward = F.mse_loss(reward_model(zk, t), r_tilde)
            loss = loss_diffusion + loss_reward
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False)
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Now the training loop!

$k = \text{torch.randint}(\dots)$: sample a diffusion step.

$t = k.\text{float}() / K$: normalize the step for the networks.

$\text{epsilon} = \text{torch.randn_like}(z_0)$: sample Gaussian noise.

$z_k = \dots$: combine clean data and noise.

$$Z_k = \sqrt{\bar{\alpha}_k} Z_0 + \sqrt{1 - \bar{\alpha}_k} \varepsilon.$$

```
# 3. DDPM: corrupt  $Z_0$  at step  $k$  and regress the exact noise epsilon.
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = DiffusionModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z0, c, r_tilde in loader:
            z0, c, r_tilde = z0.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            k = torch.randint(1, K + 1, (len(z0),), device=DEVICE)
            t = k.float() / K # Normalized k is only the network's time input.
            a = alpha_bars[k, None, None]
            epsilon = torch.randn_like(z0)
            zk = a.sqrt() * z0 + (1 - a).sqrt() * epsilon
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_diffusion = F.mse_loss(model(zk, t, dropped), epsilon)
            loss_reward = F.mse_loss(reward_model(zk, t), r_tilde)
            loss = loss_diffusion + loss_reward
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False)
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Construct noisy latent Z_k from clean latent Z_0 and Gaussian noise ϵ , with their contributions determined by cumulative signal coefficient $\bar{\alpha}_k$.

Now the training loop!

$k = \text{torch.randint(...)}$: sample a diffusion step.

$t = k.\text{float()} / K$: normalize the step for the networks.

$\text{epsilon} = \text{torch.randn_like}(z_0)$: sample Gaussian noise.

$z_k = \dots$: combine clean data and noise.

$$Z_k = \sqrt{\bar{\alpha}_k} Z_0 + \sqrt{1 - \bar{\alpha}_k} \epsilon.$$

```
# 3. DDPM: corrupt  $Z_0$  at step  $k$  and regress the exact noise epsilon.
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = DiffusionModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z0, c, r_tilde in loader:
            z0, c, r_tilde = z0.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            k = torch.randint(1, K + 1, (len(z0),), device=DEVICE)
            t = k.float() / K # Normalized k is only the network's time input.
            a = alpha_bars[k, None, None]
            epsilon = torch.randn_like(z0)
            zk = a.sqrt() * z0 + (1 - a).sqrt() * epsilon
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_diffusion = F.mse_loss(model(zk, t, dropped), epsilon)
            loss_reward = F.mse_loss(reward_model(zk, t), r_tilde)
            loss = loss_diffusion + loss_reward
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False)
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Now the training loop!

dropped: apply condition
dropout.

```
# 3. DDPM: corrupt  $z_0$  at step  $k$  and regress the exact noise epsilon.
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = DiffusionModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z0, c, r_tilde in loader:
            z0, c, r_tilde = z0.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            k = torch.randint(1, K + 1, (len(z0),), device=DEVICE)
            t = k.float() / K # Normalized k is only the network's time input.
            a = alpha_bars[k, None, None]
            epsilon = torch.randn_like(z0)
            zk = a.sqrt() * z0 + (1 - a).sqrt() * epsilon
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_diffusion = F.mse_loss(model(zk, t, dropped), epsilon)
            loss_reward = F.mse_loss(reward_model(zk, t), r_tilde)
            loss = loss_diffusion + loss_reward
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False)
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Now the training loop!

dropped: apply condition dropout.

loss_diffusion: compare predicted noise with the sampled epsilon.

```
# 3. DDPM: corrupt  $Z_0$  at step  $k$  and regress the exact noise epsilon.
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = DiffusionModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z0, c, r_tilde in loader:
            z0, c, r_tilde = z0.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            k = torch.randint(1, K + 1, (len(z0),), device=DEVICE)
            t = k.float() / K # Normalized k is only the network's time input.
            a = alpha_bars[k, None, None]
            epsilon = torch.randn_like(z0)
            zk = a.sqrt() * z0 + (1 - a).sqrt() * epsilon
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_diffusion = F.mse_loss(model(zk, t, dropped), epsilon)
            loss_reward = F.mse_loss(reward_model(zk, t), r_tilde)
            loss = loss_diffusion + loss_reward
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False)
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```


Now the training loop!

dropped: apply condition dropout.

loss_diffusion: compare predicted noise with the sampled epsilon.

loss_reward: compare predicted properties with r_{tilde} .

```
# 3. DDPM: corrupt  $Z_0$  at step  $k$  and regress the exact noise epsilon.
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = DiffusionModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z0, c, r_tilde in loader:
            z0, c, r_tilde = z0.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            k = torch.randint(1, K + 1, (len(z0),), device=DEVICE)
            t = k.float() / K # Normalized k is only the network's time input.
            a = alpha_bars[k, None, None]
            epsilon = torch.randn_like(z0)
            zk = a.sqrt() * z0 + (1 - a).sqrt() * epsilon
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_diffusion = F.mse_loss(model(zk, t, dropped), epsilon)
            loss_reward = F.mse_loss(reward_model(zk, t), r_tilde)
            loss = loss_diffusion + loss_reward
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False)
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```


Now the training loop!

dropped: apply condition dropout.

loss_diffusion: compare predicted noise with the sampled epsilon.

loss_reward: compare predicted properties with r_{tilde} .

Add the losses, backpropagate, and update both networks.

Freeze their parameters after training.

```
# 3. DDPM: corrupt  $z_0$  at step  $k$  and regress the exact noise epsilon.
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = DiffusionModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z0, c, r_tilde in loader:
            z0, c, r_tilde = z0.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            k = torch.randint(1, K + 1, (len(z0),), device=DEVICE)
            t = k.float() / K # Normalized k is only the network's time input.
            a = alpha_bars[k, None, None]
            epsilon = torch.randn_like(z0)
            zk = a.sqrt() * z0 + (1 - a).sqrt() * epsilon
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_diffusion = F.mse_loss(model(zk, t, dropped), epsilon)
            loss_reward = F.mse_loss(reward_model(zk, t), r_tilde)

            loss = loss_diffusion + loss_reward
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()

        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
    for network in (model, reward_model):
        network.eval().requires_grad_(False)
        for parameter in network.parameters():
            parameter.grad = None
    return model, reward_model
```

Now the training loop!

dropped: apply condition dropout.

loss_diffusion: compare predicted noise with the sampled epsilon.

loss_reward: compare predicted properties with r_{tilde} .

Add the losses, backpropagate, and update both networks.

Freeze their parameters after training.

```
# 3. DDPM: corrupt  $Z_0$  at step  $k$  and regress the exact noise epsilon.
def train(dataset, epochs):
    _, length, dim = dataset.tensors[0].shape
    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
    model = DiffusionModel(length, dim).to(DEVICE)
    reward_model = RewardModel(length, dim).to(DEVICE)
    optimizer = torch.optim.Adam(list(model.parameters()) + list(reward_model.parameters()), lr=LEARNING_RATE)
    for epoch in range(epochs):
        total = 0.0
        for z0, c, r_tilde in loader:
            z0, c, r_tilde = z0.to(DEVICE), c.to(DEVICE), r_tilde.to(DEVICE)
            k = torch.randint(1, K + 1, (len(z0),), device=DEVICE)
            t = k.float() / K # Normalized k is only the network's time input.
            a = alpha_bars[k, None, None]
            epsilon = torch.randn_like(z0)
            zk = a.sqrt() * z0 + (1 - a).sqrt() * epsilon
            dropped = c.masked_fill(torch.rand(len(c), device=DEVICE) < CONDITION_DROP, 2)
            loss_diffusion = F.mse_loss(model(zk, t, dropped), epsilon)
            loss_reward = F.mse_loss(reward_model(zk, t), r_tilde)
            loss = loss_diffusion + loss_reward
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            total += loss.item()
        if (epoch + 1) % 50 == 0 or epoch + 1 == epochs:
            print(f"epoch {epoch+1}: combined training loss {total / len(loader):.4f}")
        for network in (model, reward_model):
            network.eval().requires_grad_(False)
            for parameter in network.parameters():
                parameter.grad = None
    return model, reward_model
```

Reward gradient

```
# 4.  $R_{\lambda} = \sum_m \lambda_m r_{\tilde{m}}$ ; differentiate the current latent only.
def reward_gradient(reward_model, z, t, lambdas):
    with torch.enable_grad():
        # Re-enable input gradients inside sampling.
        state = z.detach().requires_grad_(True)
        R_lambda = (reward_model(state, t) * lambdas).sum(dim=1)
        grad = torch.autograd.grad(R_lambda.sum(), state)[0]
    return grad.detach()
    # Do not retain graphs across sampling steps.

def normalize_weights(lambdas):
    values = torch.as_tensor(lambdas, dtype=torch.float32, device=DEVICE)
    if values.shape != (2,) or not torch.isfinite(values).all() or (values < 0).any() or values.sum() <= 0:
        raise ValueError("Use two finite, nonnegative weights with a positive sum")
    return values / values.sum()
    # lambda controls tradeoffs, eta controls strength.
```

Reward gradient

normalize_weights(...): normalize the objective weights.

```
# 4. R_lambda = sum_m lambda_m * r_tilde_m; differentiate the current latent only.
def reward_gradient(reward_model, z, t, lambdas):
    with torch.enable_grad():
        # Re-enable input gradients inside sampling.
        state = z.detach().requires_grad_(True)
        R_lambda = (reward_model(state, t) * lambdas).sum(dim=1)
        grad = torch.autograd.grad(R_lambda.sum(), state)[0]
    return grad.detach()
    # Do not retain graphs across sampling steps.
```

```
def normalize_weights(lambdas):
    values = torch.as_tensor(lambdas, dtype=torch.float32, device=DEVICE)
    if values.shape != (2,) or not torch.isfinite(values).all() or (values < 0).any() or values.sum() <= 0:
        raise ValueError("Use two finite, nonnegative weights with a positive sum")
    return values / values.sum()
    # lambda controls tradeoffs, eta controls strength.
```

Reward gradient

`reward_model(state, t)`: predict standardized endpoint properties.

The weighted sum defines $R\lambda$.

```
# 4. R_lambda = sum_m lambda_m * r_tilde_m; differentiate the current latent only.
def reward_gradient(reward_model, z, t, lambdas):
    with torch.enable_grad():
        # Re-enable input gradients inside sampling.
        state = z.detach().requires_grad_(True)
        R_lambda = (reward_model(state, t) * lambdas).sum(dim=1)
        grad = torch.autograd.grad(R_lambda.sum(), state)[0]
    return grad.detach()
    # Do not retain graphs across sampling steps.

def normalize_weights(lambdas):
    values = torch.as_tensor(lambdas, dtype=torch.float32, device=DEVICE)
    if values.shape != (2,) or not torch.isfinite(values).all() or (values < 0).any() or values.sum() <= 0:
        raise ValueError("Use two finite, nonnegative weights with a positive sum")
    return values / values.sum()
    # lambda controls tradeoffs, eta controls strength.
```

Reward gradient

`autograd.grad(...)`: calculate its gradient with respect to the current noisy latent.

Use $\lambda = (1, 0)$ for one objective or $\lambda = (0.7, 0.3)$ for both.

```
# 4. R_lambda = sum_m lambda_m * r_tilde_m; differentiate the current latent only.
def reward_gradient(reward_model, z, t, lambdas):
    with torch.enable_grad():
        state = z.detach().requires_grad_(True)
        R_lambda = (reward_model(state, t) * lambdas).sum(dim=1)
        grad = torch.autograd.grad(R_lambda.sum(), state)[0]
    return grad.detach()

def normalize_weights(lambdas):
    values = torch.as_tensor(lambdas, dtype=torch.float32, device=DEVICE)
    if values.shape != (2,) or not torch.isfinite(values).all() or (values < 0).any() or values.sum() <= 0:
        raise ValueError("Use two finite, nonnegative weights with a positive sum")
    return values / values.sum()
```

Let's sample!

```
# 5. Sampling: CFG first; optional reward gradient then changes predicted noise.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0)):
    if c not in (0, 1) or w < 0 or eta < 0:
        raise ValueError("Use c=0/1 and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Same starting and reverse noise across comparisons.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    for k in range(K, 0, -1):
        t = torch.full((n,), k / K, device=DEVICE)
        eps_uncond = model(z, t, null)
        eps = eps_uncond
        if w != 0:
            eps = eps_uncond + w * (model(z, t, condition) - eps_uncond)
        sigma = (1 - alphaBars[k]).sqrt() # FORWARD corruption standard deviation.
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma * grad_R # s_guided=s_theta+eta*grad R; s=-epsilon/sigma.
        mean = (z - betas[k] * eps / sigma) / alphas[k].sqrt()
        z = mean + posterior_variances[k].sqrt() * torch.randn_like(z) if k > 1 else mean
    return z # No fresh noise at k=1.
```


First predict guided noise.

`torch.randn(...)`: initialize the terminal noisy latent.

```
# 5. Sampling: CFG first; optional reward gradient then changes predicted noise.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0)):
    if c not in (0, 1) or w < 0 or eta < 0:
        raise ValueError("Use c=0/1 and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Same starting and reverse noise across comparisons.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 0, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    for k in range(K, 0, -1):
        t = torch.full((n,), k / K, device=DEVICE)
        eps_uncond = model(z, t, null)
        eps = eps_uncond
        if w != 0:
            eps = eps_uncond + w * (model(z, t, condition) - eps_uncond)
        sigma = (1 - alpha_bars[k]).sqrt() # FORWARD corruption standard deviation.
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma * grad_R # s_guided=s_theta+eta*grad R; s=-epsilon/sigma.
        mean = (z - betas[k] * eps / sigma) / alphas[k].sqrt()
        z = mean + posterior_variances[k].sqrt() * torch.randn_like(z) if k > 1 else mean
    return z # No fresh noise at k=1.
```


First predict guided noise.

`torch.randn(...)`: initialize the terminal noisy latent.

`range(K, 0, -1)`: iterate backward through diffusion steps.

```
# 5. Sampling: CFG first; optional reward gradient then changes predicted noise.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0)):
    if c not in (0, 1) or w < 0 or eta < 0:
        raise ValueError("Use c=0/1 and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Same starting and reverse noise across comparisons.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    for k in range(K, 0, -1):
        t = torch.full((n,), k / K, device=DEVICE)
        eps_uncond = model(z, t, null)
        eps = eps_uncond
        if w != 0:
            eps = eps_uncond + w * (model(z, t, condition) - eps_uncond)
        sigma = (1 - alpha_bars[k]).sqrt() # FORWARD corruption standard deviation.
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma * grad_R # s_guided=s_theta+eta*grad R; s=-epsilon/sigma.
        mean = (z - betas[k] * eps / sigma) / alphas[k].sqrt()
        z = mean + posterior_variances[k].sqrt() * torch.randn_like(z) if k > 1 else mean
    return z # No fresh noise at k=1.
```

First predict guided noise.

`torch.randn(...)`: initialize the terminal noisy latent.

`range(K, 0, -1)`: iterate backward through diffusion steps.

`eps_uncond`: predict unconditional noise.

```
# 5. Sampling: CFG first; optional reward gradient then changes predicted noise.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0)):
    if c not in (0, 1) or w < 0 or eta < 0:
        raise ValueError("Use c=0/1 and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Same starting and reverse noise across comparisons.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    for k in range(K, 0, -1):
        t = torch.full((n,), k / K, device=DEVICE)
        eps_uncond = model(z, t, null)
        eps = eps_uncond
        if w != 0:
            eps = eps_uncond + w * (model(z, t, condition) - eps_uncond)
        sigma = (1 - alpha_bars[k]).sqrt() # FORWARD corruption standard deviation.
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma * grad_R # s_guided=s_theta+eta*grad R; s=-epsilon/sigma.
        mean = (z - betas[k] * eps / sigma) / alphas[k].sqrt()
        z = mean + posterior_variances[k].sqrt() * torch.randn_like(z) if k > 1 else mean
    return z # No fresh noise at k=1.
```

First predict guided noise.

`torch.randn(...)`: initialize the terminal noisy latent.

`range(K, 0, -1)`: iterate backward through diffusion steps.

`eps_uncond`: predict unconditional noise.

`eps_uncond + w * (...)`: apply CFG.

```
# 5. Sampling: CFG first; optional reward gradient then changes predicted noise.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0)):
    if c not in (0, 1) or w < 0 or eta < 0:
        raise ValueError("Use c=0/1 and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Same starting and reverse noise across comparisons.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    for k in range(K, 0, -1):
        t = torch.full((n,), k / K, device=DEVICE)
        eps_uncond = model(z, t, null)
        eps = eps_uncond
        if w != 0:
            eps = eps_uncond + w * (model(z, t, condition) - eps_uncond)
        sigma = (1 - alpha_bars[k]).sqrt() # FORWARD corruption standard deviation.
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma * grad_R # s_guided=s_theta+eta*grad R; s=-epsilon/sigma.
        mean = (z - betas[k] * eps / sigma) / alphas[k].sqrt()
        z = mean + posterior_variances[k].sqrt() * torch.randn_like(z) if k > 1 else mean
    return z # No fresh noise at k=1.
```

First predict guided noise.

`torch.randn(...)`: initialize the terminal noisy latent.

`range(K, 0, -1)`: iterate backward through diffusion steps.

`eps_uncond`: predict unconditional noise.

`eps_uncond + w * (...)`: apply CFG.

`sigma`: calculate the forward corruption standard deviation.

```
# 5. Sampling: CFG first; optional reward gradient then changes predicted noise.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0)):
    if c not in (0, 1) or w < 0 or eta < 0:
        raise ValueError("Use c=0/1 and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Same starting and reverse noise across comparisons.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    for k in range(K, 0, -1):
        t = torch.full((n,), k / K, device=DEVICE)
        eps_uncond = model(z, t, null)
        eps = eps_uncond
        if w != 0:
            eps = eps_uncond + w * (model(z, t, condition) - eps_uncond)
        sigma = (1 - alpha_bars[k]).sqrt() # FORWARD corruption standard deviation.
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma * grad_R # s_guided=s_theta+eta*grad R; s=-epsilon/sigma.
        mean = (z - betas[k] * eps / sigma) / alphas[k].sqrt()
        z = mean + posterior_variances[k].sqrt() * torch.randn_like(z) if k > 1 else mean
    return z # No fresh noise at k=1.
```

First predict guided noise.

`torch.randn(...)`: initialize the terminal noisy latent.

`range(K, 0, -1)`: iterate backward through diffusion steps.

`eps_uncond`: predict unconditional noise.

`eps_uncond + w * (...)`: apply CFG.

`sigma`: calculate the forward corruption standard deviation.

`eps = eps - eta * sigma * grad_R`:
apply reward guidance.

```
# 5. Sampling: CFG first; optional reward gradient then changes predicted noise.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0)):
    if c not in (0, 1) or w < 0 or eta < 0:
        raise ValueError("Use c=0/1 and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Same starting and reverse noise across comparisons.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    for k in range(K, 0, -1):
        t = torch.full((n,), k / K, device=DEVICE)
        eps_uncond = model(z, t, null)
        eps = eps_uncond
        if w != 0:
            eps = eps_uncond + w * (model(z, t, condition) - eps_uncond)
        sigma = (1 - alpha_bars[k]).sqrt() # FORWARD corruption standard deviation.
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma * grad_R # s_guided=s_theta+eta*grad R; s=-epsilon/sigma.
        mean = (z - betas[k] * eps / sigma) / alphas[k].sqrt()
        z = mean + posterior_variances[k].sqrt() * torch.randn_like(z) if k > 1 else mean
    return z # No fresh noise at k=1.
```

First predict guided noise.

`torch.randn(...)`: initialize the terminal noisy latent.

`range(K, 0, -1)`: iterate backward through diffusion steps.

`eps_uncond`: predict unconditional noise.

`eps_uncond + w * (...)`: apply CFG.

`sigma`: calculate the forward corruption standard deviation.

`eps = eps - eta * sigma * grad_R`:
apply reward guidance.

```
# 5. Sampling: CFG first; optional reward gradient then changes predicted noise.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0)):
    if c not in (0, 1) or w < 0 or eta < 0:
        raise ValueError("Use c=0/1 and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Same starting and reverse noise across comparisons.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    for k in range(K, 0, -1):
        t = torch.full((n,), k / K, device=DEVICE)
        eps_uncond = model(z, t, null)
        eps = eps_uncond
        if w != 0:
            eps = eps_uncond + w * (model(z, t, condition) - eps_uncond)
        sigma = (1 - alpha_bars[k]).sqrt() # FORWARD corruption standard deviation.
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma * grad_R # s_guided=s_theta+eta*grad R; s=-epsilon/sigma.
        mean = (z - betas[k] * eps / sigma) / alphas[k].sqrt()
        z = mean + posterior_variances[k].sqrt() * torch.randn_like(z) if k > 1 else mean
    return z # No fresh noise at k=1.
```

$$\varepsilon_{\text{guided}} = \varepsilon_{\text{CFG}} - \eta \sqrt{1 - \bar{\alpha}_k} \nabla_z R_\lambda$$

Subtract the reward gradient from the CFG noise prediction, scaled by guidance strength η and forward corruption standard deviation $\sqrt{(1-\bar{\alpha}_k)}$.

First predict guided noise.

`torch.randn(...)`: initialize the terminal noisy latent.

`range(K, 0, -1)`: iterate backward through diffusion steps.

`eps_uncond`: predict unconditional noise.

`eps_uncond + w * (...)`: apply CFG.

`sigma`: calculate the forward corruption standard deviation.

`eps = eps - eta * sigma * grad_R`:
apply reward guidance.

```
# 5. Sampling: CFG first; optional reward gradient then changes predicted noise.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0)):
    if c not in (0, 1) or w < 0 or eta < 0:
        raise ValueError("Use c=0/1 and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Same starting and reverse noise across comparisons.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    for k in range(K, 0, -1):
        t = torch.full((n,), k / K, device=DEVICE)
        eps_uncond = model(z, t, null)
        eps = eps_uncond
        if w != 0:
            eps = eps_uncond + w * (model(z, t, condition) - eps_uncond)
        sigma = (1 - alpha_bars[k]).sqrt() # FORWARD corruption standard deviation.
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma * grad_R # s_guided=s_theta+eta*grad R; s=-epsilon/sigma.
        mean = (z - betas[k] * eps / sigma) / alphas[k].sqrt()
        z = mean + posterior_variances[k].sqrt() * torch.randn_like(z) if k > 1 else mean
    return z # No fresh noise at k=1.
```

$$\varepsilon_{\text{guided}} = \varepsilon_{\text{CFG}} - \eta \sqrt{1 - \bar{\alpha}_k} \nabla_z R_\lambda$$

First predict guided noise.

mean = ...: calculate the DDPM
reverse mean using guided noise.

```
# 5. Sampling: CFG first; optional reward gradient then changes predicted noise.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0)):
    if c not in (0, 1) or w < 0 or eta < 0:
        raise ValueError("Use c=0/1 and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Same starting and reverse noise across comparisons.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    for k in range(K, 0, -1):
        t = torch.full((n,), k / K, device=DEVICE)
        eps_uncond = model(z, t, null)
        eps = eps_uncond
        if w != 0:
            eps = eps_uncond + w * (model(z, t, condition) - eps_uncond)
        sigma = (1 - alpha_bars[k]).sqrt() # FORWARD corruption standard deviation.
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma * grad_R # s_guided=s_theta+eta*grad R: s=-epsilon/sigma.
        mean = (z - betas[k] * eps / sigma) / alphas[k].sqrt()
        z = mean + posterior_variances[k].sqrt() * torch.randn_like(z) if k > 1 else mean
    return z # No fresh noise at k=1.
```


First predict guided noise.

mean = ...: calculate the DDPM
reverse mean using guided noise.

posterior_variances[k].sqrt(): set
the reverse noise amplitude.

torch.randn_like(z): sample the
reverse Gaussian increment.

if k > 1 else mean: finish the last
step without fresh noise.

```
# 5. Sampling: CFG first; optional reward gradient then changes predicted noise.
@torch.no_grad()
def sample(model, reward_model, n=8, c=1, w=0.0, eta=0.0, lambdas=(1.0, 0.0)):
    if c not in (0, 1) or w < 0 or eta < 0:
        raise ValueError("Use c=0/1 and nonnegative w and eta")
    lambdas = normalize_weights(lambdas)
    torch.manual_seed(123) # Same starting and reverse noise across comparisons.
    z = torch.randn(n, model.length, model.dim, device=DEVICE)
    null = torch.full((n,), 2, dtype=torch.long, device=DEVICE)
    condition = torch.full((n,), c, dtype=torch.long, device=DEVICE)
    for k in range(K, 0, -1):
        t = torch.full((n,), k / K, device=DEVICE)
        eps_uncond = model(z, t, null)
        eps = eps_uncond
        if w != 0:
            eps = eps_uncond + w * (model(z, t, condition) - eps_uncond)
        sigma = (1 - alpha_bars[k]).sqrt() # FORWARD corruption standard deviation.
        if eta != 0:
            grad_R = reward_gradient(reward_model, z, t, lambdas)
            eps = eps - eta * sigma * grad_R # s_guided=s_theta+eta*grad R; s=-epsilon/sigma.
        mean = (z - betas[k] * eps / sigma) / alphas[k].sqrt()
        z = mean + posterior_variances[k].sqrt() * torch.randn_like(z) if k > 1 else mean
    return z # No fresh noise at k=1.
```

Standard main()

```
# 7. Run CFG, single-objective steering, and scalarized multi-objective steering in order.
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, default=R00T / "esm2_example.csv")
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--samples", type=int, default=8)
    parser.add_argument("--min-polar", type=int, default=12)
    parser.add_argument("--output", type=Path, default=R00T / "esm2_diffusion_outputs")
    args = parser.parse_args()
    if args.epochs < 1 or args.samples < 1:
        parser.error("epochs and samples must be positive")
    torch.manual_seed(7)
    if DEVICE.type == "cpu":
        torch.set_num_threads(2)
    dataset, esm, tokenizer, stats = load_data(args.data)
    if not 0 <= args.min_polar <= dataset.tensors[0].shape[1]:
        parser.error("min-polar must lie between zero and the sequence length")
    model, reward_model = train(dataset, args.epochs)
    outputs = {
        "cfg": sample(model, reward_model, args.samples, c=1, w=2.0),
        "single": sample(model, reward_model, args.samples, eta=1.0, lambdas=(1.0, 0.0)),
        "multi": sample(model, reward_model, args.samples, eta=1.0, lambdas=(0.7, 0.3)),
    }
    args.output.mkdir(parents=True, exist_ok=True)
    for name, latent in outputs.items():
        if not torch.isfinite(latent).all():
            raise RuntimeError(f"Nonfinite {name} output; reduce guidance or check training")
        sequences = decode(latent, esm, tokenizer, stats, args.min_polar)
        assert all(sum(a in POLAR_RESIDUES for a in s) >= args.min_polar for s in sequences)
        fasta = "".join(f">{name}_{i+1}\n{seq}\n" for i, seq in enumerate(sequences))
        (args.output / f"{name}.fasta").write_text(fasta)
        count = sum(a in POLAR_RESIDUES for a in sequences[0])
        print(f"{name}: {sequences[0]} polar/charged residues={count}")
        print(" decoded mean composition proxies:", composition_proxies(sequences).mean(0).tolist())
    torch.save({"standardized_latents": {k: v.cpu() for k, v in outputs.items()},
              "model": model.state_dict(), "reward_model": reward_model.state_dict(),
              "stats": stats, "esm_name": ESM_NAME,
              "length": model.length, "dim": model.dim, "min_polar": args.min_polar,
              "polar_residues": POLAR_RESIDUES}, args.output / "results.pt")
    print(f"Saved latent tensors and FASTA sequences to {args.output}")

if __name__ == "__main__":
    main()
```

Standard main()

Load the data, train both networks, and produce the same three output batches: CFG, single-objective, and multi-objective generation.

```
# 7. Run CFG, single-objective steering, and scalarized multi-objective steering in order.
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, default=R00T / "esm2_example.csv")
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--samples", type=int, default=8)
    parser.add_argument("--min-polar", type=int, default=12)
    parser.add_argument("--output", type=Path, default=R00T / "esm2_diffusion_outputs")
    args = parser.parse_args()
    if args.epochs < 1 or args.samples < 1:
        parser.error("epochs and samples must be positive")
    torch.manual_seed(7)
    if DEVICE.type == "cpu":
        torch.set_num_threads(2)
    dataset, esm, tokenizer, stats = load_data(args.data)
    if not 0 <= args.min_polar <= dataset.tensors[0].shape[1]:
        parser.error("min-polar must lie between zero and the sequence length")
    model, reward_model = train(dataset, args.epochs)
    outputs = {
        "cfg": sample(model, reward_model, args.samples, c=1, w=2.0),
        "single": sample(model, reward_model, args.samples, eta=1.0, lambdas=(1.0, 0.0)),
        "multi": sample(model, reward_model, args.samples, eta=1.0, lambdas=(0.7, 0.3)),
    }
    args.output.mkdir(parents=True, exist_ok=True)
    for name, latent in outputs.items():
        if not torch.isfinite(latent).all():
            raise RuntimeError(f"Nonfinite {name} output; reduce guidance or check training")
        sequences = decode(latent, esm, tokenizer, stats, args.min_polar)
        assert all(sum(a in POLAR_RESIDUES for a in s) >= args.min_polar for s in sequences)
        fasta = "".join(f">{name}_{i+1}\n{seq}\n" for i, seq in enumerate(sequences))
        (args.output / f"{name}.fasta").write_text(fasta)
        count = sum(a in POLAR_RESIDUES for a in sequences[0])
        print(f"{name}: {sequences[0]} polar/charged residues={count}")
        print(" decoded mean composition proxies:", composition_proxies(sequences).mean(0).tolist())
    torch.save({"standardized_latents": {k: v.cpu() for k, v in outputs.items()},
              "model": model.state_dict(), "reward_model": reward_model.state_dict(),
              "stats": stats, "esm_name": ESM_NAME,
              "length": model.length, "dim": model.dim, "min_polar": args.min_polar,
              "polar_residues": POLAR_RESIDUES}, args.output / "results.pt")
    print(f"Saved latent tensors and FASTA sequences to {args.output}")

if __name__ == "__main__":
    main()
```

Since this is a latent model, we need a decoder.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```

First, recover the amino acids.

$z * z_std + z_mean$: undo latent normalization.

`esm.lm_head(latent)`: calculate vocabulary logits.

`index_select(...)`: retain the 20 canonical amino acids.

`logits.max(...)`: select the highest-scoring residue at each position.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```

First, recover the amino acids.

$z * z_std + z_mean$: undo latent normalization.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```


First, recover the amino acids.

$z * z_std + z_mean$: undo latent normalization.

`esm.lm_head(latent)`:
calculate vocabulary logits.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"] + stats["z_mean"]
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```

First, recover the amino acids.

$z * z_std + z_mean$: undo latent normalization.

`esm.lm_head(latent)`: calculate vocabulary logits.

`index_select(...)`: retain the 20 canonical amino acids.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```


First, recover the amino acids.

$z * z_std + z_mean$: undo latent normalization.

`esm.lm_head(latent)`:
calculate vocabulary logits.

`index_select(...)`: retain the 20 canonical amino acids.

`logits.max(...)`: select the highest-scoring residue at each position.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```

Enforce the count

polar_scores and
polar_choices: find the best
qualifying residue at each
position.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```

Enforce the count

polar_scores and
polar_choices: find the best
qualifying residue at each
position.

already_polar: identify
positions already belonging
to the selected set.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```

Enforce the count

polar_scores and
polar_choices: find the best
qualifying residue at each
position.

already_polar: identify
positions already belonging
to the selected set.

missing: calculate how
many additional qualifying
positions are needed.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```

Enforce the count

cost: calculate the logit loss for each possible substitution.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```

Enforce the count

cost: calculate the logit loss for each possible substitution.

topk(missing, largest=False): select the least costly substitutions.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```


Enforce the count

cost: calculate the logit loss for each possible substitution.

topk(missing, largest=False): select the least costly substitutions.

Update choices and join the residues into sequence strings.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```

Enforce the count

cost: calculate the logit loss for each possible substitution.

topk(missing, largest=False): select the least costly substitutions.

Update choices and join the residues into sequence strings.

```
# 6. One decoder, used only AFTER the flow or diffusion trajectory is complete.
@torch.no_grad()
def decode(z, esm, tokenizer, stats, min_polar=12):
    latent = z * stats["z_std"].to(z.device) + stats["z_mean"].to(z.device)
    logits = esm.lm_head(latent) # Frozen head produces one vocabulary distribution per residue.
    aa_ids = torch.tensor(tokenizer.convert_tokens_to_ids(list(AMINO_ACIDS)), device=z.device)
    logits = logits.index_select(-1, aa_ids) # Restrict the vocabulary to the 20 amino acids.
    if not 0 <= min_polar <= z.shape[1]:
        raise ValueError("min_polar must lie between zero and the sequence length")
    polar = torch.tensor([a in POLAR_RESIDUES for a in AMINO_ACIDS], device=z.device)
    polar_ids = polar.nonzero().flatten()
    best_scores, choices = logits.max(dim=-1) # Start from unrestricted amino-acid argmax.
    polar_scores, local = logits[..., polar_ids].max(dim=-1)
    polar_choices = polar_ids[local] # Best polar/charged amino acid at each position.
    for i in range(len(z)):
        already_polar = polar[choices[i]]
        missing = max(0, min_polar - int(already_polar.sum()))
        if missing:
            cost = (best_scores[i] - polar_scores[i]).masked_fill(already_polar, float("inf"))
            positions = cost.topk(missing, largest=False).indices
            choices[i, positions] = polar_choices[i, positions]
    return ["".join(AMINO_ACIDS[i] for i in row) for row in choices.cpu().tolist()]
# Exact maximum-logit decode subject to at least min_polar selected residues.
```


In main()

The final loop saves the sequences as FASTA files and stores the latents, model parameters, and normalization statistics in results.pt.

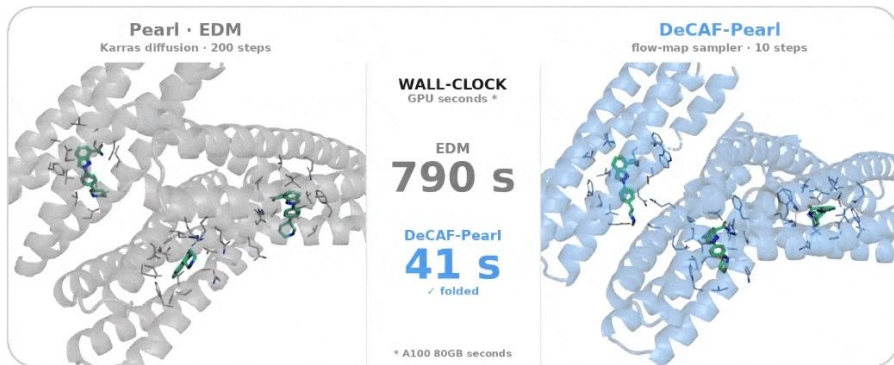
```
# 7. Run CFG, single-objective steering, and scalarized multi-objective steering in order.
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data", type=Path, default=R00T / "esm2_example.csv")
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--samples", type=int, default=8)
    parser.add_argument("--min-polar", type=int, default=12)
    parser.add_argument("--output", type=Path, default=R00T / "esm2_diffusion_outputs")
    args = parser.parse_args()
    if args.epochs < 1 or args.samples < 1:
        parser.error("epochs and samples must be positive")
    torch.manual_seed(7)
    if DEVICE.type == "cpu":
        torch.set_num_threads(2)
    dataset, esm, tokenizer, stats = load_data(args.data)
    if not 0 <= args.min_polar <= dataset.tensors[0].shape[1]:
        parser.error("min-polar must lie between zero and the sequence length")
    model, reward_model = train(dataset, args.epochs)
    outputs = {
        "cfg": sample(model, reward_model, args.samples, c=1, w=2.0),
        "single": sample(model, reward_model, args.samples, eta=1.0, lambdas=(1.0, 0.0)),
        "multi": sample(model, reward_model, args.samples, eta=1.0, lambdas=(0.7, 0.3)),
    }
    args.output.mkdir(parents=True, exist_ok=True)
    for name, latent in outputs.items():
        if not torch.isfinite(latent).all():
            raise RuntimeError(f"Nonfinite {name} output; reduce guidance or check training")
        sequences = decode(latent, esm, tokenizer, stats, args.min_polar)
        assert all(sum(a in POLAR_RESIDUES for a in s) >= args.min_polar for s in sequences)
        fasta = "".join(f">{name}_{i+1}\n{seq}\n" for i, seq in enumerate(sequences))
        (args.output / f"{name}.fasta").write_text(fasta)
        count = sum(a in POLAR_RESIDUES for a in sequences[0])
        print(f"{name}: {sequences[0]} polar/charged residues={count}")
        print(" decoded mean composition proxies:", composition_proxies(sequences).mean(0).tolist())
    torch.save({"standardized_latents": {k: v.cpu() for k, v in outputs.items()},
               "model": model.state_dict(), "reward_model": reward_model.state_dict(),
               "stats": stats, "esm_name": ESM_NAME,
               "length": model.length, "dim": model.dim, "min_polar": args.min_polar,
               "polar_residues": POLAR_RESIDUES}, args.output / "results.pt")
    print(f"Saved latent tensors and FASTA sequences to {args.output}")

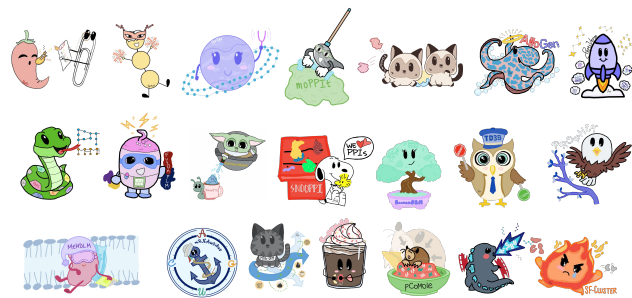
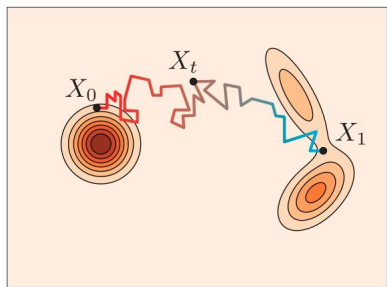
if __name__ == "__main__":
    main()
```

And that's continuous generative models!

And that's continuous generative models!

We'll cover additional special new methods later after going through core discrete frameworks (flow maps, drifting), but it's essentially just diffusion/flow matching.





Now, we are ready to enter the world of
Discrete Generative Models!

